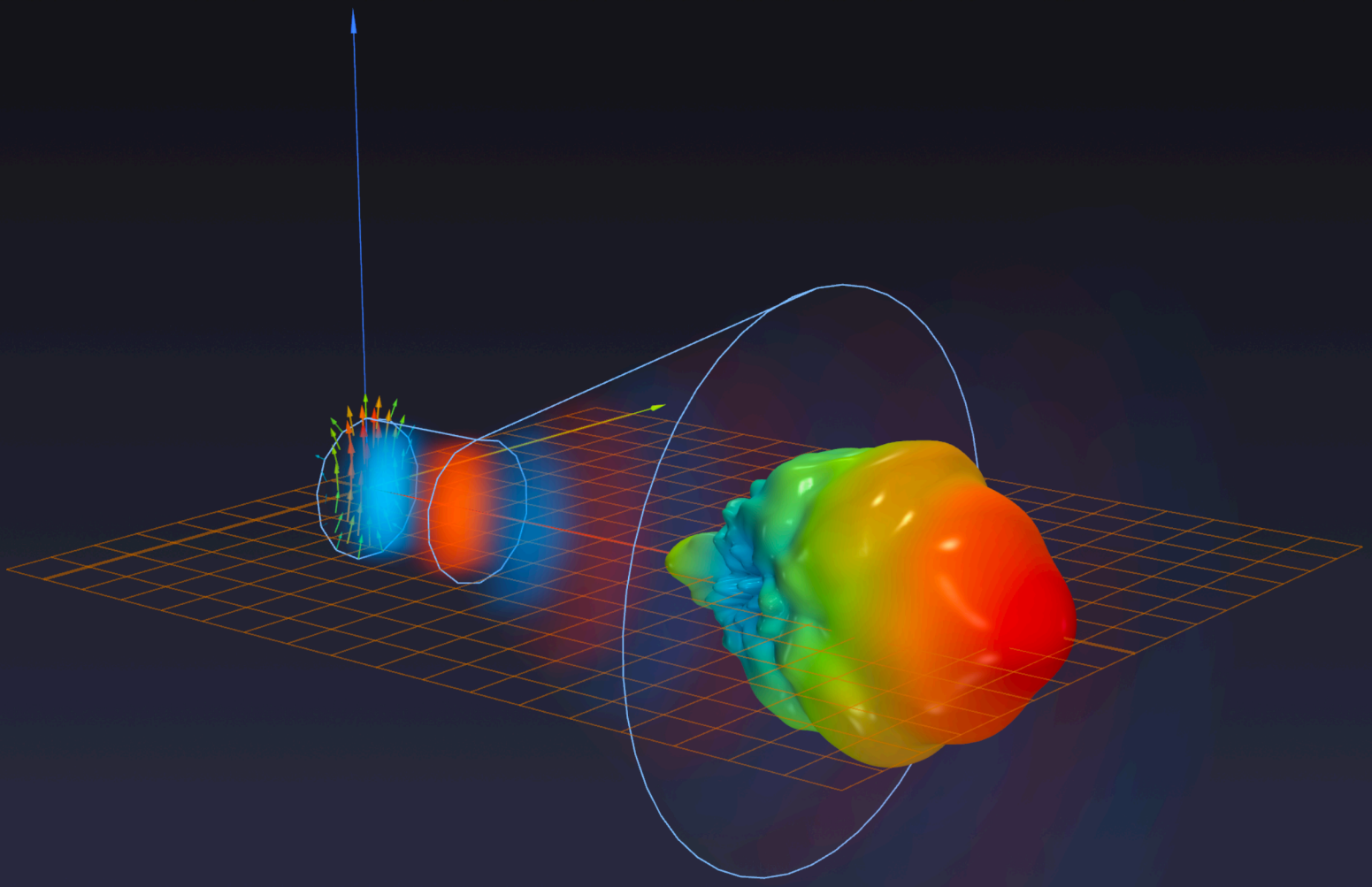
EMerge Software



EMerge User Manual Version 2.8

2026-07-30

! IMPORTANT !

This manual (Version 2.8 .x) accompanies the version 2.8 release of the FEM simulation software.

Please note that both the software and this documentation are still a work in progress. Certain features, explanations, or details may be incomplete or subject to change in future updates. If you encounter issues, errors, or missing information, we encourage you to reach out through one of the following channels:

- GitHub (issues/discussions): https://github.com/FennisRobert/EMerge
- Website: https://emerge-software.com
- Email: info@emerge-software.com

Your feedback is highly valued and will help improve both the software and its documentation.

Copyright © 2025 Robert Fennis. All rights reserved.

This manual and the EMerge software it documents are the intellectual property of Robert Fennis. Permission is hereby granted, free of charge, to any person obtaining a copy of this manual and associated software, to use, copy, modify, merge, publish, and distribute copies of the materials, subject to the following conditions:

Redistributions of this manual or modified versions must retain the above copyright notice and this permission notice. The manual and software are provided “as is”, without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose, or non-infringement. In no event shall the authors or copyright holders be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the manual, the software, or the use or other dealings in the materials. EMerge is an evolving software project. Contributions, feedback, and suggestions are welcome.

For licensing details regarding EMerge software, please refer to the LICENSE file distributed with the source code.

Contents

1 Introduction 8
1.1 What is EMerge? 8
1.2 EMerge Architectural Guardrails 9
1.3 How to read this manual 9
2 Installation 11
2.1 Installing EMerge 11
2.1.1 Windows 10 / 11 11
2.2 macOS (Apple Silicon or Intel) 12
2.2.1 Installing EMerge 12
2.2.2 Installing Numpy + Scipy with OpenBLAS (x86 ONLY) 12
2.3 Linux (Ubuntu, Fedora, etc.) 13
2.4 Installing Additional Direct Solvers 14
2.5 Which solver should I pick? 14
2.5.1 UMFPACK 14
2.5.1.1 Windows 14
2.5.1.2 MacOS (ARM and x86) 14
2.5.1.3 Linux 15
2.5.2 CuDSS 15
2.5.3 Apple Accelerate Solvers 15
2.5.3.1 Installation versioning 15
2.5.3.2 Running Accelerate in parallel 15
2.5.4 MUMPS 15
2.5.5 Cholesky through Scikit-Sparse 16
2.6 EMerge IRON 17
3 Quick Start 18
3.1 First simulation 18
3.2 Resonant Cavity 20
3.3 Step Plan 21
3.4 EMerge Behavior 22
3.5 Warning Report System 22
4 Concepts 25
4.1 The Simulation object 25
4.1.1 Loglevel 25
4.1.2 Saving and Loading 25
4.2 Settings 26
4.3 Constructive Geometry 28
4.3.1 Basic shapes 28
4.3.2 Axes, Planes and Coordinate Systems 28
4.3.3 Boolean operations 29
4.3.4 Transformations 30
4.3.5 Under the hood 30
4.3.6 Advanced geometries 31
4.4 Boundary selection 31
4.4.1 Under the hood 31
4.4.2 Passing objects 31
4.4.3 Creating selections 32
4.4.4 Object faces 32
4.4.4.1 Object face name definitions 34
4.4.5 Selecting multiple objects 34
4.4.6 Visual selection 35
4.5 Anchor points 36
4.6 Geometry Import 40

4.6.1 Step files ... 40
4.6.1.1 Adding ports from PCB files ... 41
4.6.2 Gerber files (Beta) ... 41
4.7 Materials ... 43
4.7.1 Overlapping geometries and Priority ... 43
4.7.2 Custom Materials ... 44
4.7.2.1 Thermal properties ... 44
4.7.3 Material library ... 44
4.7.4 Assigning materials to surfaces ... 45
4.7.5 Appearance ... 45
4.7.6 Drude model ... 45
4.8 Mesh generation ... 46
4.8.1 Mesh specification function ... 46
4.8.2 Trace meshing ... 46
4.8.2.1 Trace Refinement ... 47
4.9 Boundary Conditions ... 48
4.10 Running Simulations ... 49
4.10.1 Selecting a solver ... 49
4.11 Saving and Loading ... 49
4.11.1 Caching simulations ... 50
4.11.2 Solvers and Solve Routines ... 50
4.12 Simulation data ... 52
4.12.1 The SimulationData object ... 52
4.12.1.1 The .sim field ... 53
4.12.1.2 The .mw field ... 54
4.12.1.3 Selecting data by number ... 54
4.12.1.4 Selecting by parameter ... 54
4.12.2 Parameter Sweeps ... 55
5 Microwave Physics ... 58
5.1 Boundary Conditions ... 58
5.1.1 Ports ... 58
5.1.2 Modal port ... 58
5.1.2.1 Aligning modes ... 60
5.1.2.2 Port Impedance ... 60
5.1.3 Lumped port ... 61
5.1.4 User defined Ports ... 61
5.1.5 Differential ports ... 62
5.1.6 Multi-mode sweep ... 62
5.1.6.1 Manual mode settings ... 62
5.1.7 Surface Impedance and Thin Conductor ... 63
5.2 Open/Radiation Boundaries ... 64
5.2.1 PML Regions ... 64
5.2.2 Automatic open regions ... 65
5.3 Scattered Field Simulations ... 66
5.3.1 Geometry requirements ... 66
5.3.2 Boundary Condition ... 66
5.3.2.1 Radius setting ... 67
5.3.3 Running the simulation ... 67
5.3.4 Post processing ... 67
5.3.4.1 Relative fields ... 68
6 Heat Conduction Physics ... 69
6.1 Activating heat conduction physics ... 69
6.2 Stationary, Transient and Non-Linear solvers ... 69
6.3 Boundary conditions ... 70

6.3.1 Fixed Temperature (2D/3D) ... 70
6.3.2 Heat Flux (2D/3D) ... 70
6.3.3 Initial temperature (2D/3D) ... 70
6.3.4 Thermal Contact Resistance (2D) ... 71
6.3.5 Thin Conductor (2D) ... 71
6.3.6 Convection (2D) ... 71
6.3.7 Black body radiation (2D) ... 72
6.4 Thermal Solvers ... 72
6.4.1 Non-linear solver ... 72
6.4.2 Transient-solver ... 72
6.5 Post processing ... 73
6.6 Coupled electromagnetic/heat conduction ... 73
7 Post Processing ... 75
7.1 Data extraction ... 75
7.2 Visualization ... 76
7.2.1 3D Plots ... 76
7.2.2 Key-bindings ... 77
7.2.2.1 `.view()` arguments ... 77
7.2.3 Display color Themes ... 78
7.2.4 Manual 3D plots ... 80
7.2.5 Plotting syntax ... 81
7.2.6 3D Plot functions ... 82
7.2.7 Data generation ... 82
7.3 Result Analysis ... 84
7.3.1 Far-field calculation ... 84
7.3.1.1 EHDataFF class ... 84
7.3.1.2 3D Far-field plot ... 85
7.3.1.3 Symmetry planes ... 85
7.3.1.4 Spherical axis definitions ... 86
7.4 Exporting Data ... 87
7.4.1 Touchstone files .sNp ... 87
7.4.2 Step files ... 87
7.4.3 VTK files (Beta) ... 87
7.4.4 DXF Files (Beta) ... 87
7.4.5 FarField data ASCII ... 88
8 Advanced Use ... 90
8.1 Auxilliary features ... 90
8.1.1 Smartphone Notifications ... 90
8.2 Design Optimization ... 91
8.2.1 Adding optimization parameters ... 91
8.2.2 Minimization vs Maximization ... 91
8.2.3 Optimization method ... 91
8.2.4 Running the optimization ... 91
8.2.5 Optimization data ... 92
8.3 Adaptive Mesh Refinement ... 93
8.4 Symmetry planes ... 95
8.5 PCB Design ... 96
8.5.1 The basics ... 96
8.5.2 Trace thickness and Trace Materials ... 97
8.5.3 Compiling paths ... 98
8.5.4 Volumes, dielectrics and ports ... 98
8.5.4.1 Generating the PCB stack ... 98
8.5.4.2 Generating an airbox ... 98
8.5.4.3 Ground planes ... 98

8.5.5 Ports ... 99
8.5.6 Vias ... 99
8.5.7 Layers ... 100
8.5.8 Other construtors ... 100
8.5.9 Calculators ... 100
8.6 Periodic Cells ... 101
8.7 Polygons, Extrusion, Revolutions and Pipes ... 102
8.7.1 Pipes ... 103
8.7.2 Connecting Faces ... 104
8.8 Alternative basis functions ... 105
8.8.1 Order and completeness ... 105
8.8.2 Basis function sets ... 105
8.8.3 Predefined Sets ... 106
8.8.4 When to choose which one ... 106
9 Extending EMerge ... 108
9.1 Custom geometry ... 108
9.1.1 Boolean operations background information ... 109
9.1.2 Face assignment ... 110
9.2 Custom Solvers ... 112
9.2.1 Solver Methods ... 113
9.3 Boundary conditions ... 114
9.3.1 Ports ... 114
10 Performance and Optimization ... 116
10.1 Parallel Computing ... 116
10.1.1 Matrix assembly ... 116
10.1.2 Post processing ... 116
10.1.3 Matrix Factorization ... 117
10.1.4 Frequency Sweeps ... 117
10.1.5 How to set it up ... 117
10.1.5.1 Multi-threading ... 117
10.1.5.2 Multi-processing ... 117
10.1.5.3 Parallel solve of single frequencies ... 118
10.1.6 Choosing the right solver ... 118
10.1.7 RAM Management ... 122
10.1.7.1 Harddrive caching ... 123
11 Troubleshooting and FAQ ... 124
11.1 Common Issues and Solutions ... 125
11.1.1 EMerge debug messages ... 125
11.1.2 Common issues ... 125
11.2 Frequency Asked Questions (FAQ) ... 126
Index ... 127
A. Classes and functions ... 129
A.1. PCB methods ... 129
A.1.1. Stripline segments ... 129
A.1.2. Straight sections ... 129
A.1.3. Turn ... 129
A.1.4. PTurn ... 129
A.1.5. Jump ... 130
A.1.6. Curve ... 130
A.1.7. To ... 130
A.1.8. Split and merge ... 130
A.1.9. Cut ... 131

1 Introduction

Welcome to the EMerge user manual. This document is intended to guide you through the process of creating and running electromagnetic simulations using the EMerge framework. Whether you are an experienced developer working in Python or an engineer exploring computational electromagnetics, EMerge offers a versatile and powerful environment for your work.

EMerge is designed to be used in any editor or integrated development environment (IDE) of your choice. While a simple text editor like Notepad suffices, we strongly recommend an IDE that supports features like auto-completion and integrated documentation. EMerge continues to evolve, with expanding capabilities and increasingly comprehensive documentation.

This manual will help you get started with your first simulation and introduce you to the structure of the EMerge modules. You will learn how to define geometries, assign materials, apply boundary conditions, and analyze simulation results. Advanced topics such as constructive geometry, PCB design, and handling simulation datasets will also be covered in detail.

1.1 What is EMerge?

EMerge is a completely free Python based Finite Element Method solver written completely in Python with limited external libraries.

EMerges main philosophy is a streamlined user experience with as little boilerplate code as possible. This manual will cover basic to advanced features. For more information on how to build specific things you may go to the `/r/EMergeSoftware` subreddit, reach out personally through the contact information on my website or join the Discord server.

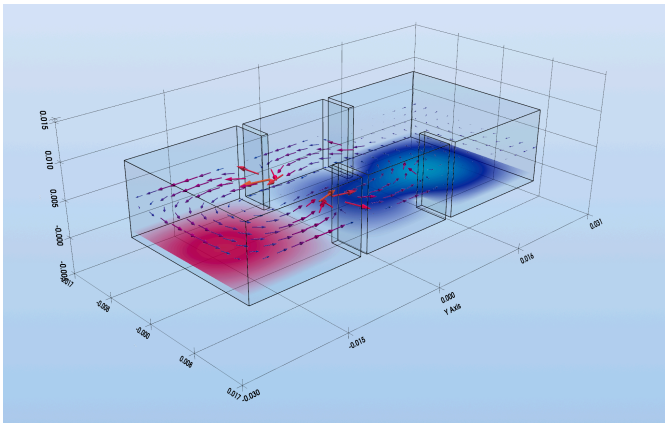


Figure 1: EH Fields inside a waveguide

1.2 EMerge Architectural Guardrails

Some FEM solver behavior is maybe desired but not always something that fits in EMerge's default operational guardrails. There are some behavioral constraints that I believe should not be restricted which "may" impact how you expect EMerge to behave. Here I summarize the important ones:

- **No internet connection** - EMerge itself will never connect to the internet by default without explicit manual task. Only the `.ping()` function will push messages to the NTFY app in **public** channels. EMerge never calls these functions without you knowing.
- **No harddrive access besides compilation** - EMerge never by default writes any files to the harddrive with exception of Numba which compiles Python functions to code which will be cached as binaries on the harddrive. EMerge **can** save various things to the harddrive but always only if the user instructs it.
- **No private information sharing** - This is of course clear from the first. EMerge will not share any private information over the internet or anywhere. Only if you write a log file and manually send that to me or online, will some information about your system be shared by you itself.

1.3 How to read this manual

This manual is written with the primary goal of informing the user how EMerge should/could be used and what features are implemented. The goal is not to give an in depth explanation of each function/feature available and how they should be used.

Every function in EMerge intended to be used by the user is documented. Modern code editors like VSCode and PyCharm will show you details on each functions implementation when you hover your mouse over the function.

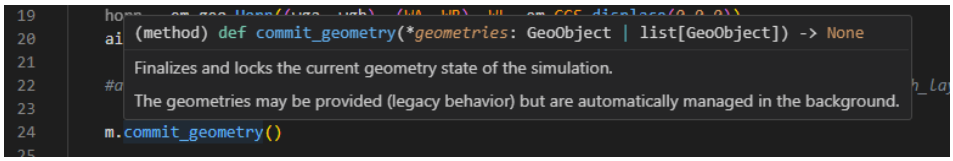


Figure 2: A demonstration of documentation being shown when hovering your mouse over a function.

While not always consistently done, some functions **not intended to be used by users** will start with an underscore like this `._some_function()`. These functions are referred to as **private** methods. In other programming languages, the compiler actively blocks the user from calling these methods outside of their scope. In Python this is not enforced. If you see a property of function starting with an underscore, take this as a message that **you should not call this function or change this attribute!**.

Beyond this, the manual speaks for itself, it is a manual. But in order to make the reading experience smoother, it is helpful to be aware of the following assumptions/rules in the background. Especially for less-experienced Python programmers.

1. The `my_name` convention

Often in this manual, we are forced to refer to some pre-existing objects. For example, if we want to show you how you can manipulate a box, we have to refer to some box you might have created before. Thus, whenever you see the name of a variable `my_box` or `my_cs` with the term `my_` in front of it, this could be any name of your choice. Its just a placeholder.

2. Repeated examples

Sometimes, EMerge has multiple ways of programming the same thing. It might happen therefore that you see examples of two lines apparently doing the exact same thing. This is intended.

3. Placeholder dots

Sometimes certain parts of the code lines are left blank because they are irrelevant to what is shown. In this case, you may see a tripple dot like so:

```python
my_object = em.some_function(...)
```

You may interpret this as: “something you have to fill in here but it is irrelevant for now.”.

The triple dots are **not Python code!**

4. List comprehensions

A confusing notation for beginner Python programmers is the asterisk star notation you sometimes will see. This can be `function(*variable)` or `function(**variable)`. This is called “list comprehension” in Python and it has the following behavior.

For the single asterisk. This:

```python
my_list = (1.2, 'some text', my_box)
some_function(my_list[0], my_list[1], my_list[2])
```

Is the same as

```python
my_list = (1.2, 'some text', my_box)
some_function(*my_list)
```

Python just unpacks the items in the list as a sequence of arguments.

For the double asterisk. This:

```python
my_dict = {"var_1": 1.0, "var_2": "some_string"}
some_function(var_1=my_dict["var_1"], var_2=my_dict["var_2"])
```

is the same as:

```python
my_dict = {"var_1": 1.0, "var_2": "some_string"}
some_function(**my_dict)
```

Python automatically uses the keys of the dictionary as key-word argument in the function call.

2 Installation

This section will discuss the necessary steps to install EMerge on your system. For information on how to install the high performant UMFPACK solver, please consult the separate UMFPACK installation manual from the EMerge website:

https://www.emerge-software.com/resources

2.1 Installing EMerge

2.1.1 Windows 10 / 11

First make sure you have Python installed. It is **not** recommended to use the Python from the Microsoft Store. This will most likely make it impossible for EMerge to compile crucial code.

For python version management it is advised to install Python and manage libraries using Conda.

https://anaconda.org/anaconda/conda

Libraries like Numpy, Scipy and others will be installed with the best and most efficient linear algebra backends like OpenBLAS etc. The next steps will show how to install EMerge using whatever Python you have available. For more information on how to manage Python using conda, please read their website.

1. **Open PowerShell**

   Press **Win + X** → choose **Windows PowerShell** (or **Terminal** on Windows 11).

2. **(Optional) Create & activate a virtual environment**

```powershell
py -3.11 -m venv .venv
.\.venv\Scripts\Activate
```

3. **Install**

```powershell
py -m pip install emerge
```

2.2 macOS (Apple Silicon or Intel)

Installing EMerge's default installation is very straight forward, just a simple installation through Python's package manager. Some additional functionality can be added to improve the performance of EMerge. For users with the new Apple Silicon chips (M1, M2, M3, M4 and M5 etc). it is **not recommended** to use conda. In fact, the usual distributions in PyPI are known to be most stable.

2.2.1 Installing EMerge

1. **Open Terminal**

   Press **⌘ + Space**, type **Terminal**, press **Return**.

2. **(Optional) Create & activate a virtual environment with the tool of your choice**

```bash
python -m venv .venv
source .venv/bin/activate
```

or using pyenv, uv or another tool.

3. **Standard installation**

The simplest way to install EMerge is by simply installing it through Pythons package manager.

```bash
pip install emerge
```

2.2.2 Installing Numpy + Scipy with OpenBLAS (x86 ONLY)

EMerge sees up to 25% improved performance on Intel Apple products with OpenBLAS compiled libraries. Do not use this on Apple Silicon. You can install these by uninstalling your existing installation and installing these wheels.

```bash
pip uninstall numpy
pip uninstall scipy
pip install --pre -i https://pypi.anaconda.org/scipy-wheels-nightly/simple scipy
# optional if not installed with scipy
pip install --pre -i https://pypi.anaconda.org/scipy-wheels-nightly/simple numpy
```

2.3 Linux (Ubuntu, Fedora, etc.)

1. **Open a terminal**

   Ubuntu**:** Ctrl + Alt + T **Fedora / others**: look for **Terminal** in your applications menu.

2. **(Optional) Create & activate a virtual environment**

```bash
python3 -m venv .venv
source .venv/bin/activate
```

3. **Install**

```bash
pip install emerge
```

4. **(Optional) Install a visualization back-end**

Some distributions of Linux do not come with a UI back end for Matplotlib. In this case you might have to install a back-end of choice.

```bash
sudo apt-get install python3-tk
```

2.4 Installing Additional Direct Solvers

Direct solvers are the core of solving the EM problems that are assembled in EMerge. Better direct solvers means faster performance.

EMerge contains a SuperLU (through Scipy) and PARDISO (through Intels MKL library) solvers by default. There are other sparse direct solvers available that perform very well.

Internally EMerge's solver backend has solver interfaces for all the different direct solvers that exist. However, dependencies are needed to actually run them. If these are installed in the Python environment EMerge will automatically detect them and take them into consideration.

Important

Because many of the solve interfaces need compiled binaries to run, installation is sometimes difficult. Many of the Python modules that offer interfaces to these solvers don't ship with the compiled binaires to actually run them. This may make installation difficult. If you experience any difficulties, feel free to reach out.

2.5 Which solver should I pick?

The best solver depends on whether you run on Windows, Linux or MacOS. The selection depends on whether you want multiple cores to work on the same frequency point (parallel solve) or if you want to dedicate one solver per core (Sequential). You can consult the following suggestions for the best solvers currently:

- **Windows 10 and Windows 11**:
  - Sequential solver: **UMFPACK**
  - Parallel solver: **PARDISO**
- **MacOS (ARM)**
  - Sequential solver: **Apple Accelerate**
  - Parallel solver: **Apple Accelerate**
- **MacOS (x86)**
  - Sequential solver: **Apple Accelerate(untested)** otherwise **UMFPACK**
  - Parallel solver: **PARDISO**
- **Linux (x86)**:
  - Sequential solver: **UMFPACK**
  - Parallel solver: **PARDIDSO**

2.5.1 UMFPACK

UMFPACK is a very powerful single threaded sparse direct solver. Depending on the operating system, installation can sometimes be a bit cumbersome.

2.5.1.1 Windows

Installation of UMFPACK is by far the simplest with conda.

bash

```
conda install conda-forge::scikit-umfpack
```

2.5.1.2 MacOS (ARM and x86)

Installing for MacOS can be a bit more tricky but with homebrew it goes quite well. Remember that the Accelerate solver is strictly faster than UMFPACK.

bash

```
brew install cmake swig suite-sparse
export PKG_CONFIG_PATH="/opt/homebrew/lib/pkgconfig:$PKG_CONFIG_PATH"
export CFLAGS="-I/opt/homebrew/include"
export LDFLAGS="-L/opt/homebrew/lib"
export CFLAGS="-Wno-error=int-conversion"
pip install meson-python ninja
pip install --no-build-isolation --no-binary=scikit-umfpack scikit-umfpack
```

2.5.1.3 Linux

For Linux, the exact steps are less clear because I do not have access to Linux machines to test the required steps. You can try the following.

```bash
sudo apt-get install libsuitesparse-dev
pip install meson-python ninja
pip install --no-build-isolation --no-binary=scikit-umfpack scikit-umfpack
```

If you do not manage to install it this way, you can use LLM's like ChatGPT, Gemini and Claude to help you out installing the Python module `scikit-umfpack`. If you succeed, please provide the required steps so that I can update the manual.

2.5.2 CuDSS

CuDSS is a powerful GPU based sparse direct solver for NVidia GPU's. It only works on machines with modern GPUs but it is limited to 32bit matrix indices so it will have issues with very large problems. It works on all machines with NVidia GPU's.

```powershell
py -m pip install emerge[cudss]
```

! WARNING !

Be very careful with the available VRAM on your machine. CuDSS will not crash but give bad results if it runs out of memory.

2.5.3 Apple Accelerate Solvers

Apple includes a very good optimized sparse direct solver for their Apple Silicon ARM chips (M1 to M5). Since 2.3 a new interface exists that should work well. Accelerate can run both in sequential and parallel mode. It can very simply be installed by doing this

```powershell
pip install git+https://github.com/FennisRobert/emerge-aasds
```

2.5.3.1 Installation versioning

Due to some API changes, older emerge version before 2.5.3 need the older interface. If you get errors please install the following:

```powershell
pip install git+https://github.com/FennisRobert/emerge-aasds@v0.2.0
```

2.5.3.2 Running Accelerate in parallel

To enable parallel solve of a single frequency problem (So not single core per frequency) you can do either of the following (depending on the version number):

```python
# Top of script
import os
os.environ("VECLIB_MAXIMUM_THREADS") = "4"
# Only now import EMerge
import emerge as em
```

Or in later versions:

```python
import emerge as em
from emerge_config import config
config.set_acc_threads(4)
```

2.5.4 MUMPS

MUMPS is a good parallel solver alternative for PARDISO as PARDISO is x86 only and thus not supported on Apple machines with ARM chips. However, Accelerate will be easier to install and faster than MUMPS. The installation only work on MacOS machines with ARM chips.

To install the mumps parallel solver, go to the resources page and download the MUMPS installation kit. This works only on MacOS.

https://www.emerge-software.com/resources

Then follow the instructions inside the installer.

2.5.5 Cholesky through Scikit-Sparse

While not strictly necessary, thermal problems can be solved slightly faster with Cholesky decomposition. A cholesky solver can be found in `scikit-sparse` which can be installed through:

```bash
pip install scikit-sparse
```

Besides installing a cholesky solver it will also add a METIS pre-ordering algorithm which will be automatically invoked in SuperLU (which does not have it yet).

2.6 EMerge IRON

! WARNING !
Emerge IRON is no longer used. All optimized algorithms are now natively included with Numba compiled code.

Depricated:
EMerge IRON is a new Rust library used to accelerate parts of the crucial code. Currently EMerge IRON only performs field interpolation but up to 100x faster than EMerge. Installation instructions can be found on GitHub. Currently you need to compile it locally with a Rust compiler. If you have a Rust compiler you can do:

```bash
pip install git+https://github.com/FennisRobert/EMerge-IRON
```

https://github.com/FennisRobert/EMerge-IRON

In the future, the METIS preordering algorithm will be added for solvers like SuperLU to gain a 2x speed boost. However, this has less priority as UMFPACK, PARDISO and Apple Accelerate solvers all have a METIS sorter build in.

3 Quick Start

3.1 First simulation

Now we will start by setting up your first simulation. You can setup a simulation in any editor: notepad, Nodepad++, VSCode, PyCharm, vim, Spyder, ... It is advised to use an IDE that support auto completion and documentation. EMerge is well-documented and continues to become even more so.

You may **copy/paste** the following code block to setup a simple simulation of a WR90 waveguide.

```python
import emerge as em

mm = 0.001 # Define a millimeter
wg_width = 22.86*mm # Width of WR90 waveguide
wg_height = 10.16*mm # Height of WR90 waveguide
wg_length = 50*mm # Arbitrary length

my_model = em.Simulation('MyFirstModel')

box = em.geo.Box(wg_width, wg_length, wg_height)

my_model.commit_geometry()

my_model.view()

my_model.mw.set_frequency_range(8e9, 10e9, 21)

my_model.generate_mesh()

my_model.view(plot_mesh=True)

port1 = my_model.mw.bc.RectangularWaveguide(box.front, 1)
port2 = my_model.mw.bc.RectangularWaveguide(box.back, 2)

fdata = my_model.mw.run_sweep()

freqs = fdata.scalar.grid.freq
S11 = fdata.scalar.grid.S(1,1)
S21 = fdata.scalar.grid.S(2,1)

from emerge.plot import plot_sp

plot_sp(freqs, [S11, S21], labels=['S11','S21'], dblim=[-60, 3])
```

Now let us look in a bit more detail what exactly is going on in each step.

python

```python
import emerge as em
```
> Loads the EMerge module in the namespace em

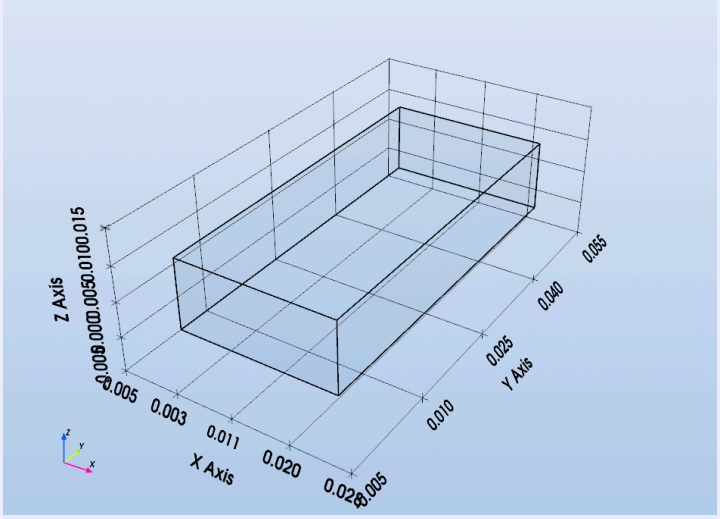
```python

mm = 0.001 # Define a millimeter
wg_width = 22.86*mm # Width of WR90 waveguide
wg_height = 10.16*mm # Height of WR90 waveguide
wg_length = 50*mm # Arbitrary length
```
> Simply define the sizes of the waveguide box

```python

my_model = em.Simulation('MyFirstModel')
```
> Generates the Simulation object that you interface with.

```python

box = em.geo.Box(wg_width, wg_length, wg_height)
```
> Generate a box consisting of air with the appropriate dimensions.

```python

my_model.commit_geometry()
```
> Commit geometry finalizes the geometry. It creates a boolean fragments and reassigns domain numbers

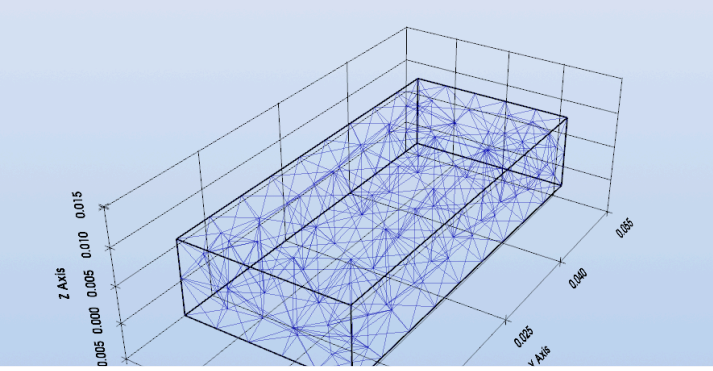
```python

my_model.view()
```
> Here we can view the geometry




```python

my_model.mw.set_frequency_range(8e9, 10e9, 21)
```
> This defines the frequency range of the simulation. This must be called before generating the mesh so that EMerge knows how fine to mesh the domain.

```python

my_model.generate_mesh()
```
> Here the mesh is actually generated along with all necessary internal data.

```python

my_model.view(plot_mesh=True)
```
> We can now view the mesh.



3.2 Resonant Cavity

We will simulate a simple rectangular resonant cavity to get familiar with model building, meshing, solving, and post-processing in EMerge.

python
```
import emerge as em
import numpy as np
```

Next, we define our dimensions for the resonant cavity.

python
```
my_model = em.Simulation('ResonanceCavity')
cm = 0.01
width = 6*cm
height = 2*cm
length = 10*cm
Nmodes = 3      #Amount of modes to solve for
f = 2e9         #First eigenfreq at around 21GHz
```

We build the geometry with one of EMerges many built-in shapes. A rectangular box of dimensions 6 × 10 × 2 cm, centered at the origin is created. Once it has been defined, we commit the geometry so the solver knows the final shape. EMerge automatically commits all geometries so there is no need for you to keep track of it all.

python
```
box = em.geo.Box(width, length, height, alignment=em.CENTER)
my_model.commit_geometry()
```

Before meshing, we provide a frequency so the solver has a reference point.

python
```
my_model.mw.set_frequency(f)
my_model.mw.set_resolution(0.1)
my_model.generate_mesh()
my_model.view(plot_mesh=True)
```

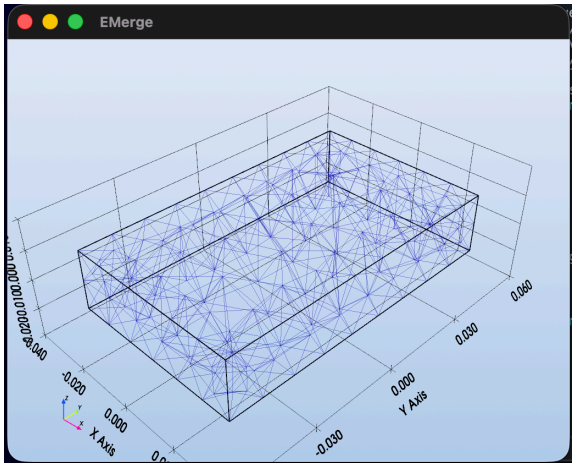


Figure 9: The window showing the resultant mesh

We use the eigenmode solver to extract resonant modes. The results are stored in data.field (vector fields) and data.scalar (scalar info).

python
```
data = my_model.mw.eigenmode(f, nmodes=Nmodes)

# Example: Access the eigenfrequency of the first solution
field = data.field[0]
print(f'Frequency = {field.freq/1e9:.2f} GHz')
#2.91GHz
```

Lastly we add 3 z-planes and plot the Ez field for the first solution data.field[0]. We also add our original box as an outline with opacity 0.1 and finally show the display.

```python
my_model.display.add_field(field.cutplane(ds=0.05*cm, z=0).scalar('Ez','real'),
cmap='rainbow')
my_model.display.add_field(field.cutplane(ds=0.05*cm,
z=height/3).scalar('Ez','real'), cmap='rainbow')
my_model.display.add_field(field.cutplane(ds=0.05*cm, z=-
height/3).scalar('Ez','real'), cmap='rainbow')
my_model.display.populate()
my_model.display.show()
```

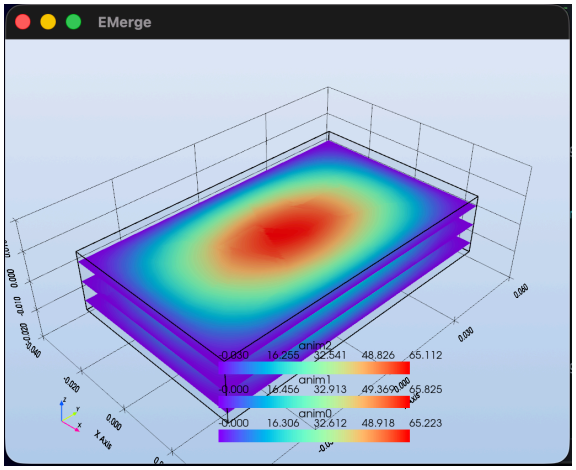


Figure 10: The resultant plot window.

3.3 Step Plan

When setting up a simulation, use the following step by step recipe to buildup your simulation file:

1. Import your modules
2. Define your constants/variables etc.
3. Create a Simulation object for your simulation with the appropriate settings 3.5. If you want to do a geometrical parameter sweep do that now.
4. Construct your Geometry
5. Call `.commit_geometry()` if the geometry is final.
6. Define your simulation frequency and mesh settings
7. Call `.generate_mesh()`
8. Specify your boundary conditions
9. Run your simulation: `.run_sweep()` etc.
10. Post-process your data

3.4 EMerge Behavior

To facilitate proper simulation modelling, EMerge has default behavior programmed in to help with simulations. Sometimes this might lead to unexpected behavior. Many of these can be changed in the user settings (See Section 4.2). Please consider the following default settings before simulating with EMerge:

General Behavior

1. The default log-level is `INFO`
2. By default, settings and logs will not be stored to files when running simulations
3. Models will only save if and only if a simulation has been performed and thus data has been changed.
4. EMerge will use Joblib for saving data. Joblib is not safe as it may execute malicious code if you load other people their models. Use `msgpack` for save serialization and loading. This is still in beta so it might crash.

Modelling Behavior

1. All geometries will by default be considered `AIR` as material.
2. In the PCB-router, all vias's are considered bulk copper domains.
3. In the PCB-router, all traces are considered copper as default material.
4. Dielectrics from the PCB router have priority 11
5. Vias from the PCB router have priority 12
6. Geometries that had their materials assigned are given a slightly higher priority when two geometries have the same priority.

Microwave Behavior

1. The default meshing resolution is 0.3 which means that the maximum edge size is $0.3\lambda\Re\left(\sqrt{\varepsilon_r\mu_r}\right)^{-1}$.
2. All exterior boundaries with no boundary-conditions assigned will automatically be assigned PEC.
3. All LumpedPort boundary conditions assume a 50Ω impedance by default.
4. All 3D volumes with a conductive material of which the conductivity is beyond some limit(Section 4.2) will be considered PEC. This is to speed up simulations.
5. All 3D volumes with a conductive material of which the conductivity is beyond some limit(Section 4.2) will be excluded from the simulation domain. The boundaries will be replaced by a SurfaceImpedance boundary condition.
6. All surfaces with a conductivity above a certain limit will either be assigned SurfaceImpedance or PEC.
7. If coordinate-/frequency-dependent conductivity values are assigned to SurfaceImpedance conditions, they will be evaluated at 1GHz and at x,y,z=0 when deciding if the previous 2 rules.

Simulation Behavior

1. EMerge will automatically do a check for free available RAM memory. It estimates that simulations might take 0.25 MB per DoF. This is not always the case. This can be turned off

3.5 Warning Report System

Simulations can sometimes be difficult to setup. Its hard to know as a user how to precisely setup the simulation correctly and as a developer, it is hard to make software that helps you in just the right way.

To improve the user experience, EMerge comes with a warning report system. This is essentially a system that, during code execution, collects a series of events the EMerge considers potentially critical for a good simulation. Some heuristic rules are defined when a situation occurs that might lead to problematic outcomes. In these cases a warning is added to the report system. At the end of the scripts execution, all these warnings are printed to the terminal. This is to prevent critical warnings from disappearing in long lists of log messages that you might overlook.

There are several situations that might trigger these warnings. Here is a (not complete list):

- Any time S-parameter matrix columns have their total power exceed 1.0. This happens quite often and is almost always due to numerical accuracy. It especially happens in simulations without methods of energy loss,
- When two geometries try to assign a material to the same domain without any explicit priority. See also Section 4.7.1,
- If a lot of frequency points are detected,
- A frequency in the THz region is detected,
- If a lumped port has extreme dimensions.

None of these warnings mean that something is going wrong. Instead, think of them as reasons why something might be wrong with the simulation.

If you find cases where you'd like a report to be made, you can always suggest them to the developer(s).

4 Concepts

4.1 The Simulation object

The main object managing your simulation model is naturally the `Simulation` class. You initialize it like this:

```python
import emerge as em
sim = em.Simulation('YourProjectName')
```

The `Simulation` class has various arguments and keyword arguments that will be described now.

- **Name**: This is the primary argument of the Simulation class. The simulation name is used for file name generation if you decide to store data to the hard drive and for identifying your model in a possible script running multiple models simultaneously.
- **loglevel**="INFO": This sets the loglevel of what will be printed in the terminal. More about that later.
- **save_file**=False: If the Simulation should save data to the hard drive after running.
- **load_file**=False: If the Simulation should attempt to load pre-existing data from a previous save.
- **write_log**=False: If a detailed log should be written to the hard-drive.
- **path_suffix**=".EMResults": The suffix for the simulation data directory.
- **store_system**="joblib": Since version 2.2 there are two different store systems for EMerge data files. The default is Joblib which serializes Python object. Since 2.2 there is also an interface with msgpack which transforms these classes back and forth between simple dictionary objects. This is safer but some issues may persist as all classes must inherit from the `Saveable` class.

4.1.1 Loglevel

The loglevel is a common concept in logging in general. The idea is that loglevels have certain severity levels:

0. Trace
1. Debug
2. Info
3. Warning
4. Error

Loggers can produce information at any of these levels. By setting a loglevel to some value, only logs of that severity level or higher will be displayed in the terminal. Thus if the log-level is 'Info' (or 'INFO' in EMerge), only Info, Warning and Error messages will be shown.

If you set `write_log=True`, the log-level of the file will always be Trace which shows more information and more detail about how the simulation ran. If you have problems during simulations, sending log-files can help with problem resolution.

Important

If you have a desire for certain information to be disclosed, feel free to send a suggestion/ticket/mail and I will include it when possible.

4.1.2 Saving and Loading

More about saving and loading is discussed in detail in Section 4.11. The basics are that data can be stored simply by setting the argument `save_file=True`. At the end of a simulation, even when it crashes, EMerge will automatically store the simulation data to a file.

It is known however that the usual `atexit` method of saving is unstable so make sure to call `mysim.save()` at the end of your file to ensure proper storing!

If you then run a different script in the same folder with the same Simulation model name with `load_file=True`, it will automatically load the data produced by the previous simulation.

This feature is convenient in cases where simulations take long to run and you want to explore different means of post processing.

How you can save data is described more generally in Section 4.12. The key point is that not all data might be easily saved and loaded as you might expect. This is because saving and loading in Python can be complicated. We are still working on a better more stable way of saving and storing data that does not fill up the hard-drive and is backwards compatibility.

4.2 Settings

The Simulation object contains a `.settings` attribute which is the main EMerge `Settings` class. This table provides an overview of all the settings, default values and meaning. You can modify settings directly:

```python
sim.settings.mw_2dbc = False
```

property	default	description
`mw_2dbc: bool`	True	If 2D geometries with Materials should automatically be assigned as either SurfaceImpedance or PEC in the Microwave physics.
`mw_2dbc_lim: float`	10.0 [S/m]	The scalar conductivity limit beyond which EMerge will assign SurfaceImpedance or PEC.
`mw_2dbc_peclim: float`	1e8 [S/m]	The scalar bulk conductivity limit beyond which EMerge will assign PEC instead of SurfaceImpedance[1]
`mw_3d_peclim: float`	1e8 [S/m]	The bulk conductivity limit beyond which 3D volumes are considered PEC.
`mw_3d_surfimplim: float`	1e4 [S/m]	The bulk conductivity limit beyond which volumetric degrees of freedom (DoF) will not be calculated and the boundary will be replaced by a SurfaceImpedance boundary condition.
`check_size: blool`	True	Stops the simulation if the bounding box of the domain requires more than 100,000 tetrahedra.
`mw_cap_sp_single: bool`	True	If individual S-parameters are hard capped at $\|S_{ij}\| \leq 1$
`mw_cap_sp_col: bool`	True	If columns of S-parameters are hard capped at 1: $\sum_{i=1}^{N} \|S_{ij}\|^2 \leq 1$. This power conservation limit is executed before the single cap and will also guarantee that individual S-parameters are smaller than 1.
`mw_recip_sp: bool`	False	Ensures that S-parameters are exactly reciprocal $S_{ij} = S_{ji}$. This is not turned on because simulations with metals might not be reciprocal.
`sim_symmetry_limit`	0.02	A threshold used for direct solver to determine if a system is to be considered

[1] PEC is more efficient to solve compared to SurfaceImpedance.

property	default	description
		numerically Symmetrical. A system $\boldsymbol{Ax} = \boldsymbol{b}$ with $\frac{\max\left(\lvert A_{ij}-A_{ji}\rvert\right)}{\max\left(\lvert A_{ij}\rvert\right)} > 0.02$ will be considered an asymmetric system.[2]
`size_check`	True	Stops the simulation if a domain has more than 700k degrees of freedom.
`auto_save`	False	Try to save the simulation automatically when it aborts early. Uses `atexit` which sometimes gives corrupted simulation results.[3]
`save_after_sim`	True	When True, EMerge will only save a simulation if and only if a simulation has actually been run. This prevents crashing simulations or just starting a file from overwriting empty simulation data.
`save_method`	`joblib`	The serialization method to use. Choices are `joblib` and `msgpack` (new). Joblib is serialized Python objects which is not safe.
`check_ram`	True	Executes a check for the available RAM memory. It will halt the simulation if it expects it to crash. It does not include Virual memory.

[2] Theoretically only matrices of systems with periodic boundaries or asymmetric material property tensors yield asymmetric matrices. Treating symmetric systems as asymmetric is just slightly slower to solve in some cases. Treating asymmetric systems as symmetric yields bad results.

[3] Use the `with` context manager in Python to run simulations safely!

4.3 Constructive Geometry

Constructive geometry is well supported in EMerge through its interface with the OpenCASCADE kernel available in GMSH. This page will walk you through the basics of constructing your geometry. You may assume that each code block you find follows the following setup:

```python
import emerge as em
m = em.Simulation('model_name')
```

4.3.1 Basic shapes

Most of the geometry options in EMerge can be accessed through its `.geo` sub-module. Below is an overview of some standard ones:

```python
box = em.geo.Box(width, depth, height, ...)
sphere = em.geo.Sphere(radius, ...)
half_sphere = em.geo.HalfSphere(radius, ...)
cylinder = em.geo.Cylinder(radius, length, ...)
xyplane = em.geo.XYPLane(...)
plane = em.geo.Plane(ax1, ax2,...)
horn = em.geo.Horn(...)
cone = em.geo.Cone(...)
disc = em.geo.Disc(...)
```

To support geometry creation, EMerge often refers to some geometric cartesian primitives. These are

- `em.CoordinateSystem()` or `em.CS()`: An object representing an orthonormal coordinate system at some origin. The standard coordinate system with the unit vectors $\hat{x} = (1, 0, 0)$, $\hat{y} = (0, 1, 0)$ and $\hat{z} = (0, 0, 1)$ at origin $\mathcal{O} = (0, 0, 0)$ is simply `em.GCS`.
- `em.Axis()`: The primitive for a given axis in 3D space. The basic axes are predefined as `em.XAX`, `em.YAX` and `em.ZAX`.
- `em.Plane()`: The primitive of two normal axes. The basic defined ones are `em.XYPlane` etc...

Often geometries are defined in some strict geometrical orientation. For example, all cylinders are always oriented along the Z-axis. To generate a cylinder along a different axis, you can pass a new coordinate system to it. You can derive coordinate systems from existing ones using methods such as `my_cs.displace(dx, dy, dz)` and `my_cs.rotate( ... )`.

```python
import emerge as em
sim = em.Simulation('myname')
box = em.geo.Box(1,1,1)
sim.view()
```

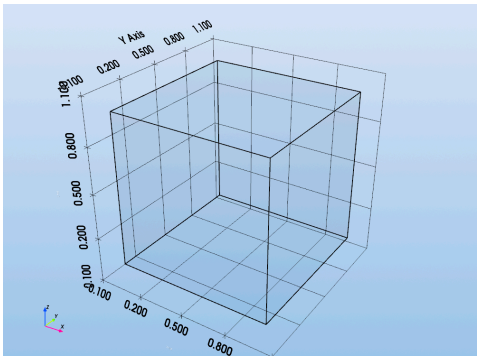


Figure 12: A simple box.

4.3.2 Axes, Planes and Coordinate Systems

The three geometric primitives called the `Axis` class, `Plane` class and `CoordinateSystem` class form a lot of the basis of CAD modeling in EMerge.

The `Axis` class is an abstraction of a 3D vector (without location). In most cases where an Axis object can be used as an argument, one may also use a simple tuple. However, the Axis may be used to simplify certain geometrical definitions.

For applications where directions or axes matter one may consult the predefined axes objects in `em.XAX`, `em.YAX` and `em.ZAX`. From these we can construct planes. For example, a `Plane` object may be created by calling the `.pair()` method. Another way is by multiplying the two Axes. For example, the XY-plane can be generated in the following ways:

python
```python
my_xy_plane = em.XAX.pair(em.YAX) #or
my_xy_plane = em.XAX*em.YAX
```

We may also use the `.cross()` and `.dot()` methods to do vector cross and dot products.

Similarly, we may also multiply a Plane with an Axis object to create a coordinate system:

python
```python
my_yzc_cs = em.YAX * em.ZAX * em.XAX
```

In cases where only one axis really matters we may also create a coordinate system from an axis as following:

python
```python
my_zxy_cs = em.XAX.construct_cs(x0, y0, z0)
```

Which will create a new coordinate system where the Z-axis is aligned with the X-axis. The `.construct_cs()` method always aligns the local Z-axis with the provided Axis object.

This can be useful when creating a Cylinder for example where the Cylinder's axis is always oriented along the provided Coordinate System's Z-axis.

Another quick way to create coordinate systems uses the `em.cs()` function. This function should be used to orient the coordinate system axes along the main Cartesian axes using a string. In order, one can define the X, then Y then Z-axis by using a letter "x", "y" or "z" for its definition. Capital letters are the negative axes. The following example creates a coordinate system with the following axes:

- $\hat{x} = (0, -1, 0)$
- $\hat{y} = (0, 0, 1)$
- $\hat{z} = (-1, 0, 0)$

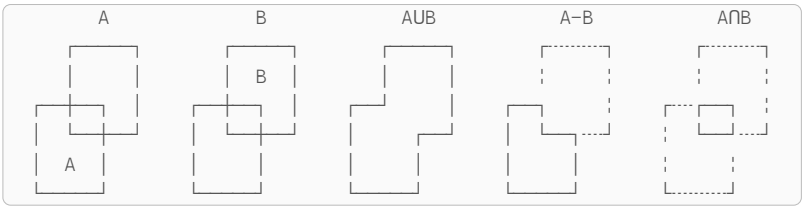
python
```python
my_yzx_cs = em.cs("YzX", origin=(x0,y0,z0))
```

4.3.3 Boolean operations

The boolean operations to subtract and unite objects are also implemented in EMerge. Here are some examples

python
```python
my_box1 = em.Box(1,1,1,position=(0,0,0))
my_box2 = em.Box(1,1,1,position=(0.5,0.5,0.5)))
# Operations
my_union = em.geo.add(my_box1, my_box2)
my_difference = em.geo.subtract(my_box1, my_box2)
my_intersection = em.geo.intersect(my_box1, my_box2)
# Or add multiple objects at the same time
my_unification = em.geo.unite(my_box1, my_box2, my_box3, ...)
```

By default, all tool objects (objects in the second argument position) will be removed from existence. You can keep these by changing the optional arguments `remove_tool=False`. The boolean operations of two shapes are understood as following:



Listing 1: Boolean operations: A, B, Union(A,B), Difference(A,B), Intersection(A,B)

```python
sim = em.Simulation('myname2')
box = em.geo.Box(1,1,1)
box2 = em.geo.Box(1,1,1, (0.5, 0.5, 0.5))
box3 = em.geo.subtract(box, box2)
sim.view()
```

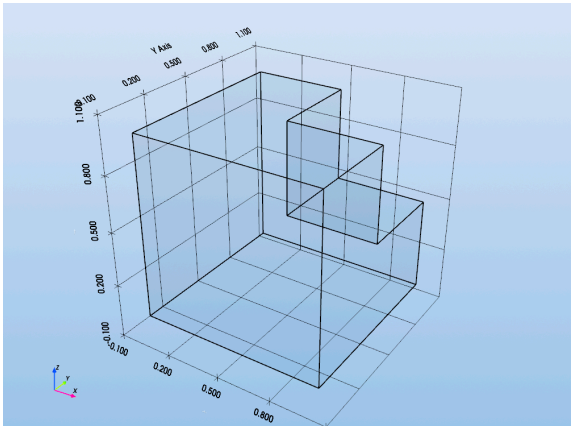


Figure 13: Two boxed subtracted from one another.

4.3.4 Transformations

There are some basic transformations implemented at this point. Below is an overview:

```python
mirrored = em.geo.mirror(my_obj, ...)
translated = em.geo.translate(my_obj, ...)
rotated = em.geo.rotate(my_obj, ...)
united = em.geo.unite(my_obj1, my_obj2, ...)
expanded = em.geo.expand_surface(my_surf, ...)
moved = em.geo.change_coordinate_system(my_obj, ...)
extruded = em.geo.extrude(my_surface, ...)
moved = em.geo.stick(obj1, anch1, anch2)
```

More will be added as EMerge develops.

4.3.5 Under the hood

All CAD objects generated by the OpenCASCADE kernel are represented using tags (integers) associated with some dimension. In GMSH this looks like a tuple of two integers. For example (3,2) may be a volume with tag 2. To simplify the interaction with this kernel, EMerge has its own class system based on the generic class type `GeoObject`. For each dimension there is the `GeoPoint`, `GeoEdge`, `GeoFace` and `GeoVolume`. These classes remember their own contents and attributes. Materials are for example also assigned to these class objects and this is also what each constructive geometry function returns. These `GeoObject` instances are thus EMerges own internal representation of what GMSH manages under the hood. After modelling and before meshing, EMerge will perform a boolean fragment splitting the geometry up in disjoint areas. Only after this step can you proceed to assigning boundary conditions. To commit the current state of your model to be the final geometry, run the `.commit_geometry()` function:

```python
my_box1 = em.geo.Box(...)
my_box2 = em.geo.Box(...)
my_model.commit_geometry()
```

4.3.6 Advanced geometries

Besides basic shapes, EMerge offers some extra features to create more advanced geometries. The exact implementation of these will not be explained here. Here is just a quick overview of the features.

- `PCB`: A class representing a PCB with convenient methods for creating copper traces, ground planes, dielectric slabs, via's etc. More in Section 8.5.
- `pmlbox`: A box that automatically adds PML regions with the appropriate material properties. Be careful with the number of layers. More in Section 5.2.1
- `Modeler`: This class is still under constriction. It is intended to simplify the process of creating many shapes in sequence. The philosophy is the same as the PCB Layouter but for generic 3D shapes.

4.4 Boundary selection

Selecting boundaries in code can be a bit tricky. A huge advantage of a GUI is that you can click on boundaries to define them. This is not as much of a luxury to us. EMerge has various different ways to select boundaries (and other domains). This page will discuss the options.

4.4.1 Under the hood

To understand how to make selections we have to know a bit better underwater how EMerge works.

Each volumetric and surface geometry in your model has its own `GeoObject` class instance. Faces, volumes that result from the fragment are separete to some extend. Within EMerge, all faces inside the geometry will have some integer number assigned to them called a tag which is defined by GMSH and OpenCASCADE. While these tags are crucial for EMerge, we do not want you to ever have to think about them.

In principle, a list of tags can serve as a selection but the numbers that are assigned are only defined at run-time. This means that if you hard code a selection with integers, it may not be valid later as the model develops. GMSH may decide to assign different numbers to those.

To help you, the user, an abstraction of a selection has been devised that you can generate while running the simulation and is called a `Selection` class object. Just like with the geometry there are `FaceSelection`, `PointSelection`, `DomainSelection` and `EdgeSelection` objects (and generic `Selection` objects). These serve to hide these changing integers to the user as they are derived in more dynamic intelligent means as the model is build. We will discuss different ways to create these selections.

We will now discuss various methods to generate selections.

4.4.2 Passing objects

In many casees when you need to specify a face for lets say boundary condition assignment, it does not matter if you pass a `FaceSelection` or just the object that is a 2D surface itself. If for example you generated a 2D PCB trace, you can pass those to boundary conditions as well. However, any object can always be turned into a selection by its `my_object.selection` property. This will just return a selection of that object. Additionally, you can bundle many objects into a single selection (as long as they share the same dimension) using:

Example
See example file: `demo_boundary_selection.py`

```python
my_selection = em.select(my_obj1, my_obj2, ...)
```

4.4.3 Creating selections

In the simulation object you'll find a `Selector` instance under the attribute named `.select`. This uses a "method chaining" philosophy to define selections. Some selection methods are generic to different dimensions (domains, faces, points etc). In that case you simply specify what you want to select by setting the selection state. The simplest way to select something is using the `.near()` method that computes if something is near a 3D coordinate.

```python
my_selection = my_model.select.face.near(2,0.5,-1)
  > selects the face closest to x=2.0, y=0.5, z=-1.0
```

! WARNING !

Due to the current architecture, Selections by the Selector class can only be made in the script that just generated the simulation. If you use a secondary script to post-process the file, the selector class will not work well anymore.

Here, `model.select` is just the Selector class instance, then the property `.face` sets the selection state to select faces. Then the `.near()` method finally creates a face selection closest to that coordinate.

There are more methods available to make selections

- `.in_plane(x,y,z,nx,ny,nz)`: Always selects a face. It will select a face if, within some tolerance, inside the infinite plane defined by the normal vector at origin `(x,y,z)` with normal vector `(nx, ny, nz)`.
- `.in_layer(x,y,z,vector)`: Selects all entities inside a 3D layer defined by the the two planes defined by coordinate (x,y,z) and normal `vector` where the length of the vector defines the thickness of the layer. This all points within the layer from x=1 to x=1.5 is computed using `select.point.inlayer(1,0,0,(0.5,0,0))`.
- `.code(' ... ')`: An experimental feature that selects based on string hashes generated by the PyVista display if you select a face with face select mode `[F]`. The codes will be printed in the terminal.

4.4.4 Object faces

CAD Geometries in EMerge sometimes have names associated to faces. Boxes for example have the faces `left` and `right` for the -x and +x size respectively as well as `front` and `back` for the Y-axis and `bottom` and `top` for the Z-axis faces. Cylinders also have `front` and `back` defined (and left, right, top and bottom). The faces can be selected using the syntax:

```python
my_box = em.geo.Box(1,1,1)
my_selection = my_box.face('front')
my_selection = my_box.front # Only available before operations.
```

At any moment, you can see what face label EMerge uses for a geometry by calling:

```python
my_model.view(face_labels=True)
```

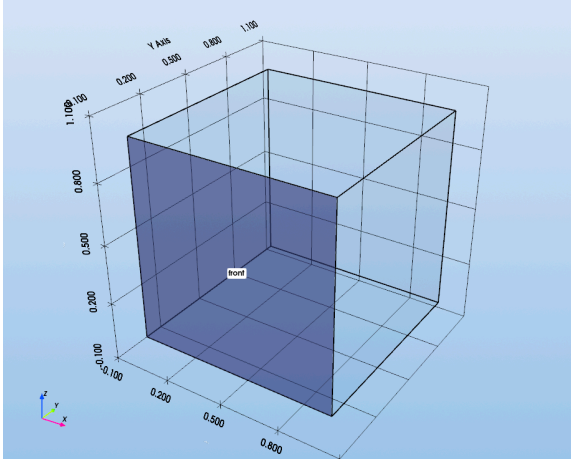


Figure 15: Selecting the front face.

The face selection is based on an origin/normal data pair that will be transformed along with the geometry. This way, after rotations and boolean operations, faces can still be called. However, the face definitions will no longer exist in the same way after boolean operations. This can be resolved using the `tool` argument. Lets say that we have two boxes that we subtract from the other. We can still select the left faces from each as following:

python
```python
my_box1 = em.geo.Box(1,1,1)
my_box2 = em.geo.Box(1,1,1,position=(0.5,0.5,0.5))
my_box3 = em.geo.subtract(my_box1,my_box2)

my_left_face_1 = box3.face('left',tool=my_box1)
my_left_face_2 = box3.face('left',tool=my_box2)
```

All boundary selection features will be inherited by the resultant objects. You thus have to specify which `tool` object has to be used when you refer to the name `left`.

You can also select multiple faces using the `.faces()` method like this `myobj.faces('front','back')`. One can also select all faces except some names using the `.boundary()` method.

python
```python
my_box1 = em.geo.Box(1,1,1)
my_selection = my_box1.boundary(except='front')
my_selection = my_box1.boundary(except=['front','back'])
```

The `.boundary` method will select all boundary faces. The `except` optional argument allows you to specify a set of faces to **not** include.

3D geometries with internal holes have a boundary on the inside and on the inside. This can sometimes get tricky when trying to select specifically the exterior face. Lets say you have an air-sphere with some internal space cut out to add a waveguide port. If you use the `.boundary()` method, it will also select the internal boundaries.

In cases where you specifically want to select external boundaries EMerge has the `.exterior_faces()` method.

The exterior faces method is essentially calling the `.boundary()` method on one of the tool objects. You simply pass the original object that would have defined its exterior. Here is an example:

python
```python
sphere = em.geo.Sphere(radius=1)
box = em.geo.Box(0.1, 0.1, 0.1, alignment=em.CENTER)
new_sphere = em.geo.subtract(sphere, box)
exterior = new_sphere.exterior_faces(sphere)
```

4.4.4.1 Object face name definitions

The following pre-existing face names are defined for geometries.

Object	Object Face	Given Label
Box	-X +X -Y +Y -Z +Z	left right front back bottom top
Cylinder, CoaxCylinder	-Z +Z	left,front,bottom right, back, top
Extrusion	First face last face	front back
Sphere	outside	outside
HalfSphere	Outside cross section	outside disc

After a box has been manipulated with boolean operations, it no longer can be considered a true Box. Any object that is not a pure base geometric entity as enlisted above will get some face names assigned automatically. EMerge auto-generate face names for all geometries that are not defined explicitly by the given class. Here is an overview of the rules:

- Any face that is orthogonal to any of the major cartesian axes will automatically get the names “-x”,“+x”,“-y”,“+y”,“-z”,“+z” assigned to them. The “-x” face will correspond to any YZ face that is at the smallest X-coordinate.
- Any other face will get a name assigned to them with an increasing integer with the convention “Face0”,“Face1” etc.

To view what names are chosen for a single object you can use the following line:

python
```
1 m.view(selections=feed.all_faces())
```

If you want to show all face names you can use:

python
```
1 m.view(face_labels=True)
```

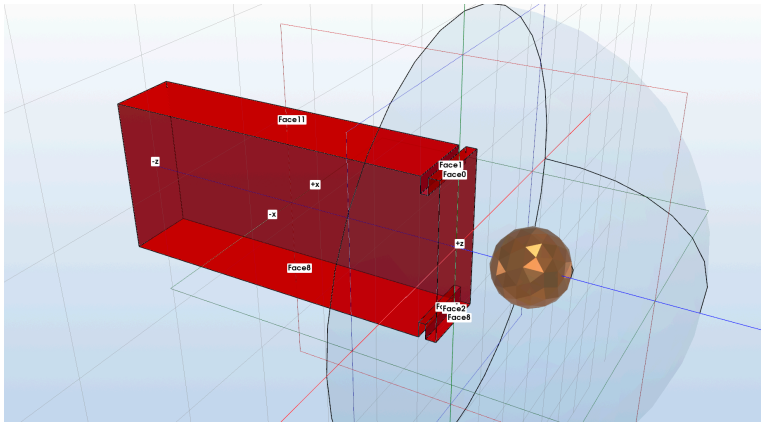


Figure 16: An example of face names assigned to a derived geometry.

4.4.5 Selecting multiple objects

Sometimes you want to pass a bunch of objects or surfaces in a single selection. Then there is a simple function `.select()`:

python

```python
my_surf1 = em.geo.XYPolygon(...)
my_surf2 = em.geo.XYPolygon(...)
my_surf3 = em.geo.XYPolygon(...)

my_selection = em.select(my_surf1, my_surf2, my_surf3)
```

This is convenient when adding a bunch of surface to a PEC boundary condition for example.

4.4.6 Visual selection

Since version 2.7, the standard `.view()` window allows for mouse click selection. Simply call `.view()` in the current geometry and right click on an object or face (`.view(face_labels=True)`) you wish to select. Use the right (→) and left (←) keys to cycle through different objects that are below the mouse. The name of the object will appear in the top of the window and also be printed in the session terminal.

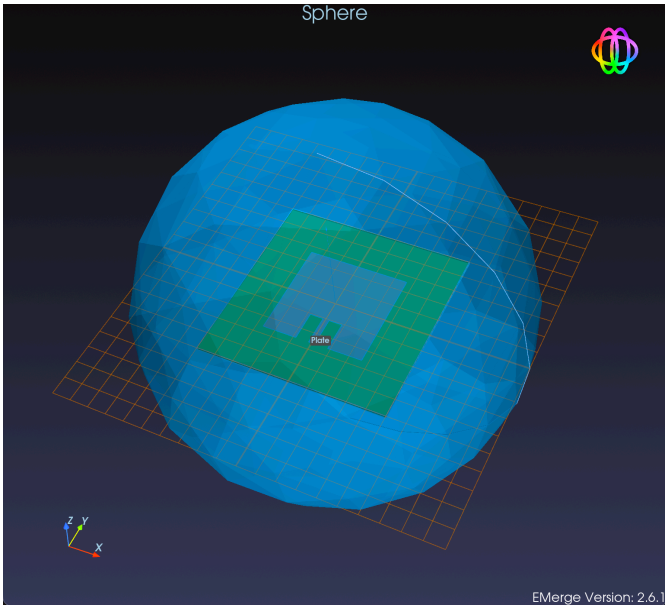


Figure 17: An example of a slection.

Afterwards, that object can be selected using

```python
model.find_geo("Sphere")
```

4.5 Anchor points

Since version 2.1, objects in EMerge will have a set of so called Anchor points. Anchor points are created at the bounding box of a geometry on each vertex, edge and face center. The anchor points also contain their own local coordinate system. Achor points can be used in two ways:

- Many functions that require coordinates also accept these Anchor points as coordinates.
- the `em.geo.stick()` method allows the user to stick anchor points to other anchor points.

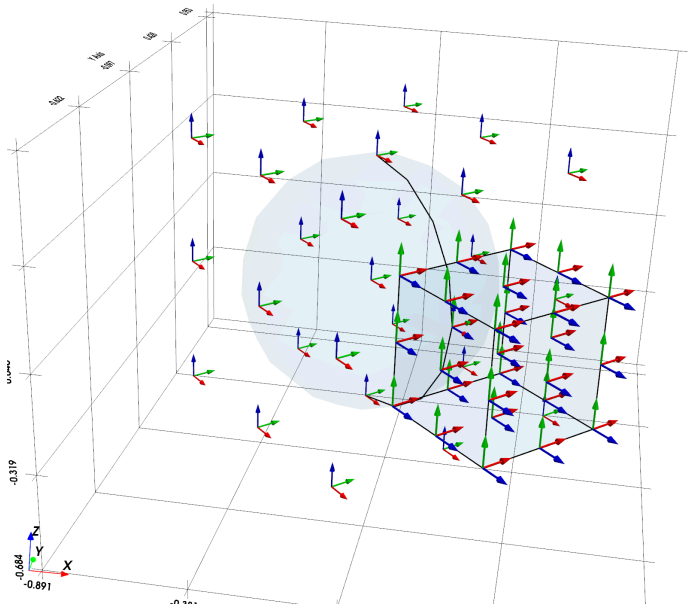


Figure 18: An example of anchor points around geometries.

The latter of the two functionalities will be discussed here. Lets start by defining two boxes:

```python
import emerge as em
box1 = em.geo.Box(1.2,0.8,0.5)
box2 = em.geo.Box(0.6,0.6,0.6, (2,0,0))
```

The sizes of these boxes are random

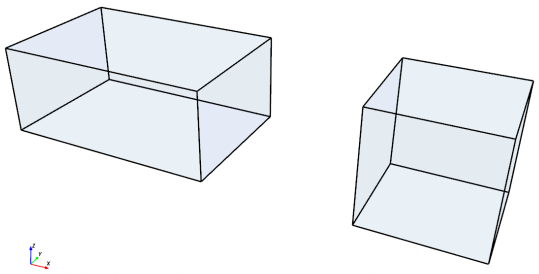


Figure 19: The two boxes

You can visualize the anchor points on these boxes by making a custom display and adding all the anchor points:

python
```python
sim.commit_geometry()
sim.quick_mesh()
sim.display.add_objects(box1, box2)
sim.display.add_anchors(box1.anch.all_anchors())
sim.display.add_anchors(box2.anch.all_anchors(), 0.5)
sim.display.show()
```

This should look something like this:

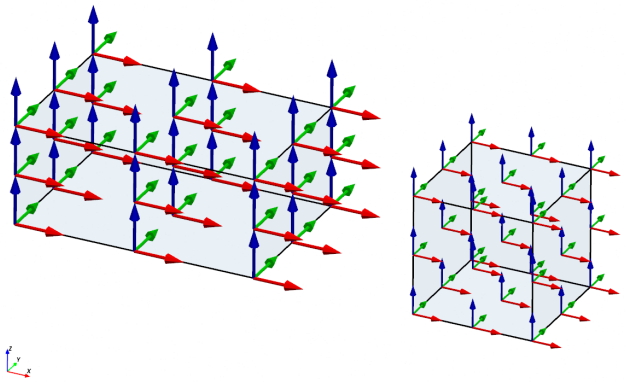


Figure 20: The two boxes

The anchor points are all contained in a so called `AnchorSet` object. In this achor set one can access the anchor points of the bounding box of the original object with properties using the following naming convention:

- Anchor points are denoted relative to the center
- Small letters are the negative cartesian axes, capitals the positive
- `x` is left (-x)
- `X` is right (+x)
- `y` is the front (-y)
- `Y` is the back (+y)
- `z` is the bottom (-z)
- `Z` is the top (+z)

They can also be cascaded for the edges and corner points

- `xy` is the front-left edge
- `XYZ` is the back right top corner
- etc.

For example, we can try to place the small box's bottom left front corner and alighn it with the long box's top left front corner like this:

python
```python
em.geo.stick(box2, box2.anch.xyz, box1.anch.xyZ)
```

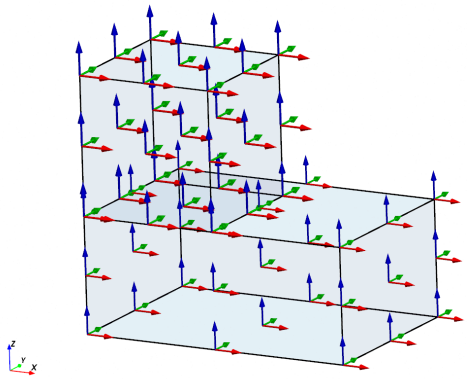


Figure 21: Small box on top of the big box.

As you can see, the small box moved where we wanted it to. Anchor points transform with the geometries so even if we rotate or mirror our original object, the anchor points and coordinate systems will move with them.

We can also use the anchor points axes systems to reorient our objects to alight with the other axis system. In this case, the coordinate systems are the same. There are some basic operations to transform anchor points to align objects differently. For example, we can choose to make the positive X-axis of our small box's anchor points align with the negative X axis of the big box. One way is to mirro the anchor point of the second box about its X-axis by calling the `.mx` (mirror-X) attribute

```python
em.geo.stick(box2, box2.anch.xyz, box1.anch.xyZ.mx)
```

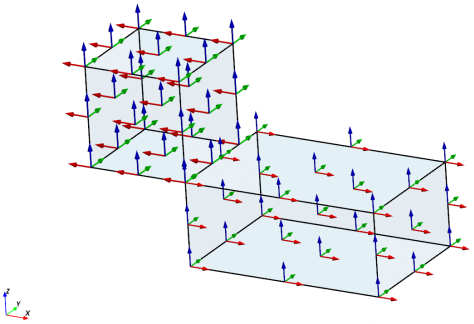


Figure 22: Small box on top of the big box mirrored in the X-axis.

Or we could rotate our big boxes achor point 180 degrees about the Y-axis to create a full 180 degree turn by calling the `.ty` method:

```python
em.geo.stick(box2, box2.anch.xyz, box1.anch.xyZ.ty)
```

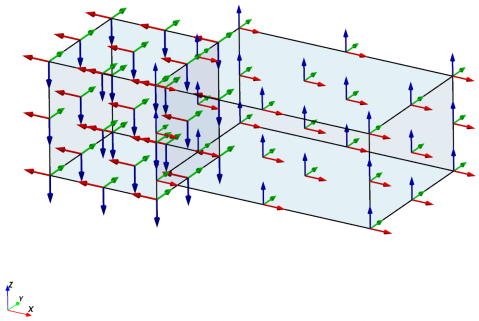


Figure 23: Small box on top of the big box rotated around the Y-axis.

4.6 Geometry Import

4.6.1 Step files

It is possible to import .step file in EMerge. To facilitate this, EMerge uses the API offered by GMSH. The current API struggles with importing labels assigned to geometries which makes identifying which objects are which slightly complicated.

To import a .step file we use the `em.geo.STEPItems()` class as following:

python
```
1 cont = em.geo.STEPItems('GeoName','my/file/name.step', unit=0.001)
```

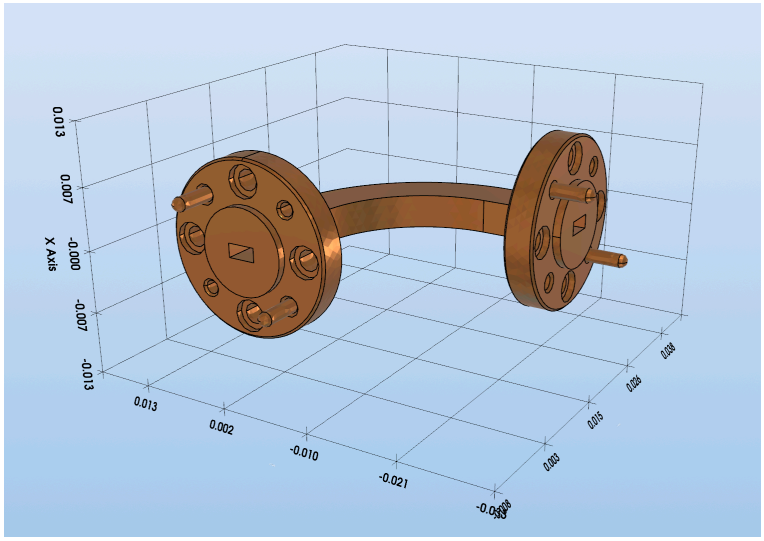


Figure 24: Quantum Microwave bend sourced from https://quantummicrowave.com/product/qmc-wb10-h90-w-band-waveguide-bend/.

This method will return a `STEPItems` object that contains all that is contained in the step file. Units that the step file is defined in are not always properly imported so make sure to set a value for the unit at say millimeters if this didn't happen already.

The container contains all your geometries as lists in the following attributes:

- `cont.volumes: list[GeoVolume]`
- `cont.surfaces: list[GeoSurface]`
- `cont.edges: list[GeoEdge]`
- `cont.points: list[GeoPoint]`

In cases where your geometry is known to be just a single geometry, you can instantly convert the `STEPItems` object into a single GeoVolume object using

python
```
1 cont = em.geo.STEPItems('GeoName','my/file/name.step', unit=0.001)
2 my_volume = cont.as_volume()
```

If it consisted of multiple volumes then they will be united. You can also unpack all your volumes using:

python
```
1 vol1, vol2, vol3 = cont.volumes
```

This does imply that you have prior knowledge of how many volumes there are. If you don't then you can first execute a quick mesh and then call `model.view(labels=True)`. Or you can call `print(cont.volumes)` to see what is being imported. Since version 2.7 you can right-click on geometries to selet them and then cycle through objects behind each other using the left and right arrow keys (see Section 4.4.6).

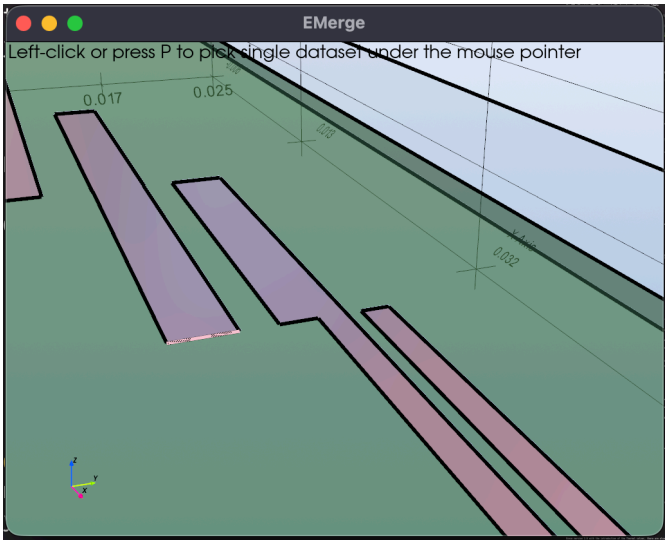
You can quickly generate an airbox around your imported model using the `.enclose()` method or the `em.geo.open_region()` methods (See Section 5.2). This will automatically

create an airbox around your geometry. If you want to use the parameters instead to fill a PML box you can also use the `.enclose_params()` method to generate the dimensions and position tuple which can be used like this:

python
```python
cont = em.geo.STEPItems(...)
pmlbox = em.geo.pmlbox(*cont.enclose_params(...), ...)
```

4.6.1.1 Adding ports from PCB files

Sometimes an imported PCB has no port faces which are to be added later. Since version 2.5.5 there is a hotkey **L** in the view mode which allows you to select edges. Upon clickin on an edge, a terminal statement will be printed that can be used to generate a port on that specific edge.



Which prints:

```
Edge tag [66] data:
    vertex[1] = (0.047500285999999996, 0.010366095600000003, 0.0)
    vertex[2] = (0.047500285999999996, 0.011470589200000004, 0.0)
    direction = (0.0, 0.0011044936000000009, 0.0)
    center = [0.04750029 0.01091834 0.        ]
    plane = em.geo.Plate(origin=(0.047500285999999996, 0.010366095600000003,
  0.0),u=(0.0, 0.0011044936000000009, 0.0),v=(x,y,z))
```

The last line can be used as a geometry input:

python
```python
plane = em.geo.Plate(
  origin=(0.047500286, 0.0103660956, 0.0),
  u=(0.0, 0.0011044936, 0.0),
  v=(x,y,z))
```

4.6.2 Gerber files (Beta)

To import a Gerber file, you can use the so called `FileBasedPCB` class. This is a derived class from the `PCB` class explained in Section 8.5. This is available in the beta section:

python
```python
from emerge.beta.gerber import FileBasedPCB

# usual setup

GerberPCB = FileBasedPCB(...) # Usual PCB construtor
GerberPCB.layer_from_file("MyGerberFile.gbr") # Layer file
GerberPCB.vias_from_file("MyExcellonDrillFile") # Drill file
```

It may be possible that some features are not supported yet. In this case, please contact me with the Gerber file so that I can support more of the Gerber features.

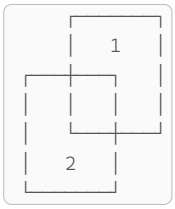
4.7 Materials

You can assign materials quite simply using the `.set_material()` method or by setting it directly. Materials can be created manually or ones can be chosen from the library in `em.lib`.

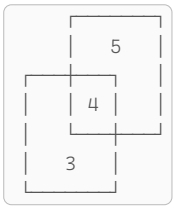
```python
my_material = em.Material(...)
my_box = em.geo.Box(...).set_material(my_material)
# or
my_box.material = em.lib.MET_COPPER
```

4.7.1 Overlapping geometries and Priority

Before the mesh is created, EMerge will execute a boolean fragment. This means that the entire simulation domain is cut up into strictly disjoint volumes. If two boxes intersect like so:



They will be cut up in three volumes like this:



As you notice, the tags have changed. What numbers GMSH decides to use is hard to predict and not relevant for the user.

To make object assignment consistent, EMerge re-maps the tags for you such that the box with tag (2) now has the tags (3,4) and box with tag (1) now has the tags (4,5). Clearly, domain (4) now is shared by both Box objects. If both have a different material, the last one created will define the material. This is because by default, all objects and domains have the same `priority` value set to `10`.

The order of generation decides which material gets assigned if the priorities are the same. But higher or lower priorities are dominant. Thus by changing the priority of an object, you may decide which material assignment is more important!

You can change the priority by calling `my_object.prio_set(5)`. This way, you can create a domain, completely envelop it with a large air-box and then set the air-box priority to (1) so that all domains keep their material assignment except for the holes left over by air. Other ways to change priority are:

- `obj.background()`: Lowers the priority 10 below the lowest current existing.
- `obj.foreground()`: Increases the priority 10 above the highest current existing.
- `obj.prio_set(value)`: Explicitly set the priority.
- `obj.prio_up()`: Increases the priority by 1.
- `obj.prio_down()`: Decreases the priority by 1.
- `obj.above(other)`: Puts the priority above some other object.
- `obj.below(other)`: Put the priority below some other object.

In cases where two geometries share the same priority, EMerge will have to make some choice on which one to use the material from. For this choice it has two fallback rules.

First, if any of the two objects specifically had its `.set_material()` method called, it will be considered more important.

If both did not have `.set_material()` called (or both had), it will use the order of definition. This conflict will also be reported in the final warnings when simulations are done. See Section 3.5.

4.7.2 Custom Materials

You can define your own materials by setting the following properties
- `er = 1.0`: Relative permittivity (real or complex) ε_r
- `ur = 1.0`: Relative permeability (real or complex) μ_r
- `tand = 0.0`: Loss tangent (real) $\tan(\delta)$
- `cond = 0.0`: Bulk conductivity (complex) σ_e

From this the relative permittivity is computed as

$$\varepsilon_{r,fin} = \varepsilon_r \cdot (1 - j\tan(\delta)) - \frac{j\sigma_e}{\omega\varepsilon_0}$$

Clearly, the relative permittivity can be over-defined by both a complex value for `er` and a chosen `tand`. Its up to the user to make a smart choice.

It is also possible to define frequency and coordinate dependent material property and to define them as rank-2 tensors.

For a simple constant tensor one can use either of the following options:

python
```python
mat1 = em.Material(er= 2.0)  # For a scalar value
> εr = 2.0
mat2 = em.Material(er= np.array([1.0, 2.0, 3.0])) # diagonal rank-2
> εr = [[1.0, 0, 0], [0, 2.0, 0], [0, 0, 3.0]]
mat3 = em.Material(er= np.array([[1,2,3],[4,5,6],[7,8,9]]))
> εr = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

To use frequency and/or coordinate dependent properties we have to use the special classes `FreqDependent`, `CoordDependent` and `FreqCoordDependent`. Each can also return either of the three formats. For a scalar value submit the `scalar` parameter as function, for the rank-2 tensor diagonal, use the `vector` argument and for the full 2-rank tensor use the `matrix` property. The `CoordDependent` and `FreqCoordDependent` classes also require the user to define a `max_value` property because the current implementation does not apply mesh discretization based on the local value. It has to know a single maximum value.

The following examples are random functions. They only indicate the syntax

python
```python
er_property = em.FreqDependent(lambda f: 2*f/1e9)
er_property = em.CoordDependent(max_value = 2.0,
              scalar= lambda x,y,z: np.sqrt(x**2 + y**2 + z**2)*2)
```

4.7.2.1 Thermal properties

Since version 2.6 with the introduction of the Thermal solver, there are also the properties:
- `cond_thermal`: Thermal Conductivity [W/mK]
- `specific_heat`: Specific heat capacity [J/(kg K)]
- `density`: Mass density [kg/m³]

The thermal conductivity can also be provided in a full conductivity tensor.

4.7.3 Material library

EMerge contains an extensive material library that is checked and updated regularly. The current library contains almost 300 materials including metals and dielectrics.

! WARNING !
Many of these materials have to be verified and may have different values than expected. Always check the values used if precise values are important!

You can access these materials in `em.lib`. For example, FR4 can be found under `em.lib.DIEL_FR4`.

4.7.4 Assigning materials to surfaces

Materials assigned to surfaces will automatically cause EMerge to create boundary conditions for you in case that the materials are conductive and conductive above a bulk conductivity specified by the settings (See Section 4.2). If the conductivity is above this level, a SurfaceImpedance boundary condition will be assigned. If the conductivity is yet above another level, PEC will be assigned. By default, this level is above any known material. The `em.lib.PEC` material has a conductivity set above this level.

This boundary condition assumes that the thickness of the surface is at least above the skin depth so that a reduced resistance caused by very thin materials is not included in the simulation. To have more control over the resistance, specify your own boundary condition for these surfaces.

SurfaceImpedance boundary conditions take measurably more time to compute than PEC. Thus, if losses are not relevant, make sure to use the PEC material whenever possible.

! WARNING !
SurfaceImpedance boundary conditions are **only** valid for **external** boundaries. Internal surfaces like thin pieces of metal that model PCB traces for example should **not have the SurfaceImpedance boundary condition assigned**. To model losses accurately, please use actual thickness instead with a SurfaceImpedance around it or the ThinConductor boundary condition which exists since 2.7.[4]

[4] When using the PCB class you can use the `thick_traces=True` property in the constructor.

In the future a special boundary condition to model thin conductors will be added instead.

4.7.5 Appearance

You can change how your material shows up in the PyVista viewer.

- **color** - To change the color set the argument `color="#a1b2c3"` to some Hexadecimal string.
- **opacity** - How translucent the material is. The value 1.0 will show up as fully opaque and 0.0 is completely transparent. Some transparency is always desired.
- **_metal** - If you set `_metal=True`, the material will be rendered with a nice shiny look!

4.7.6 Drude model

EMerge offers a convenient way to construct conductors of which the properties are defined by the Drude my_model. The model takes the following properties:

- `conductivity: float` - σ_b - Bulk conductivity [S/m]
- `tau: float` τ - Collission Time
- `er: float` ε_r - Relative permittivity
- `ur: float` μ_r - Relative permeability

Based on these values, the conductivity is computed as a function of frequency:

$$\sigma(f) = \frac{\sigma_b}{1 - j2\pi f\tau}$$

The material can be generated using the function:

```python
material = em.Material.drude_model(...)
```

4.8 Mesh generation

The core of mesh generation in EMerge is all done by the library GMSH. EMerge wraps core functionality of GMSH in its own objects that inherit from `GeoObject`. Once the geometry is defined after calling `.commit_geometry()`, the mesh is ready to be generated by GMSH[5].

Certain mesh settings can be defined by the user by interacting with the `.mesher` attribute of your `Simulation()` class instance. Below we will provide an overview of the relevant options:

- `.set_domain_size()` - Set the maximum mesh size inside a domain.
- `.set_face_size()` - Set the maximum size on a face.
- `.set_boundary_size()` - Allows the user to change the mesh size along the boundary of a surface. Useful to call on PCB traces that needs a finer mesh along the edge.
- `.set_size()` - A generic method to set the size inside/on a point, line, surface or volume.

EMerge will always set the maximum size inside some domain D using the following relation:

$$s_{max} = \frac{c_0}{f_{max}} \frac{1}{\sqrt{\Re(\mu_r)\Re(\varepsilon_r)}}$$

An example of setting the size is shown below:

```python
my_model.commit_geometry()
my_model.mw.set_frequency_range(...)
my_model.mesher.set_boundary_size(my_face, 0.002, growth_rate=5)
my_model.generate_mesh()
```

4.8.1 Mesh specification function

Below is a list of functions to specify or modify the meshing behavior of EMerge:

- `.set_boundary_size()`: Specify the size on the boundary of a geometry. The boundary of a volume is its outside surface, the boundary of a face is its perimiter edge etc.
  - The `growth_rate` parameter controls how quickly the mesh size is allowed to increase away from the boundary. `1.0` means no increase.
- `.set_face_size()`: Specify the size on an entire face
- `.set_domain_size()`: Specify the size inside a volume.
- `.set_size()`: Specify the size inside a geometry of arbitrary dimension.
- `.set_pec_face()`: Specify that inside a face, the mesh size can be arbitrarily big.
- `.set_curved_boundary_meshing()`: Specify in how many sections curved boundaries have to be meshed at minimum.
- `.set_trace_refinement()`: Automatically improve the mesh quality on RF PCB traces.

4.8.2 Trace meshing

PCB traces can be meshed finer with adaptive mesh refinement but often, manual size settings are faster to implement and can get you directly towards the right answer. The following settings for the coupled line filter example yields the following mesh result:

```python
my_model.mesher.set_boundary_size(stripline, 0.25 * mm, growth_rate=10)
my_model.mesher.set_pec_face(stripline)
my_model.mesher.set_face_size(p1, 0.5*mm)
my_model.mesher.set_face_size(p2, 0.5*mm)
```

[5] In order for a mesh to be generated, certain physics aspects might need to be defined. For the Microwave physics, a crucial parameter to be defined is the frequency range. You can set the frequency range by calling either `.set_frequency()` or `.set_frequency_range()` on the `Simulation.mw` attribute.

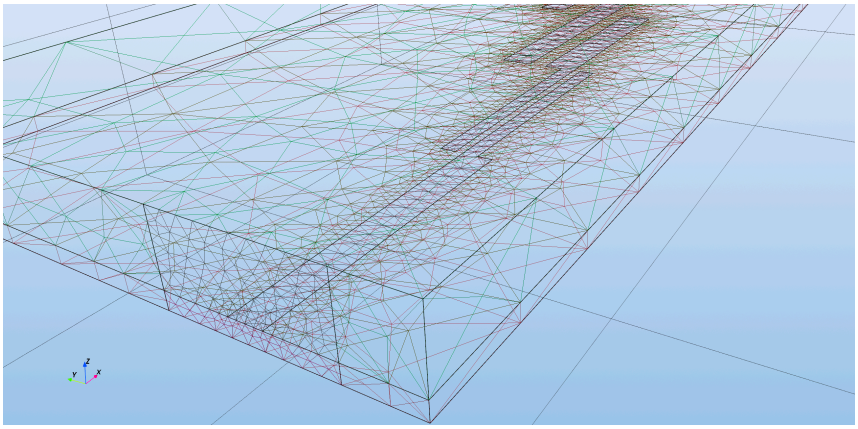


Figure 28: The surface mesh of the stripline traces.

4.8.2.1 Trace Refinement

Another option is to use the `.set_trace_refinement()` function. This function will look for line segments that are parallel and close together and apply a mesh size constraint based on the distance.

The trace refinement function applies to a set of conductors. The extra parameter is the so called **quality factor**. This factor is by default 3 and represents the size of the mesh as a function of the gap between two lines. If the quality factor is 3 and the gap between two lines is 1mm then the mesh size there will be set at 0.33mm.

The following code with yield the following mesh:

```python
my_model.mesher.set_trace_refinement(stripline)
my_model.mesher.set_pec_face(stripline)
my_model.mesher.set_face_size(p1, 0.5*mm)
my_model.mesher.set_face_size(p2, 0.5*mm)
```

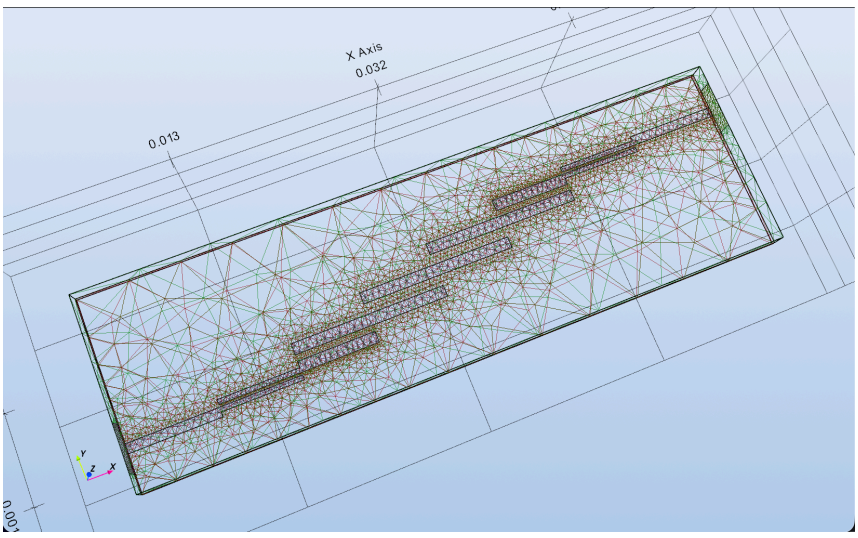


Figure 29: The surface mesh of the stripline after using trace refinement..

4.9 Boundary Conditions

Boundary conditions in EMerge are specific to each physics. The boundary conditions may be created by calling the appropriate method on the specific `BoundaryConditions` instance of your physics:

```python
my_model = Simulation()
#code
my_model.mw.bc.PEC(my_face_selection)
```

Besides passing selections, it is also possible to pass 2D geometries (instances of GeoSurface) to a face-based boundary condition.

Boundary conditions must, at all time, be defined after the boolean fragment of the geometry. This happens at least when the method `.commit_geometry()` is called. That method will automatically be called once you call `.generate_mesh()`. EMerge will not be able to properly associated faces to them if a boolean fragment still has to occur.

It is best practice to define all your boundary conditions after mesh generation.

If you need more help with generating selection, you can consult Section 4.4.

The microwave physics boundary conditions are discussed in Section 5.1.

The heat condution boundary conditions are discussed in Section 6.3.

4.10 Running Simulations

Each physics in EMerge[6] has its own solution methods. For the Microwave physics module, the most important ones are the `.run_sweep()` method, `.eigenmode()`, `.modal_analysis()` and `.run_scattered()`. Calling such a function directly starts the simulation process.

[6] Only Microwave physics is currently implemented in EMerge

4.10.1 Selecting a solver

You can force which solver EMerge uses by using the `.set_solver()` method. You can either construct your own `Solver` class instance or use the `Enum` called `EMSolver` which shows a list of known EM solvers. You have access to this function in both your Simulation class and the physics solveroutine.

```python
my_model.set_solver(em.EMSolver.UMFPACK)
  # or
my_model.mw.solveroutine.set_solver(em.EMSolver.UMFPACK)
```

4.11 Saving and Loading

You don't always want to rerun a long simulation if it turns out you made an error in your post-processing step. In order to simplify this workflow, EMerge has a functionality to save and load simulation data.

To save simulation data, you simply set the `save_file=True` argument in the `Simulation()` class and call `mymodel.save()` afterwards. Doing this will cause EMerge to automatically create a folder in the current path and store all the data inside of it. The file name will be based on your simulation name (first argument you set). Then you can access the same data by running the same or a different Python script with the argument `load_file=True`.

The simulation data can be accessed as following.

```python
## FILE 1
import emerge as em

my_model = em.Simulation('MySimMode', save_file=True)
# setup simulation
data = my_model.mw.run_sweep()
my_model.save()
```

```python
## FILE 1
import emerge as em

my_model = em.Simulation('MySimMode', load_file=True)
# setup simulation
data = my_model.data.mw
```

If you forget to call the save method, EMerge will try to save on exit but it has been noticed that the `atexit` module in Python is not called in time to save data without corruption. Another option for a graceful save even if your simulation crashes is by using a context manager for your simulation:

```python
## FILE 1
import emerge as em

with em.Simulation('MySimMode', save_file=True) as my_model:
  # setup simulation
  data = my_model.mw.run_sweep()
```

After running all the code inside the context block (with or without crash) the `__exit__` dunder method will automatically call the save method. If you want to explicitly save the file you can use the `.save()` function.

4.11.1 Caching simulations

To simplify the process of saving and loading, the Simulation object has to handy functions called `.cache_build()` and `.cache_run()`. These can be used to let EMerge detect if you have to rerun your simulation or not.

You can use `.cache_build()` to only rerun your simulations if the entire build code + simulation call is identical. It knows when to rerun a simulation by simply reading the Python code and checking if its 1-to-1 the same.

```python
import emerge as em
from emerge.plot import plot_sp
my_model = em.Simulation('cache_example')
if m.cache_build():
  box = em.geo.Box(0.02286, 0.05, 0.01016)
  my_model.mw.set_frequency_range(8e9, 10e9, 31)
  my_model.generate_mesh() #test
  my_model.mw.bc.RectangularWaveguide(box.face('front'), 1)
  my_model.mw.bc.RectangularWaveguide(box.face('back'), 2)
  my_model.mw.run_sweep()

data = my_model.data.mw
plot_sp(data.scalar.grid.freq, [data.scalar.grid.S(1,1),
data.scalar.grid.S(2,1)])
my_model.save()
```

A disadvantage of this use is that on later runs, variables like `box` are not defined anymore as that entire section in `.cache_build()` is not executed. Sometimes this is exactly what you want but maybe not always. You can also just rerun the actual `run_sweep()` section by using:

```python
import emerge as em
from emerge.plot import plot_sp
my_model = em.Simulation('cache_example')

box = em.geo.Box(0.02286, 0.05, 0.01016)
my_model.mw.set_frequency_range(8e9, 10e9, 31)
my_model.generate_mesh() #test
my_model.mw.bc.RectangularWaveguide(box.face('front'), 1)
my_model.mw.bc.RectangularWaveguide(box.face('back'), 2)
if my_model.cache_run():
  my_model.mw.run_sweep()

data = my_model.data.mw
plot_sp(data.scalar.grid.freq, [data.scalar.grid.S(1,1),
data.scalar.grid.S(2,1)])
my_model.save()
```

Because you didn't call the `run_sweep()` method, your simulation date from previous runs can be retrieved from `my_model.data.mw`.

! WARNING !

While active, this feature is instable and might not yield the behavior that you want. Alays make sure to add `.save()` at the end of your script to ensure that your data is properly stored!.

4.11.2 Solvers and Solve Routines

Each physics module in EMerge has its own `SolveRoutine` object. For the microwave physics, this is in the attribute of the physics called `.solveroutine`. The solveroutine takes care of all the steps needed to solve your problems.

First there is the `.solve()` method which finds the solution to a sparse linear system:

$$Ax = b$$

Where $\boldsymbol{A}$ and $\boldsymbol{b}$ are known and $\boldsymbol{x}$ is to be solved. The Physics module takes care of the assembly of the matrix $\boldsymbol{A}$ and vector $\boldsymbol{b}$ and also interprets the meaning of the solution vector $\boldsymbol{x}$. THe second method is `.eig()` which finds the solutions to the generalized eigenvalue problem:

$$\boldsymbol{A}x = \lambda \boldsymbol{B}x$$

This is used for example when running an Eigenmode analysis.

For the solving of the system of expressions $\boldsymbol{A}x = \boldsymbol{b}$ EMerge has several `Solver` options.

- **SuperLU (SciPy)**: A single threaded left looking supernodal direct solver. (In stock installation)
- **UMFPACK (scikit-umfpack):** A single threaded multi-frontal direct solver. Requires manual installation (multi-processing only)
- **PARDISO**: A parallel direct solver.[7]
- **cuDSS**: NVidias proprietary direct solver. Can be installed with the latest machines using `pip install emerge[cudss]`. Requires a modern NVidia card with sufficient VRAM.
- **emerge-aasds**: Special single threaded direct solver for Apple ARM machines. Install using `pip install git+https://github.com/FennisRobert/emerge-aasds`

[7] PARDISO only works on machines with either Intel or AMD architectures

The solvers are conveniently packaged in the enumerator called `EMSolver`. You can both change and disable solvers using the following commands:

```python
my_model.set_solver(em.EMSolver.PARDISO) # Enable
my_model.mw.solveroutine.disable_solver(em.EMSolver.SUPERLU) # Disable
```

Because PARDISO is already multi-threaded it will not be used for distributed solves. If at any point you turn on parallel-processing, EMerge will always default to SuperLU as its direct solver. For more performance on larger models, UMFPACK, CuDSS and Apple Accelerate solvers are available and recommended. It requires entry point protection because it can only work with Python's multi-processing. More on this in Section 10.1.

EMerge currently does not support iterative solvers like GMRES and BiCGStab because it does not have a good implementation of appropriate pre conditioners required for the type of problems being solved in Electromagnetics. On top of this, direct solvers have the advantage of solving all other ports with almost no extra time as it may reuse the LU decomposition.

4.12 Simulation data

Simulations run in EMerge return `SimulationDataset` objects that contain the results of your simulation. The structure of these datasets is a bit complex to navigate. However, there is a reason for the organization.

- The dataset will remain valid when new physics are added such as heat transfer in Solids.
- The dataset functions for frequency sweeps, frequency sweeps plus parametric sweeps and even optimizations with unstructured data when added to EMerge.

EMerge stores each single simulation result as a separate entry in a long list with all parameters it was run for.

For all scalar output variables such as S-parameters, you can call a function `.grid` that will put those results into an N-dimensional data structure corresponding to your simulation setup. We will now go over the simulation data classes in EMerge and how to use them.

4.12.1 The SimulationData object

Each Simulation object has a single SimulationDataset instance in `.data` that contains all your simulation data. This class has three attributes that categorizes the data.

```python
my_moddel.data.globals: dict[str, Any]
my_moddel.data.mw: MWData
my_moddel.data.sim: DataContainer
```

The **globals** attribute is meant for users to store arbitrary parameters and data in. You can use this to store anything that you want access to later.

```python
data.globals['myname'] = my_data
```

! WARNING !
Make sure that all data you store is pickle-able, otherwise saving files is not possible.

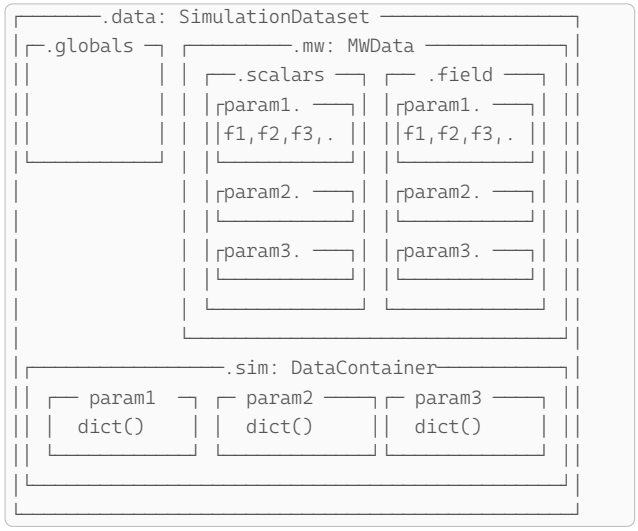
The data generated by all RF/Microwave simulations is stored in the `.mw` field.

Lastly, there is the `.sim` attribute. The `.sim` field can be used to store all other information that is not directly associated with the output of a simulation itself but information that went into executing it.

A simple rule is:

1. If the data is generic on your entire simulation, it should be in `.globals`
2. If the data is generic to a specific parameter sweep iteration it is in `.sim`
3. All data related to a specific physics simulation is contained in `.mw` or others once more physics are implemented.

Another way to think of the organization of the data is displayed in this diagram: In general there are three levels to simulation data in EMerge. These are explained in the following diagram.



Listing 4: The architecture of the SimulationDataset class. Each frequency result gets grouped in a single set which is valid for a given geometry with specific parameters. The 3D field data resides in the .field attribute and the scalar data in .scalar. The same parametric organization also exists in parallel in the .sim DataContainer. You talk to both the .scalars and .sim fields in a similar way.

4.12.1.1 The .sim field

Without going into detail of the class organization, you can think of the `.sim` field as a dictionary that is specific to each simulation iteration. Later parameter sweeps will be discussed in Section 4.12.2. For now, what is crucial is that each time you run a separate parameter sweep instance, EMerge will generate a new dictionary for you to store data in. You can simply put anything inside it by calling:

python
```python
model.data.sim['myname'] = my_data
```

To access the data later, you have to specify which permutation of the parameter sweep you need. Here is an example of that:

python
```python
for param1, param2 in my_model.parameter_sweep(True,
                                               pname1=(1,2,3),
                                               pname2=(4,5,6)):
    # setup your simulation
    my_model.data.sim['myname'] = my_data

my_recalled_data = my_model.data.sim.select(pname1=2, pname2=6)['myname']
```

Notice that the parameter names we chose in the parameter sweep correspond to the ones we see when retrieving our data. Each time a new parameter sweep is started, EMerge automatically creates a new dictionary for you to put the data in and puts that as the current “active” dictionary. Internally this is the `.default` field. To simplify things, the following calls will grant you access to the same field

python
```python
m = em.Simulation('MyModel')

box = em.geo.Box(1,1,1)

m['mybox'] = box
m.data.sim['mybox'] = box
m.data.sim.default['mybox'] = box
```

4.12.1.2 The .mw field

The microwave data is contained in the `.mw` field of our simulation data. This means all data that is generated through a single or multiple calls to `.run_sweep()`, `.eigen_modes()` and/or `.modal_analysis()` inside a simulation file. Each entry internally is just stored as a list. For the heat-conduction module the field is `.hc`.

The data is split up in two sets, scalar data and field data. These are put in the fields:

- `.scalar`: All scalar data such as the frequency point, S-parameters, ko etc
- `.field`: The solution field. This contains the solution vector with basis-function amplitudes and the mesh needed to compute the field at various points

Both the `.scalar` and `.field` are instances of the `BaseDataset` class. We will now explain how these are used. Both the `.field` and `.scalar` object are just a list of data entries. Whenever you run a frequency sweep, each sweep will store its data into a new entry in the dataset. The only exception are port sweeps, the output fields for each port are contained in the same entry.

The simulation frequency is stored as a value for the `freq` variable. Any simulation will thus at least have an entry for the simulation frequency. If you run a parametric sweep, those variables will be added to the list of variables. Staying with just a frequency sweep for now, your data may be accessed or found as following.

4.12.1.3 Selecting data by number

You can find a specific solution simply by an index. The following will select the results of the second simulation run. If its just a frequency sweep, this would be the second frequency point.

python

```python
S11 = my_model.data.mw.scalar[2].S(1,1)
Efield = my_model.data.mw.field[2].interpolate(...).E
```

4.12.1.4 Selecting by parameter

If you have a specific frequency say 1.2GHz that you ran the simulation for, you can get the results using:

python

```python
S11 = my_model.data.mw.scalar.select(freq=1.2e9).S(1,1)
Efield = my_model.data.mw.field.select(freq=1.2e9).interpolate(...).E
```

If you don't know exactly what the frequency value is or struggle to find it due to floating point rounding, you can approximate it using `find`. This will simply return the closest match

python

```python
S11 = my_model.data.mw.scalar.find(freq=1.2e9).S(1,1)
Efield = my_model.data.mw.field.find(freq=1.2e9).interpolate(...).E
```

Very often you want to get your results in the array structure that you simulated. To do this you can generate a `gridded` version of your dataset. This **only** works for scalar data

python

```python
freqs = my_model.data.mw.scalar.grid.freq
S11 = my_model.data.mw.scalar.grid.S(1,1)
```

If these lines get too long for your liking, you can simply name the grid data:

python

```python
my_grid = my_model.data.mw.scalar.grid
freqs = my_grid.freq
S11 = my_grid.S(1,1)
```

Notice also that the `.mw` dataset is what is returned by `.run_sweep()` so usually you can just do:

```python
my_data = my_model.mw.run_sweep()
my_grid = my_data.scalar.grid
freqs = my_grid.freq
S11 = my_grid.S(1,1)
```

If you select just a single instance, the `.S(i,j)` method will just return a single complex number. If you call this on the `.grid` field you will get a `(N_freq,)` sized Numpy array with all your data. If you ran a parameter sweep with permutation space (N,M) for example, you will get a `(N,M,N_freq)` shaped N-dim array. We will discuss more about this now.

4.12.2 Parameter Sweeps

Parameter sweeps are done simply by calling the `.parameter_sweep()` method on the `Simulation` object. The first argument of this function is a mandatory boolean that tells EMerge if the geometry can be reused or if it has to be reconstructed. If it can be reused, the mesh will not be reset. This is typically the case if you sweep over boundary condition parameters or material assignments. In those cases often the mesh can be kept the same.

The second arguments is simply a list of keyword arguments. The name of the keyword is the name of the parameter and the second must be an array like object that contains the parameter values.

The `.parameter_sweep()` function will then iterate over all possible permutations of those parameters. The parameter sweep object then returns the parameter for each loop **in alphabetical order** For example, the following parameter sweep will produce the following iterations.

```python
import emerge as em
my_ = em.Simulation('pcb sim')
for pA, pB in m.parameter_sweep(True, paramB=[1,2,3],
paramA=[400,500,600,700]):
> the name paramA and paramB here…(see note below)
    # iter1: pA = 400, pB = 1
    # iter2: pA = 500, pB = 1
    # iter3: pA = 600, pB = 1
    # iter4: pA = 700, pB = 1
    # iter5: pA = 400, pB = 2
    # iter6: pA = 500, pB = 2
    # iter7: pA = 600, pB = 2
    # iter8: pA = 700, pB = 1
    # iter9: pA = 400, pB = 3
    # iter10: pA = 500, pB = 3
    # iter11: pA = 600, pB = 3
    # iter12: pA = 700, pB = 1
    my_.mw.set_frequency_range(1e9, 2e9, 21)
    pecbc = my_.mw.bc.PEC(my_face)
    data = my_.mw.run_sweep()
    data['pec'] = pecbc
data.scalar.find(paramA=500, paramB=3).S(1,1)
pec_500_3 = data.sim.find(paramA=500, paramB=3)['pec']
> …is the same paramA and paramB used here.
```

! WARNING !

The parameters will always be returned in alphabetical order. The order in which keyword arguments are provided cannot be used in Python.

All simulation results generated in this sweep are automatically stored for the parameter iteration. The name `paramB` for example will be used internally as a string and passed to MWData class so that it can store it for each iteration. In the last line you see how you use the same string name to retrieve the data.

Because in this case your length vs width sweep is a structured dataset, you can grid your output data in N-dimensional arrays.

python
```python
# Previous code

freq = data.scalar.axis('freq') # Only the freq values
paramA = data.scalar.axis('paramA') # Only the paramA axis
paramB = data.scalar.axis('paramB') # Only the paramB axis
S11 = data.scalar.grid.S(1,1) # The total Ndimensional dataset
> S11.shape = (4,3,21) Dims: paramA x paramB x Frequency
```

The order of the parameters is **always alphabetical**. However, the frequency axes will always be the last.

You can also select all datasets with a single condition using the `.filter` method. For example:

python
```python
S11  = data.scalar.filter(freq=1.2e9)
```

Will Look for all datasets that have a frequency exactly equal to `1.2e9`. You can select any field value using:

python
```python
Ez = data.field.select(freq=1e9, width=2*mm, length=6*mm).interpolate(...).Ez
```

5 Microwave Physics

This chapter will discuss all things specific to the Microwave physics module.

5.1 Boundary Conditions

The Microwave physics module has only boundary conditions that can be assigned to boundaries (2D elements). All boundary conditions in the microwave module are "exclusive" which means that only one boundary condition may be assigned to any given boundary.

The following boundary conditions are available:

- `PEC`: Perfect Electric Conductor
- `PMC`: Perfect Magnetic Conductor
- `AbsorbingBoundary`: Absorbing or Radiation boundary condition
- `Periodic`: Periodic boundary condition
- Ports:
  - `RectangularWaveguide`: Rectangular waveguide wave port
  - `Coaxial`: Coaxial waveg port
  - `ModalPort`: Generic wave port (performs eigenmode analysis)
  - `LumpedPort`: Lumped port (internal to the domain only)
  - `FloquetPort`: Plane wave port for periodic cells
  - `ScatteredField`: For scattered field simulation plane wave excitation.
- `SurfaceImpedance`: For a finite conductivity of an external boundary face.
- `ThinConductor`: For thin metal traces with a finite conductivity.
- `LumpedElement`: Lumped impedance boundary condition
- `PML`: Not a boundary condition but if you are looking for it, they do exist (see Section 5.2.1)

5.1.1 Ports

Ports are a fundamental part of microwave physics. They are the responsible boundary condition for injecting energy into the simulation domain and letting it escape. In total EMerge supports five types of port boundary conditions:

- **Modal Port**
- Predefined modes:
  - **Rectangular Waveguide** and **coaxial**
  - **User Defined**
  - **Floquet Port**
- **Lumped Port**

We can categorize these port boundary conditions in two groups:

1. Internal ports: Only the lumped port. Are defined inside the simulation domain
2. External ports: All others. These are defined at the exterior boundary of your simulation domain.

Under the hood, each port is an implementation of a so called "Robin" boundary condition, also known as a "boundary condition of the third kind".

For non-lumped ports, it is required to know in some way or another what the electric field mode is and the propagation constant of the mode. There are two ways in which these mode fields could be determined. The simplest way applies to all ports with a known aperture field. Currently this includes the Rectangular Waveguide and the Floquet port (no grating modes). The second option is to use a eigenmode analysis to find the port mode.

5.1.2 Modal port

As mentioned before, the modal port is a port boundary condition that solves for the propagation mode itself in a modal analysis. This is an eigenmode study that has to be run before the frequency domain study.

By default, EMerge will run these modal analysis studies automatically when running the frequency domain study. Effectively, if a modal port doesn't have any port modes defined

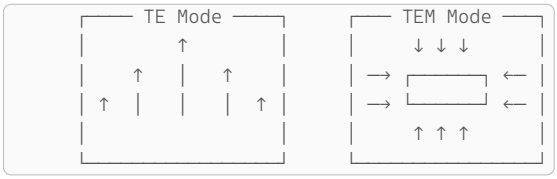
for it, it'll try to compute them with default settings. It is also possible to first compute the port modes manually.

There are quite a lot of settings to consider when executing a modal analysis.

There are two important properties to figure establish before executing a boundary mode study.
- If the port supports a TEM-field (instead of TE or TM)
- If the port consists of multiple different dielectric properties

All TEM modes must by consist of at least two unconnected conductors. Ports with more than 2 conductors don't work as well currently. This will be implemented later.

Examples are striplines, microstrip lines and coax etc. If a port is analyzed as a TEM port, the propagation constant is expected to be greater than or larger than the free space propagation constant (depending on the dielectric materials involved). For non-TEM ports like waveguides, you often are dealing with a single conducting surface (such as a rectangular waveguide).



Listing 5: An example of a TE mode left and a TEM mode right.

If you execute a modal analysis manually using the `.modal_analysis()` method of the Microwave module, you can set `modetype='TEM'` to analyse the mode as a TEM mode. The solver will take a best guess for the approximate range of possible out-of-plane propagation constants. On top of that, after solving, an integration line connecting the two conductors will be established that is subsequently used to compute a voltage. By default voltage together with the out of plane power flux is used to compute the ports characteristic impedance (More in Section 5.1.2.2):

$$Z_{0,TEM} = \sqrt{\frac{V^2}{P_{out}}}$$

The integration line will always be taken as the shortest path.

If your port has more than 2 conductors, you can specify which two objects make up the positive and negative terminal using the `.set_terminals()` method. Additionally, you can manually provide an integration line by specifying two coordinates using the `.set_integration_line()` method.

For non-TEM modes, no Zo will be computed.

If you don't execute the modal analysis manually, the solver will base its decision on the `modetype` property of the `ModalPort` boundary condition you can set when defining it:

```python
port_obj = my_model.mw.bc.ModalPort(port_faces, 1, modetype='TEM')
#or
model.mw.modal_analysis(port_obj)
```

Multiple Materials

The other property that is important to define is if the port consists of multiple different dielectric materials. While it seems possible to infer this from the material properties on the boundary, it is not clear apriori if the different materials are even part of the modal field. For example, a bit of dielectric material could exist inside a shielded conductor that does not take part of the dominant mode. Thus, human input is required. The reason this matters is because for all ports with homogeneous dielectric properties, the field-mode only needs

to be computed at one frequency. The propagation constant at all other frequencies can then be extrapolated using the expression:

$$k_z(f) = \sqrt{k_{z,ref}^2 + \left(\frac{2\pi f}{c_0}\right)^2 \left(1 - \left(\frac{f_{ref}}{f}\right)^2\right)}$$

This property can be defined by setting:

```python
port_obj = my_model.mw.bc.ModalPort(port_faces, 1, mixed_materials=True)
```

By default this property is set to False which means that EMerge will not rerun the modal analysis at every frequency where it actually should.

Mode analysis setup

Depending on the size of the problem, finding the appropriate boundary modes may prove difficult. In cases where the automatic determination fails or is too slow, a manual modal analysis can help. The following properties of the `modal_analysis()` function are relevant:

- `nmodes: int` - The number of modes to solve for. When using an indirect solver, this can shorten the total simulation time. All modes are sorted by total energy of the solved mode. The highest energy mode will be selected as first mode. For direct solve setup this setting wont matter too much. The selected mode can be changed by changing the field `.selected_mode = 1 or higher`.
- `direct: bool` - Defines if the direct eigenmode solver is used. Direct solvers are guaranteed to find the right modes but are very slow for large problems. The iterative solver has to search for the mode in a region defined by the expected propagation constant and is much faster. If the iterative solver is used, the `AutomaticRoutine` searches around the predefined expected propagation constant. The iterative solver can find many spurious modes around any propagation constant so an intelligent search must be used. The automatic routine will go for the direct solver if the problem is small enough.
- `target_kz` - Defines the out of plane propagation constant to search at. You can use this number with the iterative solver if you have a good guess for the expected mode constant.
- `target_neff` - Defines the search area by the effective mode constant. This is defined as: $n_{eff} = \sqrt{k_z \,/\, k_0}$.
- `freq` - Defines the frequency at which the modal analysis is done. In case of a homogeneous port, this frequency will define at which frequency the port mode is computed.
- `number_of_modes` - Defines how many modes are to be included in the port sweep. Defaults to `1` for just the first mode.

5.1.2.1 Aligning modes

In some cases we wish to make sure that the mode fields computed (in case of multi-mode ports) are aligned in the right direction. This can be done by providing a list of Axis objects (or vector tuples). The modes will be ordered such (in numbering) that they correspond to the order of vectors. If for example a circular waveguide has a mode in the +Z and +X direction they can be sorted using

```python
mode.align_modes(em.XAX, em.ZAX)
```

Besides aligning different modes, the `align_modes` function also makes sure that the resultant modes consistently point in the same direction between simulations. This is because for eigenmode solvers, the field phase $\psi = \exp(j\omega t)$ is a degree of freedom that it cannot keep consistent.

5.1.2.2 Port Impedance

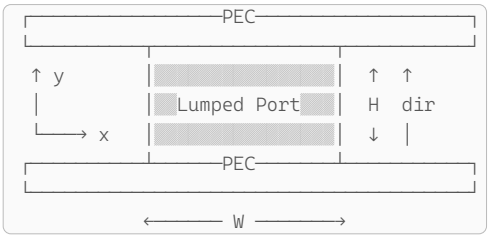
Since version 2.4.1, port impedance calculation options have been added using the `impedance_definition` argument which can take the following values:

- `PV`: Power Voltage definition: $Z = V^2/2P$

- `PI`: Power Current definition: $Z = 2P/I^2$
- `VI`: Voltage Current definition: $Z = V/I$

5.1.3 Lumped port

A lumped port is a convenient port to used inside a domain. It can be used whenever the energy is locally injected as a field that can be assumed as uniform within a surface. The lumped port must be placed between two conductors. To compute the proper mode field, the solver needs to know what the height is of the port (distance between the conductors it is connected), the width (the distance along the orthogonal direction) and the characteristic impedance of the source injecting the power in the domain. Lastly, the user needs to define the field direction as a vector in 3D space. This is the vector along the height direction.



Listing 6: A lumped port illustrated to connect two PEC surfaces with the indication of the width (W) and the port height (H) and the field direction (dir).

If you construct a lumped port surface with the `PCB` object, the information of the port geometry will be stored internally in the surface and passed automatically to the lumped port object. Only the characteristic impedance then needs to be defined.

```python
port_face = layouter.lumped_port(layouter.load('port_reference'))

## setup

port_bc = my_model.mw.bc.LumpedPort(port_face, 1, Z0=50.0)
```

One might also want to manually define a lumped port on a given surface. In this case all the properties described above must be defined.

```python
port_bc = my_model.mw.bc.LumpedPort(port_face, 1, width=width, height=height,
  direction=em.ZAX, Z0=50.0)
```

5.1.4 User defined Ports

Since version 1.1, EMerge also allows for user defined ports. The class is called `UserDefinedPort`. It is based on the following crucial parameters:

- `Ex: Callable:` The Ex-field as a function
- `Ey: Callable:` The Ey-field as a function
- `Ez: Callable:` The Ez-field as a function
- `kz: Callable:` The out of plane propagation constant function
- `modetype: "TEM","TE","TM"`: The mode type.

The field functions are a function of k_0 and the global x,y and z coordinates and have the following default values.

$$E_{x(k_0,x,y,z)} = 0$$
$$E_{y(k_0,x,y,z)} = 0$$
$$E_{z(k_0,x,y,z)} = 0$$
$$k_{z(k_0)} = k_0$$

For the field components it is important to always return arrays, even if the field is a constant.

```python
def my_Ey_function(k0, x, y, z) -> np.ndarray:
  return Ey * np.ones_like(x)
```

5.1.5 Differential ports

It is possible two combine two single ended ports into one differentially excited port. This is entirely done in post-processing. You simply excite the simulation with two separate ports and then call:

```python
sim_data = my_model.mw.run_sweep()

gridded = sim_data.sclar.grid
gridded.combine_ports(i1,i2)
```

This will combine two ports with indices i1 and i2 into one port with a differential and common mode S-parameter. These will be placed in the same two numbers:

- Index i1 will become the differential i1+i2
- Index i2 will become the common-mode i1+i2

You can also call `.combine_ports(i1,i2)` on a simulation field data. Then if you set the excitation of port 1 or port 2 it will automatically excite the differential and common mode ports.

5.1.6 Multi-mode sweep

If ports support multiple modes with the same out-of-plane propagation constant, these will be automatically included in the port boundary sweep. Instead of having the S-parameter index as simply being the port number, the modes will be categorized as a floating point index like this `grid.S(1.2, 1)` for port 1 mode 2 etc.

The Floquet ports will automatically sweep across both modes. For Modal ports you can tell EMerge to excite both modes setting the `number_of_modes=2` parameter in the constructor:

```python
sim.mw.bc.ModalPort(face, 1, modetype=..., number_of_modes=2)
```

After doing this EMerge will sweep the first N modes that are available sorted by out-of-plane propagation constant.

5.1.6.1 Manual mode settings

If we assume that some port has N mode fields all with the same propagation constant, we can in theory define each mode j on the port essentially as a sum of these independent modes i:

$$E_j(x,y) = \sum_{i=1}^{N} E_{0,i} E_{i(xy)}$$

In other words, the port modes could be just whatever fundamental modes are computed for the port or any linear combination between them.

If you specify the `number_of_modes` field, EMerge will automatically pick the fundamental modes as being the modes that are sweeped over.

You could also pick any linear combination of these modes but you would have to set these values manually.

To do so, create a dictionary where the keys start at the integer 1 which are mapped to tuples that contain complex ampltidudes of each of the modes that exist for the port.

Example
See example file: `demo16_differential_common_mode`

```python
port = sim.mw.bc.ModalPort(face, 1, modetype=...)
port.port_modes = {
   1: (1.0 + 0.0j, 0.0 + 0.0j),
   2: (0.0 + 0.0j, 1.0 + 0.0j),
}
```

The above example is effectively what would come out if one where to set the number of modes field to 2 as the first mode is just 1.0 times the first mode, and the second mode 1.0 times the second mode etc.

5.1.7 Surface Impedance and Thin Conductor

The surface impedance and thin conductor boundary conditions are used to give a resistive loss to a 2D boundary. Additionally, both the SurfaceImpedance boundary condition and ThinConductor will model the skin effect and surface roughness optionally.

They are to be used in two different cases:

Surface Impedance Use the `SurfaceImpedance` if one side of your boundary is touching a conducting surface or the outside of your simulation domain (E-fields on one side only)

`Thin Conductor` Use the `ThinConductor` boundary condition when your conductor is a thin metal trace for example in an EM environment. In other words if E-fields can exist on both sides.

The difference between these two is crucial. A finite conductivity means that the Electric field can be non-zero tangent to the surface. The boundary condition models the appropriate resistive loss.

If the surface is a microstripline for example the E-field on one side can be notibly different than the E-field on the other side because the trace is often thicker than the skin-depth. Because the E-field can be different on either sides you need a different Degree of Freedom for the E-field on either side.

! WARNING !

The ThinConductor boundary condition is new in 2.7 and uses a new degree of freedom splitting algorithm to separate the E-field on either side. It is possible that you run into errors when this occurs. If you suspect anything, please send a representative model to info@emerge-software.com so that the algorithm can be improved.

5.2 Open/Radiation Boundaries

In EM FEM software there are usually two ways to generate an open boundary:

1. Absorbing boundary condition
2. Perfectly Matched Layer

The Absorbing boundary condition has many different names. Sometimes its called an open boundary, scattering boundary, radiation boundary or absorbing boundary. They are usually all the same.

EMerge calls the an absorbing boundary condition and it can be assigned using `model.mw.bc.AbsorbingBoundary()`. It is the easiest to setup. The absorbing boundary condition can come in various types. The first and simplest is the first order boundary condition. It absorbs best under normal incidence and worse as the incident angle increases. One can also opt for a second order boundary condition, these come in various types that are tuned to absorb energy from a wider range of angles with the downside of decreasing broadside absorption. The order can thus be either `1` or `2` and the type any letter from the set `ABCDE`.

```python
abc = my_model.mw.bc.AbsorbingBoundary(my_sel, order=2, abctype='C')
```

The default option in EMerge is a second order boundary of type B. The graph below shows the reflection coefficient for various angles of incidence for each type:

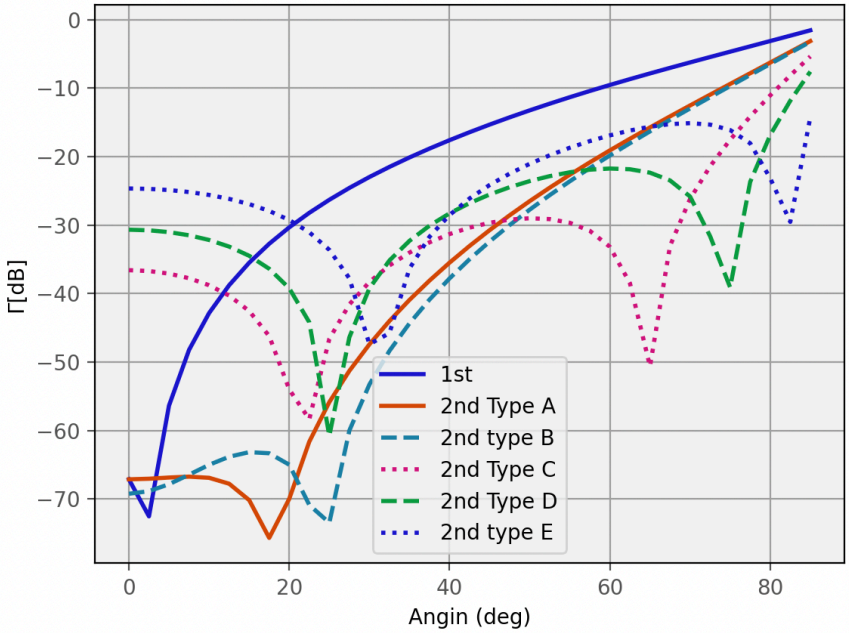


Figure 33: Various absorbing boundary conditions of different order and type.

5.2.1 PML Regions

The second option for generating open regions is the Perfectly Matched Later. This layer is made up of a special material with a tuned lossy dielectric constant and magnetic permeability. The latter suggests the existence of a magnetic current conductivity which of course can't exist. However, mathematically we can assign a complex value to the relative permeability μ_r.

EMerge currently only offers the `em.geo.pmlbox()` function to create both an air-box plus a series of PML boxes.

```python
air, *pml = em.geo.pmlbox(W,D,H,pos, top=True, bottom=True, ...)
```

The command as shows allows the user to turn **on** PML boxes on specific sides of the domain. EMerge will automatically also add the corner and edge boxes to connect the domains.

An important parameter to improve matching is the thickness and the number of mesh layers.

The thickness can generally be advised to be somewhere between half a wavelength and one wavelength. The number of mesh layers is usually best chosen between 4 and 6.

More mesh layers will yield an improved result, however, the number of mesh layers will determine the total maximum mesh edge size inside the PML domain.

$$ds_{\max} = \frac{T}{N_{\text{mesh}}}$$

Where $ds_{\max}$ is the maximum mesh element size, T the PML layer thickness and N_{mesh} the number of mesh layers.

The total number of mesh elements in a volume scales with $1 \,/\, \left(ds_{\max}^3\right) \propto N_{\text{mesh}}^3$. The number of mesh elements due to a thicker region only grows proportional to the thickness T.

Thus increasing the number of mesh layers may result in a huge number of mesh elements because the outer surface area is also meshed more finely. In these cases, it might work better to also increase the total PML size. This will improve matching and also prevent over-meshing. In the figure below you can see how increasing the number of mesh layers (left to middle) introduces a lot of mesh elements. Increasing the thickness (right) lowers the total number of mesh elements.

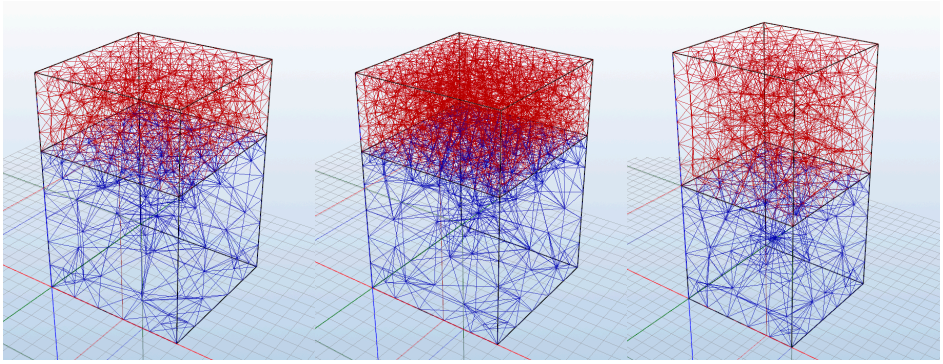


Figure 34: Three PML meshes (red) with left: 4 Mesh layers, middle: 6 mesh layers and right: 6 Mesh layer with twice the thickness.

5.2.2 Automatic open regions

You can quickly generate open boundaries by using the `em.geo.open_region()` method which creates a simple bounding box with some margins provided as optional arguments in the `open_region()` function. You can also add an open region with PML boxes by calling `em.geo.open_region_pml()`.

! WARNING !

The `em.geo.open_region()` method only creates the air-box. You still have to define the Absorbing Boundary Condition (ABC) afterwards.

5.3 Scattered Field Simulations

Since EMerge version 2.5 Curfman, a first implementation of the scattered field formulation was introduced. The scattered field simulation is one where you expose the simulation domain by a plane wave to compute the scattered field off of objects.

We will quickly discuss which ingredients are needed for a scatterd field simulation.

5.3.1 Geometry requirements

In terms of geometry, it is quite straight forward. All you have to do is enclose some object by an air box with sufficient marging. More distance is better in terms of the absorbption of EM radiation. However, this will also increase the simulation time and RAM requirements.

In many cases it is advised to have a separate boundary for the far-field integration. The external boundary of the simulation domain is less accurate than a boundary close to or even on the scattering object. A simple setup of a scattering metal sphere is shown in Figure 35.

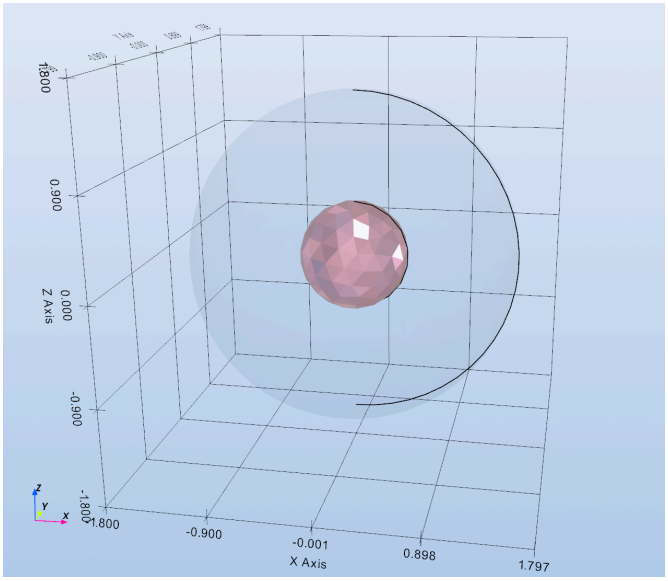


Figure 35: The Simulation geometry of a PEC sphere.

5.3.2 Boundary Condition

For a scattered field simulation you need only one boundary condition, namely the `ScatteredField` boundary condition. This boundary condition needs to be applied to the external terminating boundary of your domain, in this case the outside of the air sphere.

```python
scatter_bc = my_model.mw.bc.ScatteredField(air.boundary())
```

The boundary condition takes three arguments:

- `boundary`: The boundary assignment of the boundary condition.
- `power_density`: The time averaged power density of the plane wave in Watts per square meter. This is best kept at 1 $\frac{W}{m^2}$ for RCS simulations.
- `cs`: This is the coordinate system of the plane wave. Currently only used to determine the phase origin. The coordinate system axes are not yet used.

To change the incidence angle and polarization of the plane wave you have to manually add a set of variations using the `.set_excitations()` function. The exposed plane wave is defined by three angles: θ,φ and ψ(Polarization). EMerge will eventually support multiple

Example
See example file: `Demo20_RCS.py`

definitions for this plane wave but currently, only the `EA` definition is used. The expressions and diagram is shown below:

$$
\begin{aligned}
k_{x(\theta,\varphi)} &= k_0 \cos(\theta)\cos(\varphi) \\
k_{y(\theta,\varphi)} &= k_0 \cos(\theta)\sin(\varphi) \\
k_{z(\theta,\varphi)} &= -k_0 \sin(\theta) \\
E_{x(\theta,\varphi,\psi)} &= \sin(\theta)\cos(\varphi)\cos(\psi) - \sin(\varphi)\sin(\psi) \\
E_{y(\theta,\varphi,\psi)} &= \sin(\theta)\sin(\varphi)\cos(\psi) + \cos(\varphi)\sin(\psi) \\
E_{z(\theta,\varphi,\psi)} &= \cos(\theta)\cos(\psi)
\end{aligned}
$$

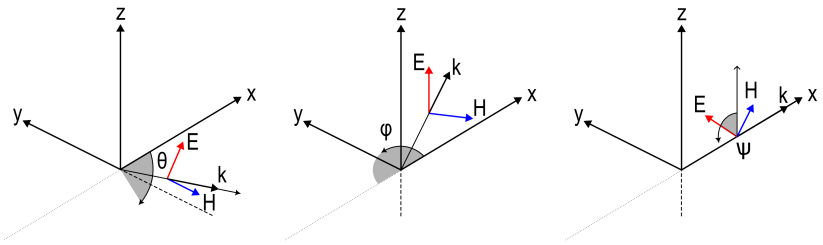


Figure 36: An illustration of the θ, φ and ψ axes of the default Elevation/Azimuth system.

As an argument for the `.set_excitations()` function you can either provide a scalar value or list/array of numbers. For multiple incident plane waves, it is advised to always use this function to iterate through different excitations since extra excitations are solved for about 1/40th the simulation cost after the first. You can consider each excitation as a different port mode.

5.3.2.1 Radius setting

For improved absorbing performance of the radiation boundary you can optionally provide a value for the radius property. This radius value should be the radius of the spherical surface in case a spherical air volume is used. This will apply a correction term that slightly improves the absorption of radiated waves.

```python
scatter_bc.radius = 0.15
```

5.3.3 Running the simulation

After setting up the boundary condition you can run the simulation by calling the `.run_scattered()` function which works very much the same as `.run_sweep()`. It is worth noting that scatterd field simulations typically run slower than normal simulations because due to the physics and geometry of typical scattered field simulations. This is to be expected.

5.3.4 Post processing

Post processing also works very similar to normal S-parameter like simulations. However, there are some pionts to be addressed.

First, as mentioned before, it is optimal to pick a far-field integration boundary as close to the scattered object as possible. Far-field calculations use the Stratton-Chu integrals which are exact solutions of the Maxwell's equations. They essentially allow to perfectly compute the E and H field outside an enclosed surface for all source terms inside the surface. This means that as long as your scattered object are contained in the mathematical surface, the far-field integration is exact. If you pick a boundary on the simulation domain, numerical phase dispersion can accumulate which yields errors in the final result.

In the case of a scattering sphere (as shown in Figure 35), you may put the integration surface on the boundary of the sphere!

python
```
field = sim_data.field[0]
ff3d = field.farfield_3d(scattering_object.boundary())
ff2d = field.farfield_2d(em.XAX, em.YAX, scattering_object.boundary())
```

5.3.4.1 Relative fields

In scattered field simulations you can decompise the total field solution in an addition of the background field and scattered field:

$$E_{tot} = E_{bg} + E_{scat}$$

The term scattered field and relative field are used interchangably. The solution you plot is always the total field. To plot the scattered field only you can call the property `.relative` on any field object:

python
```
total_field = data.field[0]
relative_field = data.field[0].relative
```

The relative field will evaluate the background field on the final field solution and subtract it from the total field.

For far-field integrations you **do not** have to use the relative field. The reason is that the background field is due to current sources that are outside the integration boundary. Therefore, this farfield integral automatically yields o external to the boundary.

python
```
d = my_model.display
d.populate()
d.add_field(field.relative.grid(N=1000).vector('E'))
d.add_field(field.relative.grid(N=10_000).scalar('Ey','real'),
  symmetrize=True)
d.show()
```

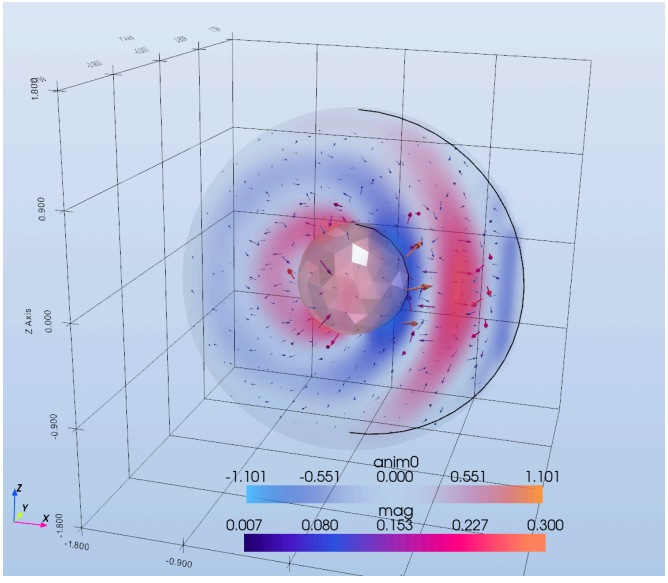
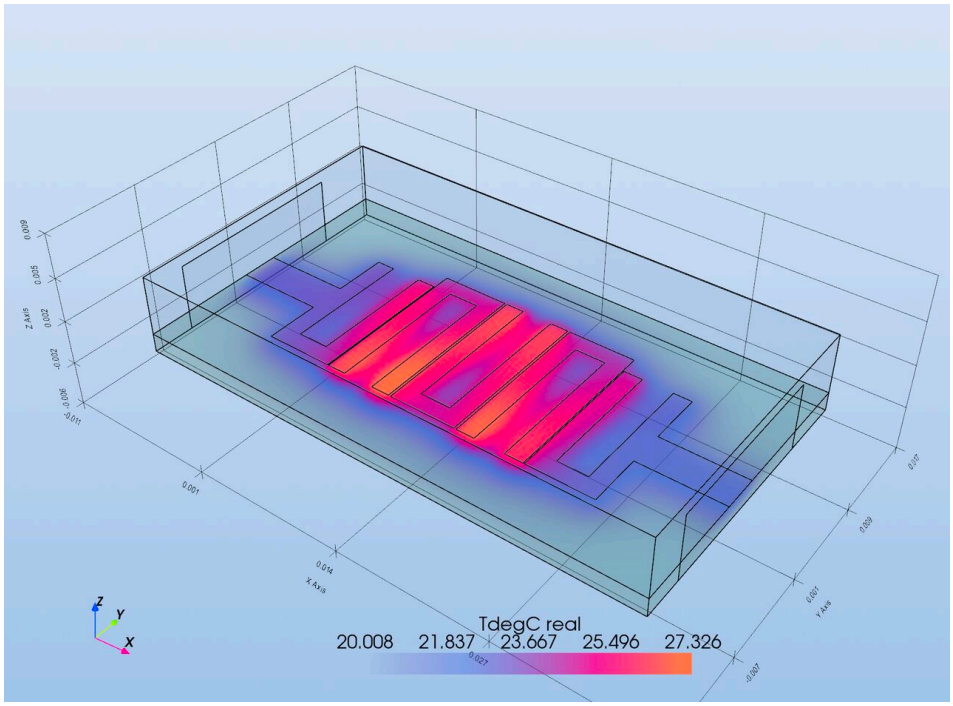


Figure 37: The scattered field off of a PEC sphere.



6 Heat Conduction Physics

In this chapter we will discuss how to use the heat conduction physics module also abbreviated with `hc`. Heat conduction physics involves only the conductive transport of heat through static materials. Notably, it does not include any flow of fluids or air.

The heat conduction interface is available under your `Simulation` object under `.hc`.

```python
mymodel = em.Simulation('myname')
mymodel.hc
```

6.1 Activating heat conduction physics

In principle, heat conduction physics is always available. The only point where activation becomes critical is in the base mesh generation.

The microwave physics module puts its own constraints on the maximum mesh size where this is based on the highest simulation frequency. For heat conduction physics no such constraint exists.

It is for this reason that in case where no EM simulation is included, the physics can be explicitly turned on or off:

```python
mymodel.set_physics(microwave=False, heatconduction=True)
```

6.2 Stationary, Transient and Non-Linear solvers

The current implementation of the heat conduction module comes with three separate solvers:

1. Steady State (Linear) - `.run_steady_state()`
2. Steady State (Non-Linear) - `.run_steady_state_nl()`
3. Transient Solver (Linear) - `.run_transient()`

A non-linear transient solver is being worked on.

6.3 Boundary conditions

The heat conduction physics module comes with the following boundary conditions. Note that some boundary conditions exclusively apply to surface domains (dimension = 2) where as others apply volumetrically (dimension = 3).

- `FixedTemperatureVolume` - 3D
- `HeatFluxVolume` - 3D
- `InitialTemperatureVolume` - 3D
- `FixedTemperatureBoundary` - 2D
- `HeatFluxBoundary` - 2D
- `InitialTemperatureBoundary` - 2D
- `ThermalContact` - 2D
- `ThinConductor` - 2D
- `Convection` - 2D
- `BlackBodyRadiation` - 2D (non-linear)

Some boundary conditions can only be exclusively assigned to geometric entities. Meaning that if that boundary condition is applied, none other can. For other boundary conditions this is not the case. If a boundary condition can only be exclusively applied to a geometric entity, it will automatically remove those entities from other boundary conditions.

6.3.1 Fixed Temperature (2D/3D)

The heat condition physics always solves for the temperature. The **fixed temperature** boundary conditions simply fix the temperature on boundaries and volumes. These degrees of freedom then do not have to be solved for. This is also known as a Dirichlet boundary condition:

$$T(\boldsymbol{r}, t) = T_{\text{fixed}(\boldsymbol{r},t)} \text{ for } \boldsymbol{r} \in \partial\Omega$$

They can be instantiated through:

python

```
mymodel.hc.bc.FixedTemperatureVolume(volume, Temp)
mymodel.hc.bc.FixedTemperatureBoundary(surface, Temp)
```

6.3.2 Heat Flux (2D/3D)

Heat flux boundary conditions injects heat energy either volumetrically ($\frac{W}{m^3}$) or on a surface ($\frac{W}{m^2}$). It is implemented as a Neumann boundary condition:

$$-k\nabla T \cdot \boldsymbol{n} = q_{\text{in}(\boldsymbol{r},t)} \text{ for } \boldsymbol{r} \in \partial\Omega$$

For boundary based heat flux one simply uses:

python

```
mymodel.hc.bc.HeatFluxBoundary(surface, q)
```

For volumetric heat flux, one can either supply a constant heat flux or a function which is a function of x,y and z:

python

```
mymodel.hc.bc.HeatFluxVolume(surface, heatflux=qv)
mymodel.hc.bc.HeatFluxVolume(surface, heatflux_func = lambda x,y,z: ...)
```

For volumetric heat flux the heat gets injected according to the function:

$$\rho c_p \frac{\partial T(\boldsymbol{r}, t)}{\partial t} = \nabla \cdot (k \partial T(\boldsymbol{r}, t)) + Q \text{ for } \boldsymbol{r} \in \Omega$$

6.3.3 Initial temperature (2D/3D)

The initial temperature boundary condition is primarily useful for transient simulations as for steady state simulations it does not affect the outcome. It may only change the convergence of the non-linear solver. An initial temperature can be provided using either of the following:

```python
mymodel.hc.set_initial_temperature(T0) # entire domain
mymodel.hc.bc.InitialTemperatureBoundary(surface, T0)
mymodel.hc.bc.InitialTemperatureVolume(volume, T0)
```

6.3.4 Thermal Contact Resistance (2D)

The thermal contact resistance or **Thermal Contact** boundary condition is an interface condition that defines a heat flux per unit surface area as a function of a thermal delta between the two sides. Numerically, a boundary gets split into two sides such that the heat flux on either side is defined by the contact conductance h_c in W/m^2K

The boundary-condition is defined as:

$$\boldsymbol{q}_1 \cdot \boldsymbol{n}_1 = -\boldsymbol{q}_2 \cdot \boldsymbol{n}_2 = h_c\left(T_1 - T_2\right)$$

The thermal contact boundary condition can be used to model non-ideal thermal interfaces between two objects. Think of a PCB pressed to a heat sink with a think air gap in between.

Contact type	Contact Resistance [W/m²K]
Aluminum Heatsink to Copper (Dry, Screwed)	4,000
Aluminum to Aluminum (Dry, Chassis Mount)	3,000
PCB to Aluminum Chassis (Direct Contact, Screwed)	800
Transistor (TO-220) to Heatsink (Dry)	2,500
Transistor (TO-220) to Heatsink (With Thermal Paste)	60,000
PCB/Component to Heatsink (With Thermal Pad)	12,000
Component to PCB via Thermal Vias (Copper filled)	45,000

Limitations: The thermal contact boundary condition can only be applied between two separate geometries. It cannot be applied to a boundary that is inside the same volume.

It can be implemented as following:

```python
mymodel.hc.bc.ThermalContact(surface, hc)
```

6.3.5 Thin Conductor (2D)

The thin conductor boundary condition models the conductance of a very thin conducting material without having to explicitly model it as a 3D geometric entity. It can be used to model heat conduction through PCB traces for example. The governing equation is:

$$-\nabla_\Gamma \cdot \left(\kappa t \nabla_\Gamma T\right) = 0$$

It can be implemented using:

```python
mymodel.hc.bc.ThinConductor(surface, material, thickness)
```

6.3.6 Convection (2D)

The convection boundary condition models a finite heat transport to an ambient temperature heat sink. It is implemented as a robin boundary condition:

$$-k\nabla T \cdot \boldsymbol{n} = h_{\text{conv}}\left(T - T_{\text{amb}}\right) \text{ for } \boldsymbol{r} \in \partial\Omega$$

This boundary condition is often used for non-ideal contact of an electronic component to a thermal mass like an aluminium body that can be modeled as a thermal heat sink. The finite thermal conductivity models the fact that the contact conduction is non-ideal.

It is implemented using:

```python
```

```
mymodel.hc.bc.Convection(surface, hc, Tamb)
```

6.3.7 Black body radiation (2D)

Black body radiation is the loss of heat due to black body radiation. The governing equation is:

$$-k\nabla T \cdot \boldsymbol{n} = \varepsilon\sigma\left(T^4 - T_{\text{surr}}^4\right) \text{ for } \boldsymbol{r} \in \partial\Omega$$

Because the fourth power of the temperature is present, you need to run this boundary condition using the non-linear solver.

The boundary condition does not implement any means of optics in which other objects with black body radiation boundary conditions expose each other to exchange heat information.

The boundary condition requires two parameters:

- ε - Emissivity (between 0 and 1)
- T_{surr} - The ambient temperature of the surrounding.

6.4 Thermal Solvers

For solving thermal problem, EMerge uses both direct and iterative solvers. Direct solvers are the default.

Because thermal problems are semi-positive definite, Cholesky decomposition can be used instead of full LU which is more RAM efficient and faster.

The cholesky solver is only available in `scikit-sparse`. If you install scikit sparse it will be automatically invoked:

bash

```
pip install scikit-sparse
```

In cases where RAM memory becomes a constraint, you can use iterative solvers as well with a simple block-Jacobi preconditioner. To activate it use:

python

```
mymodule.hc.run_steady_state(direct=False, preconditioner=True)
```

The pre-conditioner is optional but it typically will improve convergence.

6.4.1 Non-linear solver

The non-linear solver is required if you use a black-body radiation boundary condition. The current implementation uses Anderson Acceleration to improve convergence.

It can be called using:

python

```
mymodel.hc.run_steady_state_nl()
```

It has three optional arguments:

- `max_nonlinear_iter: int = 50`: The number of total iterations
- `nonlinear_tol: float = 1e-6`: The convergence tolerance
- `anderson_m: int = 5`: The Anderson update window size

The non-linear solver is nothing less than a repeated call to the direct linear solver. Because the heat flux depends on the temperature and the temperature depends on the outward heat flux, an assumption must be made for the temperature on the boundary when assembling the Robin boundary condition. The Anderson update uses a window of previous solutions to make a better guess for the steady state temperature.

6.4.2 Transient-solver

There is also a transient solver for time dependent heating. Make sure that the initial temperatures are well defined.

The time dependent solver has two iteration methods:
- Backward Euler
- Crank-Nicolson

The backward Euler method is unconditionally stable but has a worse accuracy given the time step.

The Crank-Nicolson algorithm has a better accuracy for larger time steps but may oscillate if the time step is too large.

You can invoke the transient solver using:

```python
mymodel.hc.run_transient(T_total, T_step, stepping_algorithm='BackwardEuler')
```

6.5 Post processing

Just like the microwave module, the heat conduction physics module returns a `HCData` object. Inside this object there are also the `.field` properties for all the 3D field solution data and `.scalar` for global quantities. In the case of heat-conduction, the `.scalar` data field contains no information.

```python
data = mymode.hc.run_steady_state()
```

If a single steady-state solution is computed, the output of that solution is in:

```python
data = mymode.hc.run_steady_state()
field_solution = data.field[0]
```

In case you ran a transient simulation, each time step will be available in each field solution:

```python
data = mymode.hc.run_steady_state()
for field in data.field.iter():
    print(field.time)
# or select
field_at_time = data.field.find(time=some_time)
```

The heat condition module currently has 2 different fields, the temperature and volumetric heat flux. The temperature is available in two units: `.Tz` (Kelvin) and `.TdegC` (degrees Celcius). The heat flux is in the field `.q`.

6.6 Coupled electromagnetic/heat conduction

It is possible in EMerge to perform a simple one way coupled EM heating simulation. In theory one can also customize a two-way heat conduction simulation but that is not implemented natively yet.

To couple RF losses into the thermal simulation, one can simply inject the losses of a prior simulation through the `.CoupledEMHeating` boundary condition.

First you must perform some EM simulation which yieled a field solution and select some solution.

```python
data = mymodel.mw.run_sweep()
my_solution = data.field.find(freq=..., ...)
```

This solution can be directly imported as a volumetric heat source using the `.CoupledEMHeating` function:

```python
mymodel.hc.bc.CoupledEMHeating(domain, my_solution, excitation)
```

Example
See example file: `heatconduction/Demo3_RF_heating.py`

The function takes three arguments:

- `domain`: This is the domain in which you wan the heat flux to be imported. If you want all of them you can add: `em.select(*mymodel.all_geos())`
- `mwfield`: The microwave field solution
- `excitation_W`: The port excitation in Watts

The last parameter is a list of port excitation but instead of amplitudes provided in Watts. The order follows the port number. So if you have a three port network of which port 1 and 3 are excited by 50 W you add `excitation_W = [50, 0, 50]`.

7 Post Processing

In this chapter we will discuss the general post processing philosophy of EMerge. Because EMerge currently only has Microwave physics, we will focus on all the implementation specifics for the microwave module.

7.1 Data extraction

As was discussed in Section 4.12 the simulation data is split in two categories:

1. Scalar data
2. Field data

For the microwave physics, Scalar data includes quantities like the simulation frequency, ko and S-parameters. More will be added as development continues.

The field data contains the solution vector of each simulation plus a special object called a `FEMBasis` that implements the interpolation of these solutions. To illustrate how these behave, suppose we have just run a frequency domain sweep.

python
```python
my_data = my_model.mw.run_sweep()

my_field_at3GHz = data.mw.field.select(freq=3e9)
```

The code above will select the field solution corresponding to a simulation run at 3GHz (assuming that was run)[8]. We can use this result object to ask for the fields at some coordinate:

python
```python
xs, ys, zs = #your definition of coordinates

ehfield = my_field_at3GHz.interpolate(xs, ys, zs)
```

[8] In case a simulation is not exactly run at any given frequency, you can use the `.find(freq=3e9)` method instead which will return the closest solution.

The interpolate function will compute the E and H fields at the provided coordinates and returns an `EHField` object that contains the fields plus coordinates of those fields. You can access the fields by calling its attributes:

python
```python
Ex = ehfield.Ex
Ey = ehfield.Ey
Ez = ehfield.Ez
E = ehfield.E
  > E will have shape (3,N)
H = ehfield.H
... etc
```

There are various methods available to efficiently generate grid points and execute interpolation at the same time. For example a slice in the X, Y or Z plane can easily be generated using:

python
```python
Esheet = field_3GHz.cutplane(ds=0.002, y=5*mm).E
  > This will generate a rectangular grid of points space 2mm apart in the XZ-
  plane through he entire domain at y = 5mm
Evol = field_3GHz.grid(ds=0.002).E
  > Will sample the field in a 3D grid inside the entire domain with a regular
  grid spacing at 2mm.
```

7.2 Visualization

This visualization section will focus on two forms of visualization, 2D plots and 3D plots.

7.2.1 3D Plots

EMerge has a default implementation of PyVista for 3D visualization. Inside the Simulation object is an instance of the custom class `PVDisplay` under the attribute name `Simulation().display`. This display class object can be used to pass objects to in order to visualize fields and geometries.

First, the simplest way to view your geometry is to use the simple function `.view()` on your simulation my_model.

If you run the `.view()` method **before** generating the mesh, EMerge will render a quick coarse mesh as currently only meshed geometries can be shown. To visualize just the geometry, we have to fall back on the GMSH ui by calling `.view(use_gmsh=True)`.

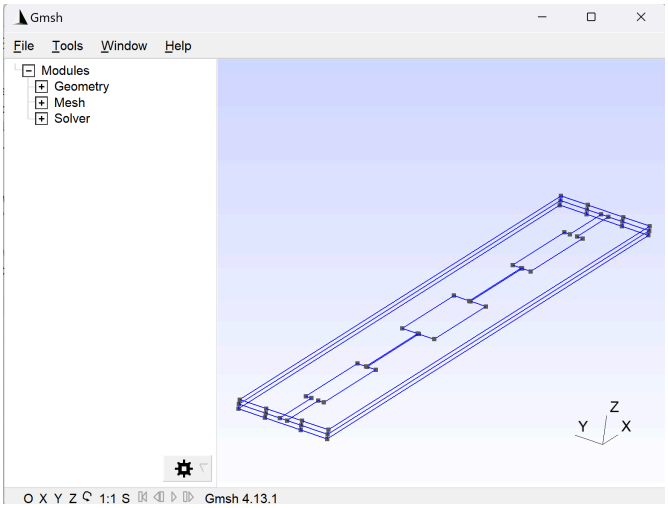


Figure 39: The GMSH geometry view window.

If you call the view method **after** mesh generation, the 3D model will be displayed using the PyVista plotter.[9].

[9] PyVista cannot (yet) display arbitrary geometries as EMerge has not implemented a way to convert the internal representation of volumes and surfaces to objects that PyVista can display

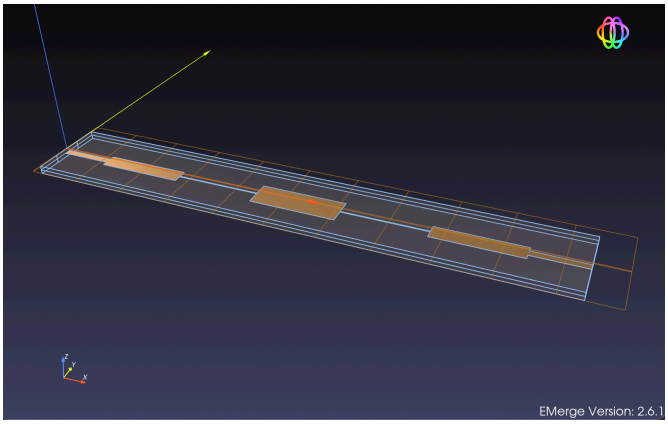


Figure 40: The EMerge geometry view window (version 2.7).

It is possible to measure distances between nodes of the mesh by pressing **M** on the keyboard. Afterwards, you can click on nodes to measure the distance between them. Each new click will select a new point such that the last two points clicked will be measured. Below in the plot window, you can see a decomposition of the distances in the individual components.

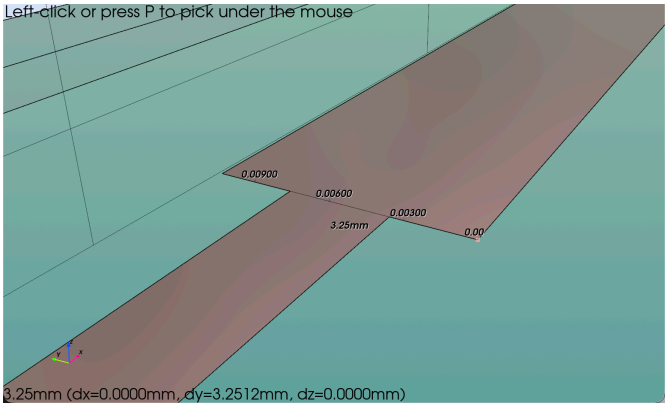


Figure 41: An example of the measurement in EMerge.

7.2.2 Key-bindings

There are several key bindings available in the normal viewer:

- **X** - View from the X-axis
- **Y** - View from the Y-axis
- **Z** - View from the Z-axis
- **C** - Switch between perspective and isometric rendering
- **I** - View from a clean XYZ perspective
- **Q** - Clean termination of the view window (essential when closing animations).
- **M** - Engages measurement mode
- **N** - Engages in an improved node selection mode
- **L** - Engages in a line/edge selection mode
- **W** - Enable wireframe rendering
- **E** - Exits the view (default)
- **Right** - Forward cycle through the selected items.
- **left** - Backwards cycle through the selected items.

7.2.2.1 `.view()` arguments

The view method has some convenient arguments to help you gain the information you need!

- `selections=[]`: A list of objects or selections you want to highlight in the view.
- `use_gmsh: bool=False`: If you want to use the GMSH viewer to display the geometric state or mesh.
- `plot_mesh: bool=False`: Turn this on if you want to view the mesh of your model instead of the materials/geometry
- `volume_mesh: bool=True)`: This is by default true causing the mesh view to display the volumetric mesh. If you turn this off, you will only see the mesh on surfaces of geometries.
- `opacity: float = None`: With this parameter you can globally change the opacity of all geometries Lower this to make everything more transparant or less.
- `labels: bool = False`: If you turn this on, EMerge will add a name label for each geometry in your view. The name can be assigned or changed manually. It is also auto-generated in many cases.
- `bc: bool = False`: If you turn this on, the view will show you all the different boundary conditions (if defined) applied to their respective surfaces.
- `face_labels: bool = False`: Turn this on to show all the names of each face in the geometry. This can be convenient if you need to select a face from an important geometry or complex shape that does not have a trivial face name associated with it.
- `bw: bool = False`: Will show the geometry in a black and white view that is appropriate for 3D construction drawings for example.

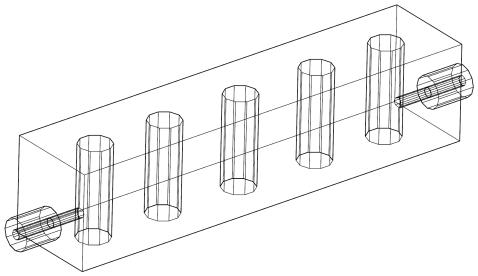


Figure 42: A black and white rendition of a geometry.

7.2.3 Display color Themes

Since the 2.0 release there are customizable color themes available.

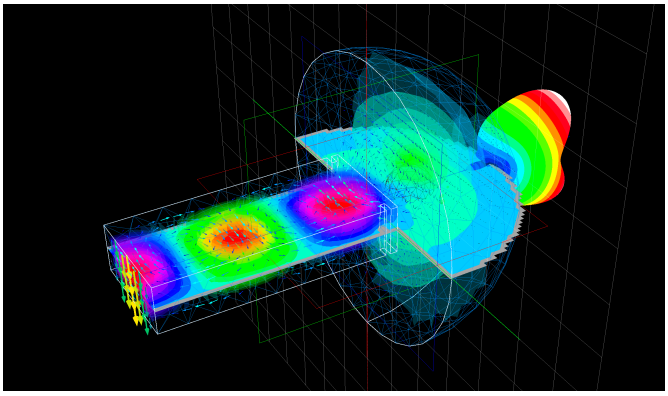


Figure 43: An example of the Vintage theme.

You can set a theme using:

```python
my_model.display.set_theme(mytheme)
```

Some basic themes are available in

```python
from emerge.themes import Vintage, Document, Tron
```

You can also make your own themes as following:

python

```python

class MyTheme(em.EMergeTheme):

  def define(self):

      # Generic Controls
      self.aa_active: bool = True
      self.aa_mode: str = "msaa"
      self.aa_samples: int = 8

      # Background
      self.backgroung_grad_1: str = "#a8c4e7"
      self.backgroung_grad_2: str = "#dbe5f6"

      # Axis and Grids
      self.draw_xplane: bool = False
      self.draw_yplane: bool = False
      self.draw_zplane: bool = False
      self.draw_xgrid: bool = False
      self.draw_ygrid: bool = False
      self.draw_zgrid: bool = True
      self.draw_xax: bool = True
      self.draw_yax: bool = True
      self.draw_zax: bool = True

      self.axis_color: str = "#000000"
      self.axis_x_color: str = "#ff007b"
      self.axis_y_color: str = "#88ff00"
      self.axis_z_color: str = "#003cff"

      # Grids
      self.grid_color: str = "#8e8e8e"
      self.grid_width: float = 0.5

      # Labels
      self.label_color: str = "#FFFFFF"
      self.text_color: str = "#000000"

      # Geometry
      self.geo_edge_color: str = "#000000"
      self.geo_edge_width: float = 2.0
      self.geo_mesh_width: float = 1.0
      self.geo_mesh_color: str = "#000000"

      # Materials and rendering
      self.render_shadows: bool = True
      self.render_metal: bool = True
      self.render_style: Literal['surface', 'wireframe', 'points'] =
  'surface'
      self.render_mesh: bool = False
      self.render_metal_roughness: float = 0.3
      self.render_min_opacity: float = 0.0

      # Color modifiers
      self.brightness: float = 1.0
      self.bleding_sequence: list[tuple[str, str, float]] = []

      # Colormaps
      self.cmap_npts: int = 256
      self.default_amplitude_colormap: str = "amplitude"
      self.default_wave_colormap: str = "wave"

      self.colormaps: dict[str, tuple[list[str], list[float]]] = {
          'amplitude': (["#1F0061","#4218c0","#2849db", "#ff007b",
  "#ff7 51"],
```

The properties are defined as following (str colors are hexadecimal so `"#ff007b"` for example):

- `aa_active: bool` - If anti-aliassing is turned on
- `aa_mode: str` - Anti aliassing mode (MSAA (default), FXAA, SSAA).
- `aa_samples: int = 8` - Anti aliassing samples
- `background_grad_1: str` - Background gradient color (bottom)
- `background_grad_2: str` - Background gradient color (top)
- `draw_xplane: bool` - If an X-plane should be drawn (also for Y and Z)
- `draw_xgrid: bool` - If an X-grid should be drawn
- `draw_xax: bool` - If an X-axis should be drawn (in the origin)
- `axis_x_color: str` - The X(YZ) axis color
- `grid_color: str` - The background grid line color
- `grid_width: float` - The background grid line width
- `label_color: str` - The face and object name label box color
- `text_color: str` - The label and axis label text color
- `geo_edge_color: str` - The color for geometry edges (outer)
- `geo_edge_width: float` - The width of geometry edges (outer)
- `geo_mesh_color: str` - The color of geometry face mesh lines (inside)
- `geo_mesh_width: float` - The width of the geometry face mesh lines (inside)
- `render_shadows: bool` - If shadows should be rendered from lighting.
- `render_metal: bool` - If Physica Based Rendering should be used (PBR)
- `render_style: str` - How to render faces (“surface”, “wireframe” or “points”)
- `render_mesh: bool` - If the face mesh lines should be rendered `geo_mesh_color` etc.)
- `render_metal_roughness: float` - A coefficient for the roughness of metals (between 0 and 1, 0 is perfectly shiny).
- `render_min_opacity: float` - A minimum opacity for all geometries.
- `cmap_npts: int` - How many interpolation points to use for custom color maps.
- `default_amplitude_colormap: str)` - The name of the default colormap for amplitude plots (non symmetric)
- `default_wave_colormap: str` - The name of the default colormap for wave plots (symmetric color scales).
- `colormaps`: A dictionary of colormaps defined with the keys being the names and then a tuple of:
  - `colors: list[str]` - A list of colors (with or without alpha channel)
  - `positions: list[float]` - A list of each colors position along the range from 0.0 to 1.0.
- `line_color_cycle: list[str]` - A list of colors to use for mesh plots.
- `brightness: float = 1.0` - A brighness modifier (multiplies each RGB value).
- `blending_sequence: list[tuple[str, str, float]]` - A list of blending sequencies to apply over geometry colors. A blending effect is defined as following:
  - `name: str` - The name of the blending effect (normal, multiply, screen, overlay, darken, lighten, color-dodge, color-burn, hard-light, soft-light, difference, exclusion, luminosity, 8bit, ansi)
  - `color: hex` - The color to blend over the material color
  - `alpha: float` - The alpha strength of the blending layer.

7.2.4 Manual 3D plots

It is also possible to construct a 3D plot window manually. The PVDisplay class offers many convenient methods to add different objects to plots. Below, some examples are given on how to efficiently add objects and cut-planes to the window.

Example

See example file: `demo18_plotting_and_visualization.py`

```python
my_model.display.add_object(diel, opacity=0.2)
> The opacity argument can be used to make objects transparent
my_model.display.add_object(trace)
my_model.display.add_object(vias)
my_model.display.add_quiver(*data.item(3).cutplane(ds=0.001,
z=-0.00025).vector('E'))
> After generating the cutplane data, the .vector() method of the dataset
object can be used to return all the required arguments for a vector plot:
x, y, z, Ex, Ey, Ez = data.vector('E')
my_model.display.add_field(data.item(3).cutplane(ds=0.001,
z=-0.00075).scalar('Ez','real'))
my_model.display.show()
```

7.2.5 Plotting syntax

In order to facilitate the generation of plots, EMerge uses the so-called “list comprehension” syntax and the convenient `add_field()` method.

The `add_field()` method can add `EHField` objects generated by for example cutplane

```python
mymodel.display.add_field(data.field[index].cutplane(0.001,
z=-0.005).scalar('Ex','abs'), symmetrize=True)
```

This automates unpacking the data and calling `add_surf()`, `add_quiver()` and `add_trisurf()` manually. We will now explain how to make plots manually as well.

In EMerge, various plot functions require multple arguments with data. For example, the `.add_surf(X,Y,Z,Value)` function requires a 2D matrix of X, Y and Z coordinates and a same shaped matrix with the values to plot.

It is quite cumbersome to generate all of this data manually each time you wish to plot a field. Lets take a simple example of a cutplane at `z=-0.005` for example.

```python
xs = np.linspace(xmin, xmax, 51)
ys = np.linspace(ymin, ymax, 101)
X, Y = np.meshgrid(xs, ys)
Z = np.ones_like(X)*(-0.005)
result = data.field[index].interpolate(X,Y,Z)
mymodel.display.add_surf(X,Y,Z,result.Ez.real)
```

The `.interpolate` function returns an `MWField` object that contains both the coordinates at which it was sampled and the field-values. There is a much easier way to do this. First, we can use a build in method to generate a set of points at a certain height using the `.cutplane()` method. This automatically detects a bounding box, samples with a certain discretization size and at a cartesian plane defined by either the optional arguments `x`, `y` or `z`. Then we can extrac

```python
result = data.field[index].cutplane(0.001, z=-0.005)
X = result.x
Y = result.y
Z = result.z
F = result.Ez.real
mymodel.display.add_surf(X,Y,Z,F)
```

We could also generate the coordinates and field in one go by using the `.scalar()` method which returns a `PlotFieldData` object that contains the coordinates and field quantity and some extra information to use by the `add_field()` method to know which plot function to use. The exact argument in order needed for a scalar surface field plot:

```python
result = data.field[index].cutplane(0.001, z=-0.005)
plot_data = result.scalar('Ez','real')
X, Y, Z, F = plot_data.xyzf
mymodel.display.add_surf(X,Y,Z,F)
```

As you might expect, the even shorter way to do this is:

```python
result = data.field[index].cutplane(0.001, z=-0.005)
mymodel.display.add_surf(*result.scalar('Ez','real).xyzf)
```

or even:

```python
mymodel.display.add_surf(*data.field[index].cutplane(0.001,
  z=-0.005).scalar('Ez','real).xyzf)
```

To ensure backwards compatibility, you can also omit the `xyzf` call as EMerge knows that the class itself can also be unpacked:

```python
mymodel.display.add_surf(*data.field[index].cutplane(0.001,
  z=-0.005).scalar('Ez','real`))
```

After the unpacking of the arguments, you can still continue adding positional arguments yourself and also keyword arguments. For example, you may add `symmetrize=True` to automatically generate a symmetric color scale for some value range $[-V, V]$.

```python
mymodel.display.add_surf(*data.field[index].cutplane(0.001,
  z=-0.005).scalar('Ez','real'), symmetrize=True)
```

7.2.6 3D Plot functions

To demonstrate important plot features, the following small demo-script can be used as reference.

```python
model = em.Simulation(...)
box = em.geo.Box(...)
sphere = em.geo.Sphere(...)
cylinder = em.geo.Cylinder(...).hide() # Will not be shown
selection = em.geo.select(...)

display = my_model.display # Name assignment for convenience only
display.add_object(box)) # adds a box
display.add_objects(cylinder, sphere) # Add multiple objects
display.add_object(selection) # Can also add eslections
display.add_field(plotdata) # Generic for adding plot data.
display.add_surf(X,Y,Z,Field) # Add a surface plot. (N,M)
display.add_contour(X,Y,Z,Field) # Add a 3D contour plot (N,M,K)
display.add_quiver(x,y,z,vx,vy,vz) # Add a quiver plot
display.add_scatter(x,y,z) # Add simple points
display.add_farfield(field) # 3D farfield plot
display.add_trisurf(x,y,z,f,tris) # adds an unstructured trisurf plot
display.add_text(test, color, position) # Add text in view
display.animate().add_surf() # Animates a plot if data is complex.
display.show() # start the window
```

7.2.7 Data generation

As mentioned before the field results in the MWField class have convenience methods to generate EHField class instances that carry specific data. Below is a simple overview op functions available.

Example
See example file: `demo18_plotting_and_visualization.`

- `.boundary(face_selection)`: Compute the EM fields on a face boundary
- `.cutplane(gridsize, axis)`: Computes the EM fields on a cartesian cutplane.
- `.cutplane_normal(point, normal, points)`: Computes the EM fields on a cutplane specified by a coordinate and a normal.
- `.grid(spacing)`: Computes the EM fields on a 3D grid inside the domain specified by a uniform point spacing.

7.3 Result Analysis

7.3.1 Far-field calculation

In many applications you wish to compute the radiation far-field. This can easily be done in EMerge after defining an integration boundary. The boundary must either be the boundary between an air-domain and PML domain or the same boundary (or a subset of) an absorbing boundary. Integrating boundaries that are PEC for example will yield strange results.

```python
import emerge as em
from emerge.plot import plot_ff

m = em.Simulation('ffsim')

## setup domain

abc_selection = air_box.outside()

m.mw.bc.AbsorbingBoundary(abc_selection)

data = m.mw.run_sweep()

ffdata = data.field[0].farfield_2d((0,0,1),(1,0,0), abc_selection)
plot_ff(ffdata.ang, ffdata.normE)
```

A little bit about the `farfield_2d( ... )` function arguments

- `ref_direction: tuple[float, float, float]`: The direction defined as `angle=0`
- `plane_normal: tuple[float, float, float]`: An axis vector normal to the scan plane. For angles in the YZ-plane you give the normal as (1,0,0).
- `faces: FaceSelection`: The boundary to integrate

Optionally you can define the angular range and step size. In the example above the far-field is computed for the first frequency sweep point (`data.field[0]`). The function returns a `EHDataFF` instance which contains the results in a convenient format.

7.3.1.1 EHDataFF class

The `EHDataFF` (Previously `FarFieldData` on lower versions.) class is a simple container for the θ and φ angles and the complex E and H-fields. It contains dynamic properties that can compute derived quantities like the Poynting vector etc. The following properties are available:

- `.Ex, .Ey, .Ez, .Hx, .Hy, Hz`: The individual components
- `.normE, .normH`: The complex normal E and H fields
- `.Sx, .Sy, .Sz, .normS`: The Poynting vector
- `.Etheta, .Ephi`: The theta and phi components of the E-field (Ludwig-3 definition)
- `.Erhxp, .Elhcp`: The Left and right hand circular polarization amplitudes
- `.AR`: The Axial Ratio

The θ and φ axes are in `.theta` and `.phi`. For farfield data generated by a single cut-plane the local in-plane angle is in `.angle`.

More conveniently you can further specify far field components that work similar to the `EHDataFF` class using the following attributes:

- `.E`: The E-field
- `.H`: The H-field
- `.dir`: The Directivity
- `.gain`: The Gain

From this you can then specify `.x` and `.rhcp` etc:

- `.x`: The X-component
- `.y`: The Y-component

- `.z`: The Z-component
- `.theta`: The θ-component (Ludwig 3)
- `.phi`: The φ-component
- `.rhcp`: The Right-Hand circular polarized component
- `.lhcp`: The Left-Hand circular polarized component
- `.AR`: The Axial Ratio
- `.norm`: The magnitude norm

So for example you can do:

```python
ffdata = data.field[0].farfield_2d(...)
gain_rhcp = ffdata.gain.rhcp
directivity_norm = ffdata.dir.norm
```

7.3.1.2 3D Far-field plot

For a 3D Far-field plot you can use the 3D far-field calculator. It will return a `FarfieldData` instance that contains the spherical coordinates and cartesian E and H field components.

Since 2.7 you can use the `.add_farfield3d()` method on this FarFieldData object:

```python
ff3d = data.field[i].farfield_3d(boundary_selection, origin=(0, 0, 0))
model.display.add_farfield3d(ff3d, rmax=40 * mm, offset=(0, 0, 40 * mm))
```

This method has several arguments to size the farfield plot to your liking. Optionally you can also manually add it:

The `FarfieldData object` has a method `.scalar()` that returns the required ingredients to directly use it in the display class's `.add_surf()` method:

```python
m.display.add_field(data.field[1].farfield_3d(abc).scalar('normE'))
```

```
> You may wish to set the following options:
• In surfplot() you can add:
  ▸ dB=True – For dB scale
  ▸ dblim=-10 – To set -10dBi as R=0
  ▸ rmax=… – To define at what absolute size the diagram is shown
• in add_field():
  ▸ cmap = 'viridis' – Or another option for a better color scale.
  ▸ symmetrize = False – To prevent the color scale from going to -lim to
    +lim.
```

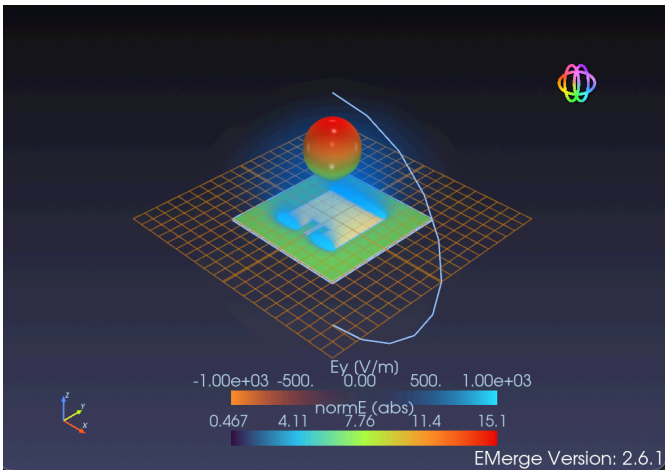


Figure 44: A far-field diagram of the patch antenna.

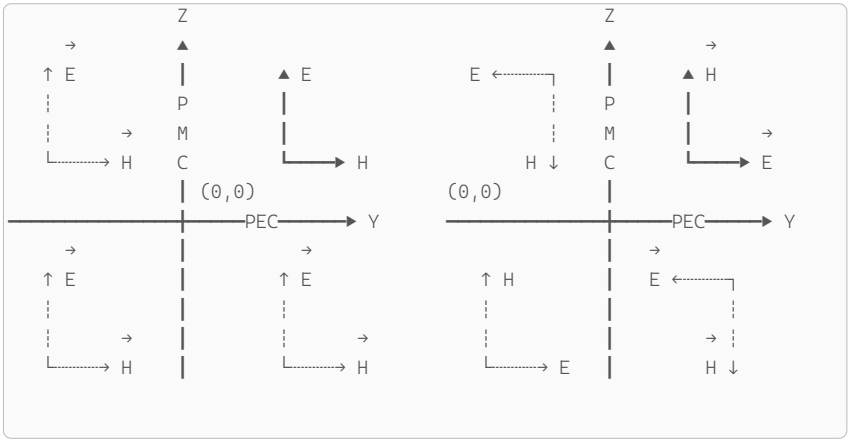
7.3.1.3 Symmetry planes

All far-field calculators allow for the implementation of basic cartesian symmetry planes as long as the planes are either $x = 0, y = 0$ or $z = 0$. To add a symmetry plane, you add a

string to a list in the optional `.syms` argument. For an E-plane symmetry reflecting about the XZ-plane (y=0), you simply add `Ey` in the list. If the symmetry plane is a PMC plane, you can add `Hy` for example. An example is provided below:

```python
data.field[3].far_field3d(surface, syms=('Ez','Hy'))
```

Which adds the following symmetry:



Listing 13: A PMC symmetry plane in $y = 0$ and a PEC symmetry plane in $z = 0$ showing the reflection of E_z, H_y on the left and E_y, H_z on the right.

7.3.1.4 Spherical axis definitions

You can also define the exact far-field points to compute the far field at. The following spherical coordinate system definition is used (ISO 80000-2:2019):

$$\hat{x} = \sin(\theta) \cdot \cos(\varphi)$$
$$\hat{y} = \sin(\theta) \cdot \sin(\varphi)$$
$$\hat{z} = \cos(\theta)$$

This yields the following definition for the far-field θ and φ unit vectors:

$$\theta_x = -\cos(\theta) \cdot \cos(\varphi)$$
$$\theta_y = -\cos(\theta) \cdot \sin(\varphi)$$
$$\theta_z = \sin(\theta)$$
$$\varphi_x = -\sin(\varphi)$$
$$\varphi_y = \cos(\varphi)$$
$$\varphi_z = 0$$

7.4 Exporting Data

7.4.1 Touchstone files .sNp

There currently only exists a build in export functionality for the Touchstone format. This implementation can be accessed in the `MWScalarNdim` which is returned when you access the attribute `data.scalar.grid`. The recommended usage is as following:

python

```python
data = my_model.mw.run_sweep()

data.scalar.grid.export_touchstone("filename.snp", Z0ref=50, format="RI",
custom_comments=["Custom comment line 1","Custom comment line 2"],
funit="GHz")
> The Z0ref argument defines reference impedance which will be used to
renormalize the entire S-matrix to that impedance value.
```

7.4.2 Step files

You can also save your geometry to a step file like this:

python

```python
...
mymodel.commit_geometry()
mymodel.export('myfile.step')
```

Finally, you may export all the items in a plot window to `.vtk` files using the `.save_vtk()` method on your display class. Fill in the name of a folder as multiple .vtk files will be generated. This is still a feature in development.

python

```python
...
mymodel.display.save_vtk('mypath')
```

7.4.3 VTK files (Beta)

You can export all the geometries in a current PyVista display to a set of VTK files as following:

python

```python

my_model = em.Simulation(...)

#popoulate a display

my_model.display.save_vtk('my_path_name')
```

EMerge will save each element to its own unique VTK file in a directory name that you specify.

7.4.4 DXF Files (Beta)

It is possible to export all geometries (or a subset off) on a specific z-height or the different layers of PCB's to DXF polygons. This works as following.

To export files on a specific height you do the following:

python

```python
from emerge.beta.dxf import export_dxf

my_model = em.Simulation(...)

# build your geometry

my_model.generate_mesh()

export_dxf(my_model, 'myfilename', z_height=[0, -th*mm])
```

You can also add a list of geometry selections which you want to include as optional parameter.

! WARNING !
It is mandatory to build a mesh first as the current implementation uses the Mesh to build and later simplify all the geometry polygons. Curves surfaces are “not” included as curved in the DXF export currently.

To export a PCB layer you do the following:

python
```python
from emerge.beta.dxf import pcb_to_dxf

my_model = em.Simulation(...)

my_pcb = em.PCBNew(...)

my_pcb.compile_paths()

pcb_to_dxf(pcb, 'myfilename')
```

This will automatically export each PCB layer geometry to an individual DXF file.

7.4.5 FarField data ASCII

You can export far-field data using the `FarFieldExporter` class in `emerge.write`.

python
```python
import emerge as em
from emerge.write import FarFieldExporter

sim = em.Simulation(...)
# setup here
data = sim.mw.run_sweep()

theta = np.linspace(0, np.pi, 21)
phi = np.linspace(-np.pi, np.pi, 41)

exportdata = [field.farfield_3d(rad_boundary, theta, phi) for field in
   data.field]

ec = FarFieldExporter('some_filename', exportdata, precision=6)

ec.addcol().Ex
ec.addcol().Ey
ec.addcol().Ez

ec.write()
```

You have to compute the 3D farfield patterns yourself first using `.farfield_3d()`. All the farfiel data objects can be passed in a list to the exporter. To export columns you simply call `.addcol()` and then add attributes as if you are referencing data fields of the farfield data class. The data comes out like this:

```txt
% Theta(deg)
0 9 18 27 36 45 54 63 72 81 90 99 108 117 126 135 144 153 162 171 180
% Phi(deg)
-180 -171 -162 -153 -144 -135 -126 -117 -108 -99 -90 -81 -72 -63 -54 -45 -36
  -27 -18 -9 0 9 18 27 36 45 54 63 72 81 90 99 108 117 126 135 144 153 162 171
  180
% Frequencies(Hz)
8e+09 8.5e+09 9e+09 9.5e+09 1e+10

# 8e+09 Hz
$ Theta(deg);  Phi(deg);  Etheta[re];  Etheta[im]
0              -180       -0.114743    0.16379
9              -180       -0.186912    0.125031
18             -180       -0.237857    0.0901795
27             -180       -0.267473    0.0720824
36             -180       -0.276075    0.0756024
45             -180       -0.264132    0.0978891
54             -180       -0.232867    0.131457
63             -180       -0.184167    0.167617
72             -180       -0.11996     0.197944
81             -180       -0.0427145   0.213691
```

The rules for this format are quite simple:

- At the top of the file the theta and phi angles are introduces by a header starting with `%` followed by `Theta` or `Phi` with the unit `rad` or `deg`.
- The individual axes are space separated on the line following it.
- The frequency axes is then shown, also starting with a `%` symbol and a unit.
- The frequency data is also provided by a space separated list.
- For each frequency, a flattened table is provided.
- The block starts with a `#` symbol followed by the frequency
- The table column names are provided first on a line starting with a `$` symbol.
- The header names are separated by a semicolon `;`.
- The data is fixed width separated.
- The farfield components in complex value are always split in separate columns (re and im) specified by square braces `[re]` and `[im]`.

You can read the data as well into a dictionary of Numpy arrays:

```python
from emerge.read import import_farfield
data = import_farfield('some_filename.emff')

```

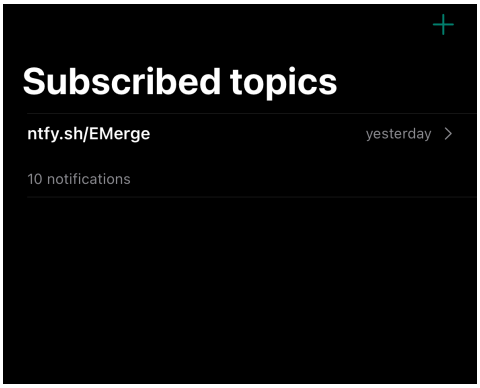
8 Advanced Use

8.1 Auxilliary features

8.1.1 Smartphone Notifications

Since 2.4.0, you can use the `model.ping("message","my_channel)` function to send a message to the NTFY app you can download from the app stores on iOS and Android.

The NTFY app works with **public** channels. Before version 2.6.1 EMerge used the public "EMerge" channel. However, it is adviced to specify your own channel to keep messages private. It is only private in so far as nobody else is subscribed to that channel. **there is no way to shield others from subscribing to your channel, never share private data!!**.



In the top-right corner press the (+) button. Then simply fill in `EMerge` as topic name and press 'Subscribe'. Then, any time you call `model.ping('some message')` it will appear on your smartphone as long as you are on the same WiFi.

```python
model.ping("Your message", channel="yourchannel")
```

8.2 Design Optimization

The first version of optimization is available since EMerge version 2.2. It uses Scipy's minimization algorithms underwater but it makes it available in a for loop type notation so that one does not need to put the simulation inside of a function.

The setup of an optimization will be described below with a simple placeholder set of variables named `var1` and `var2`.

We start by setting up our optimization. The optimizer class is available under our `Simulation` object under the name `.opt`.

8.2.1 Adding optimization parameters

To run our optimization we have to create variables that we will be using in our optimization loop to optimize. we do this with the `.add_param()` function

python

```python
sim = em.Simulation('simname')
sim.opt.add_param('var1', 1.0, (-1.0, 2.0))
sim.opt.add_param('var2', 2.0, (0.0, 3.0))
```

The arguments for this function are first the name, then the initial value and finally the bounds (minimum and maximum value).

8.2.2 Minimization vs Maximization

Usually, the optimisation module in Scipy does minimization only. It is always possible to turn a minimization into a maximization by taking the negative of our parameter. If this is annoying to keep track off you can call `sim.opt.maximize()` which will toggle on automatic maximization.

8.2.3 Optimization method

You can set the optimization methods that are available in Scipy's optimization module. They have to accept bounds and be deterministic. The options that you can currently use are:

- `"Powell"`
- `"L-BFGS-B"`
- `"TNC"`
- `"COBYLA"`
- `"COBYQA"`
- `"SLSQP"`
- `"trust-constr"`
- `"Powell"`

To set a method simply do:

python

```python
sim.opt.method = 'Powell'
```

8.2.4 Running the optimization

To run the optimization we call a for loop on our optimizer. The order that the parameter will be passed as a tuple will be the same as the order in which we generated them. We run our optimization with `.run()` which accepts an integer that limits the maximum number of iterations.

To optimize, we have to tell our optimizer how well it is doing. This can simply be done by passing some custom defined metric/number that it is supposed to maximize. We pass this number using the `.update(metric)` method.

```python
sim = em.Simulation('simname')
sim.opt.add_param('var1', 1.0, (-1.0, 2.0))
sim.opt.add_param('var2', 2.0, (0.0, 3.0))
sim.opt.method = 'Powell'

for v1, v2 in sim.opt.run(100):
  # generate geometry
  data = sim.mw.run_sweep()

  metric = #some calculation
  sim.opt.update(metric)
```

8.2.5 Optimization data

As is discussed in chapter Section 4.12, the dataset that is returned and maintained in our simulation object `sim.data.mw` contains a list of all the simulation we ever ran including previous optimizations. Because of this, the parameter space is not organised in a nice structure which means we cannot call the usual `.grid` on the `.scalar`. In order to essentially isolate only the data from our latest run, since version 2.2.1 you have to use the `.slice_set()` method to isolate some slice of all the simulations. This takes one or two integer arguments that act like `my_list[i1,i2]`. Assuming that you run `Nfreq` frequency points per optimisation run, you can thus get a dataset of only the last optimisation using:

```python
sim = em.Simulation('simname')
sim.opt.add_param('var1', 1.0, (-1.0, 2.0))
sim.opt.add_param('var2', 2.0, (0.0, 3.0))
sim.opt.method = 'Powell'

sim.mw.
for v1, v2 in sim.opt.run(100):
  # generate geometry
  sim.mw.set_frequency_range(f1,f2,Nfreq)
  data = sim.mw.run_sweep()

  gridded = data.scalar.slice_cet(-Nfreq).grid
  metric = #some calculation
  sim.opt.update(metric)
```

You can define any metric you want.

8.3 Adaptive Mesh Refinement

Starting from EMerge version 1.1, a beta version of adaptive mesh refinement is available. Since 2.0 it is included in the primary library. This feature is still being worked on and may not function optimally yet but it works quite well.

To use it, simply call `.adaptive_mesh_refinement()` before you run your simulation.

```python
my_model = em.Simulation('ModelName')

#setup

my_model.adaptive_mesh_refinement()
my_model.mw.run_sweep()
```

The following arguments are available:

- `max_steps: int` - The maximum number of convergence steps before it gives up and proceeds with the simulation.
- `min_refined_passes: int` - The minimum number of refined passes before it considers the simulation converged. It defaults to 1.
- `convergence: float` - The S-parameter convergence as complex magnitude $\max\left(\| S_{ij}^{n-1} - S_{ij}^{n} \|\right)$. It defaults to 0.02 (-34dB)
- `magnitude_convergence: float` - The convergence purely based on the magnitude so $\max\left(| \| S_{ij}^{n-1} \| - \|S_{ij}^{n} \| |\right)$. Defaults to 1 so it won't ever be used but you can set it lower if you want
- `phase_convergence: float`; The convergence criterea for the phase in degrees (defaults to 180 degrees).
- `refinement_ratio: float` - This parameter determines how much smaller tet edges will be made in the refined regions. Setting this much lower will add many more tet elements. This parameter will be modified during the S-parameter convergence to keep the mesh size increase within limits. So without changing that limit, it will automatically stabilize around some value. Defaults to 0.75.
- `growth_rate: float` - How fast tets around a refinement point can grow in size. Defaults to 3 (minimum >1).
- `minimum_refinement_percentage: float` - The minimum a mesh is supposed to increase in size in terms of tetrahedra. If the mesh grows less in a pass, it will remesh with a smaller refinement ratio. If the mesh increases more than twice the minimum percentage, the refinement ratio will be adjusted down. Defaults to 20.0 (20%).
- `error_field_inclusion_percentage: float` - How many tets with a high posteriori error estimate are refined. Defaults to 5.0 or 5%.
- `frequency: float | list[float]` - The frequency at which the refinement is done. Defaults to the highest frequency in the current simulation setup. You can also provide multiple frequencies to evaluate the error at.
- `show_mesh: bool` - If a surface mesh is shown in between steps (halts the refinement). Default is False. Useful for debugging.
- `max_tets: int` - The maximum number of tetrahedra before which the refinement is halted. Defaults to 1 million tets.

We can see the effects of AMR in this Vivaldi antenna example. The basic mesh on the surface of the PCB is seen here.

In the refined version, a much higher resolution is achieved around the feedline without needing to go through a complicated procedure of defining where the mesh should be refined yourself.

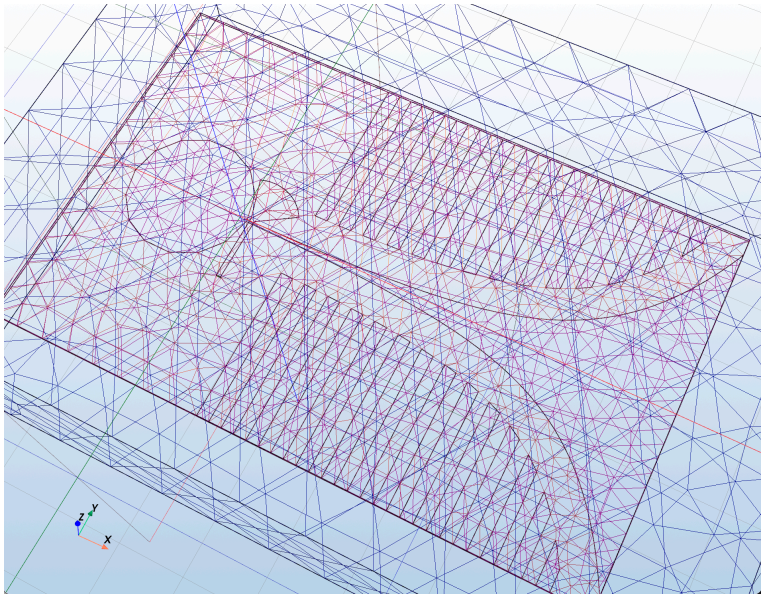


Figure 47: Before AMR.

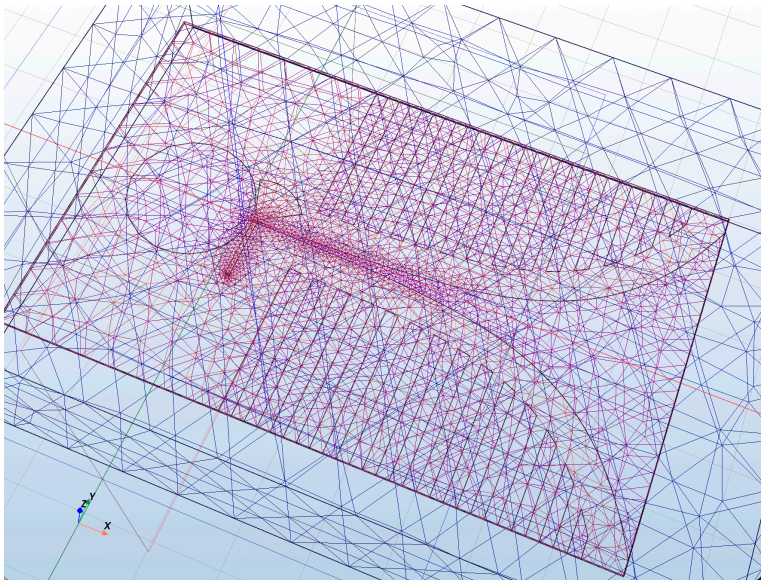


Figure 48: Before AMR.

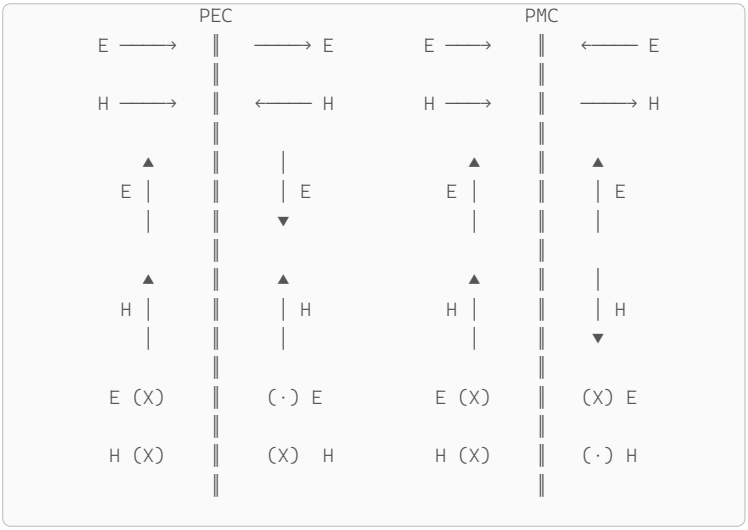
Adaptive mesh refinement in the current implementation requires GMSH to generate a completely new mesh with a new set of size constraints. It is hard to define a-priori how many extra tetrahedra will be added given a certain size constraint. This is why you'll notice that it may take up to to 3 tries for EMerge to get a desired mesh growth. Size constraints make the meshing procedure slower, especially if your geometry consists of a lot of edges. Its computationally difficult to subdivide a mesh without significantly degrading the quality of the tetrahedra. Some existing solvers have found a good efficient procedure for this but an alternative is not yet present.

8.4 Symmetry planes

Symmetry planes in EM simulations are a method of reducing your simulation domain by a factor of 2, 4 or even 8 (in rare cases). Symmetry planes can be implemented if your geometry and E-fields can be completely flipped along a given axis. There are two types of symmetry:

- E-symmetry (PEC)
- H-symmetry (PMC)

To implement a symmetry plane, the entire boundary at that symmetry plane must be considered as either a PEC for E-plane symmetry or PMC for H-plane symmetry.



Listing 14: The different types of symmetries showing how different vectors flip depending on the type of boundary condition.

A convenient way to select all boundaries to assign either PEC or PMC is by using the `.inplane()` method of the selector class:

```python
symm_boundary = my_model.select.inplane(x0, y0, z0, vx, vy, vz)
XYplane = my_model.select.inplane(0,0,0, 0, 0, 1)
XZplane = my_model.select.inplane(0,0,0, 0, 1, 0)
YZplane = my_model.select.inplane(0,0,0, 1, 0, 0)
```

8.5 PCB Design

Routing PCB traces can be very time consuming if you have to construct it from individual rectangles. To simplify the process, EMerge comes with a `PCBNew` that drastically simplifies the process. This page will guide you through the basics of the layouter.

! WARNING !
Since EMerge version 2.2, the PCB class has been replaced by PCBNew in order to separate the new behavior that layers will be counted from index 0. Use the PCBNew class until the old PCB class is depricated.

More information on the details of the PCB router you can consult Section A.1.

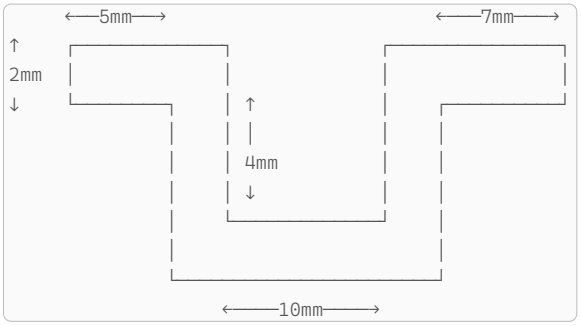
8.5.1 The basics

A `PCBNew` can be created from the `.geo` submodule.

```python
import emerge as em
mm = 0.001
m = em.Simulation('pcb sim')
pcbl = em.geo.PCBNew(1.54, mm, material=em.lib.FR4)
```

The `PCBNew` class itself is not a geometry object. It is simply an interface to create them efficiently. The PCB class heavily utilizes a so called **method chaining** philosophy. Method chaining involves around methods returning itself or different classes so that a next instruction can be directly called on the resultant output. For the PCB class, traces can be created using the `.new()` method. This will return a `StripPath` object which represents a single path from start to finish.

To simplify the typing, all dimensions in the PCB class class including the thickness are provided in a specific unit. In the example above, we defined the unit as `mm`. Thus, all numbers plugged in will be measured in millimeters. Lets start by showing how to make the following strip-line design:



Listing 15: The stripline path dimensions.

This trace does the following:

Step 1: Start with a width of 2mm and move 5mm forwards
Step 2: make a 90 degree turn right
Step 3: Go 4mm forward
Step 4: make a 90 degree turn left
Step 5: go 10 mm forward
Step 6: make a 90 degree turn left
Step 7: Go 4mm forward
Step 8: make a 90 degree turn right
Step 9: Go 7mm forward

To create this trace in EMerge you can simply do the following:

```python
pcb.new(0,0,2,(1,0)).straight(5).turn(90).straight(4)\
 .turn(-90).straight(10).turn(-90).straight(4)\
 .turn(90).straight(7)
```

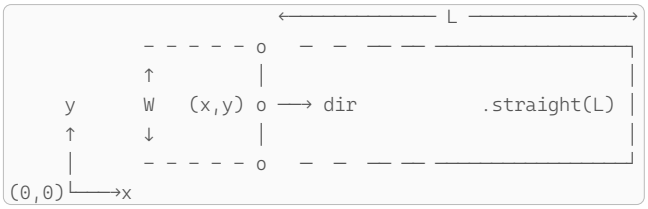
With every call to `.straight()` or `.turn()` the same `StripPath` object is returned after which you can call a new routing instruction. There are many instructions available

- `.straight()` – Go a certain distance straight
- `.turn()` – Make a turn (positive is clockwise)
- `.via()` – Add a via and start a new trace at a different depth
- `.short()` – Convenient way to add a quick short circuit at the current place.
- `.taper()` – Add a straight section tapering to a different width
- `.jump()` – Stop the current trace and continue somewhere else. (Useful for coupled line filters for example.
- `.macro()` – Use the EMerge routing macro language to define a path
- `.to()` – Auto route towards a certain destination
- `.stub()` – Add a finite length straight section branching out from the current point
- `.store()` – Remember the current point for later use.
- `.split()` – Remember the current point and go along a different route,
- `.merge()` – Move back to the last call of `.split()`
- `.curve()` – make a circular curve left or right.
- `.lumped_element` - Creates a lumped element surface and jumps the router forward. The size is specified by a string such as “0402”.
- `.bck` – Move half a line width back
- `.skip` – Skip the next half linewidth

For many of these commands, the width can also be changed in order to make things like stepped impedance filter.

- `.pturn()` – Make an in-place turn. (equivalent to `back.turn().skip`)

At any point during the routing process, the current position is a mix of an (x,y) coordinate plus a direction and a width. Calling the `.straight()` method for example simply adds a piece of stripline in that current direction with that width (if no other width is provided).



Listing 16: The current router position at (x,y) with some width W can be be extended by a distance L using the `.straight(L) method.`

8.5.2 Trace thickness and Trace Materials

In the current implementation of EMerge, losses in traces can only be modeled by giving traces a physical thickness. In terms of the solution, giving a couple of microns thickness in the simulation makes almost no difference. However, for losses it matters a lot.

The `trace_material=` property allows you to set a material for the traces. If this is not the default `PEC` material, the PCB class will assing the `ThinConductor` boundary condition.

If the `thick_traces=True` property is set, the traces will be modeled as a thin 3D geometry instead with a `SurfaceImpedance` boundary condition automatically assigned to them. For more information see Section 5.1.7.

8.5.3 Compiling paths

In order to turn the paths into actual geometries, you have to use the `.compile_paths()` method on the `PCBNew`. This will turn all instructions into actual polygons. Each individual path will be rendered to its own geometry unless you turn the optional `merge=True` argument on.

python

```
my_trace_list = my_pcbl.compile_paths(merge=False)
my_trace = my_pcbl.compile_paths(merge=True)
  > if merge is True, only a single GeoPolygon object will be returned
```

! WARNING !

Paths may not end where they start, the compiler is not able to deal with that. To make circular paths possible you may call the split method before a straight section which will stop the last path and start a new one.

8.5.4 Volumes, dielectrics and ports

After traces have been define, it is possible to define the bounds for the PCB. The bounds will be based by the bounding box of the PCB in the XY-plane. You can add margins to keep the bounding box some distance away from the traces.

python

```
pcbl.define_bounds(left=5*mm) # if mm is the unit, use (left=5)
  > Determines the bounds with a 5mm margin on the left
```

After defining the bounds, you can add volumes.

8.5.4.1 Generating the PCB stack

To create the dielectric, use the `.generate_pcb()` method. Optional arguments allow you to split the volume into different sub-volumes for each z-height detected. The z-height is defined uniquely for each strip path. The material will be automatically assigned

python

```
pcblayers = pcbl.generate_pcb()
```

By default a list of layers is returned from bottom to top.

8.5.4.2 Generating an airbox

You can create air above your domain up to a certain height using the `.generate_air()` function

python

```
air = pcbl.generate_air(height=10*mm)
```

To add an airbox below the PCB you can set `bottom=True` in the `generate_air()` function.

8.5.4.3 Ground planes

To create a 2D plane, simply use the `.plane()` method. This can be used to create random rectangles or planes that occupy the entire bounds. The PCB has an optional argument `layers` that allows you to define some amount of copper layers (min 2). The `pcbl.z(1)` will allow you to get the z-height of each layer.

python

```
pcbl = em.geo.PCBNew(1.5, mm, layers=3)
pcbl.set_bounds(xmin=0, ymin=0, xmax=10, ymax=10)
  > The bounds must be defined before planes can be generated. This setting is
  arbitrary
ground = pcbl.plane(z=pcbl.z(0))
middle = pcbl.plane(z=pcbl.z(1))
top    = pcbl.plane(z=pcbl.z(2))
```

8.5.5 Ports

You can automatically create port rectangles to use for lumped ports (inside domain) or modal ports (boundary of domain). To create such a port surface, you must specify a certain point on the PCB path. At any point during routing, you may call the `.store('my_name')` method or simply through indexing or function calling. This will record the current position under that name. Then you can create a port for that point using the `.modal_port()` for modal port faces for ports on the boundary of your simulation domain or `.lumped_port()` if they are inside your domain.

```python
port_face_1 = pcbl.modal_port('p1')
lumped_port_face = pcbl.lumped_port('p2'))
```

An example including both the store and load commands is provided below.

```python
import emerge as em

mm = 0.001
w1 = 1.2 #mm
w2 = 1.8 #mm
L = 20 #mm
my_model = em.Simulation('impjump')

pcbl = em.geo.PCBNew(1.5, mm)

pcbl.new(0,0,w1,(1,0)).store('p1').straight(L).straight(L,w2)['p2']

modal_port1 = pcbl.modal_port('p2')
lumped_port = pcbl.lumped_port('p2')
# more code

my_model.mw.bc.ModalPort(modal_port1, 1, modetype='TEM')
my_model.mw.bc.LumpedPort(lumped_port, 2, Z0=50.0)
```

If you pass a lumped_port rectangle to the `LumpedPort` boundary-condition constructor, the width, height and orientation information will be automatically passed on.

8.5.6 Vias

Vias can be generated during the routing process using the `.via()` method. It has quite a lot of arguments to tune the behavior of the via:

- `znew: float` The new Z-height
- `radius: float` The via radius
- `proceed: bool = True`, If you want to proceed with routing afterwards. (False for shorts to ground)
- `direction: tuple[float, float] | None = None`, The new departure direction after moving planes.
- `width: float | None = None` The new departure line width
- `extra: float | None = None` The amount of extra line distance beyond where the via is placed (overlap).
- `segments: int = 6` The number of via polygon sections.
- `hole_radius: float | None = None` The radius of the ground plane hole punched in each generated plane.
- `hole_skip_layers: list[int] | None = None` Which ground plane layers should be skipped.
- `reverse: float = 0` A distance to move the via back relative to the current position and movement direction.

Whenever you create a new ground plane using the `.plane()` method after having created the vias, holes will be punched through it if a `hole_radius` is specified. If you want such a

via to connect to a ground plane you can put the ground plane layer number in the skipped layer list.

You can create the actual via cylinders using the `.generate_vias()` method

```python
all_vias = pcbl.generate_vias(merge=True)
```

8.5.7 Layers

The PCBNew class constructor has a layer argument that defines the number of layers including the top and bottom (2 minimum). The z-height of any part of the pcb can then simply be retrieved using `pcb.z(0)` for the bottom, etc.

Listing 17: The order of the layer numbering in the PCB class with n layers.

Additionally, one may also specify a dielectric layer stack as a list of `em.PCBLayer` objects. This can be done using `em.PCBNew(stack=[ ... ])`.

Specify only the dielectric layers, the copper layers are numbered at the top, bottom and in between each layer.

! WARNING !
Since version 2.2 the PCB class will be temporarily depricated for PCBNew where counting starts from index 0. In future updates, PCBNew will be merged with PCB. Until then PCB will maintain the old counting system.

8.5.8 Other construtors

Besides the ones mentioned here, the PCB layouter has other functions to generate geometries.

- `.plane( ... )` - Adds and returns a rectangular surface inside the PCB.
- `.add_poly(xs, ys, z)` - Adds a polygon to be generated during the `.compile_paths` call.
- `.stub( ... )` - A convenience function to create a radial stub which uses the `.add_poly()` method.

8.5.9 Calculators

To facilitate design, the PCB class offers a `calc` attribute that contains convenient functions. For now it just has an impedance calculator. Here are some examples:

Width calculator

The `.z0` function in the calculator offers a convenient interface to compute a required width given a desired impedance. The layer determines if the microstripline definitions are used or stripline impedance.

```python
width = pcb.calc.z0(50.0, layer=-1)
  > Computes the width of a 50Ω line on the top layer.
```

8.6 Periodic Cells

To facility periodic environments, EMerge also has the `PeriodicCell` class feature. There are two types of periodic cells: **`em.HexCell()`: A generic cell for hexagonal periodicity** `em.RectCell()`: A generic cell for rectangular periodicity

All periodic cells are always periodic in the XY-plane. These offer a convenient interface to set up periodic boundary conditions and scan settings. To setup a periodic environment you can use the following steps:

```python
periodic_cell = em.HexCell()
> Setup a cell definition
volume = periodic_cell.volume(0, H)
> Create a volume region that conforms to this cell automatically
model.set_periodic_cell(periodic_cell)
> Call this BEFORE setting up your mesh. The mesh on all periodic faces will automatically be copied.
model.mw.bc.floquet_port(z=...)
> If a periodic cell is defined, you can automatically generate a floquet port.
periodic_cell.set_scanangle(30,45)
> Defines the scan angle for the simulation. The appropriate phase shift will be automatically computed.
```

Given a scan angle θ, φ, the propagation constants defining the phase shift are defined as following (ISO 80000-2:2019):

$$k_x = k_0 \sin(\theta)\cos(\varphi)$$
$$k_y = k_0 \sin(\theta)\sin(\varphi)$$
$$k_z = k_0 \cos(\theta)$$

Example
See example file: `demo7_periodic_cells.py`

8.7 Polygons, Extrusion, Revolutions and Pipes

EMerge has its implementation of the extrusion, revolution and pipe operation made for the `XYPolygon` class. The idea is that one can define a polygon in a local 2D plane (XY) which can then be embedded in 3D space using the `CoordinateSystem` class. After a polygon is defined it can be revolved, extruded and piped (extrusion along a path).

The `XYPolygon` class has a convenient mechanism to define polygons.

First you can initiate an XYPolygon with a sequence of x and y coordinates:

```python
polygon = em.geo.XYPolygon(xs,ys)
```

It is also possible to leave it empty and add coordinates using the `.extend()` method:

```python
polygon = em.geo.XYPolygon().extend(xs, ys)
```

In the same way, you can also extend the polygon with a parametric curve that takes two functions for the x and y coordinates $f_x(t)$ and $f_y(t)$ which will be called from some range $t = [0, 1]$ (by default). The curve will be internally approximated by a piecewise linear line (no curved boundaries). The number of points used depends on two parameters:

- `xmin: float` - The minimum allowed step size (prevents over describing)
- `tolerance: float` - The maximum linear deviation that is allowed from the actual curve in meters.

Additionally, the `reverse=True` argument can be used with the `.parametric` function to simply reverse the coordinate order after the creation of the curve.

! WARNING !

An `XYPolygon` object is **not** a geometry yet in the EMerge domain. It must first be embedded into 3D space by defining a coordinate system (Defaulting to the global coordinate system) by calling the `.geo` method.

After finalizing the polygon, it can be revolved around some axis by using the `.revolve` function or extruded by using the `.extrude` function. With extrusions, the start face will always be named `.face('front')` and the ending face `.face('back')`.

```python
mm = 0.001

W = 30*mm
H = 12*mm
wr = 5*mm
g = 1*mm

my_model = em.Simulation('RWG')

xpts = [-W/2, -W/2, -wr/2, -wr/2, wr/2, wr/2,
          W/2, W/2, wr/2, wr/2, -wr/2, -wr/2]
ypts = [-H/2, H/2, H/2, g/2, g/2, H/2, H/2,
         -H/2, -H/2, -g/2, -g/2, -H/2]
wg = em.geo.XYPolygon(xpts, ypts).extrude(80*mm, em.YZPLANE.cs(0,0,0))
```

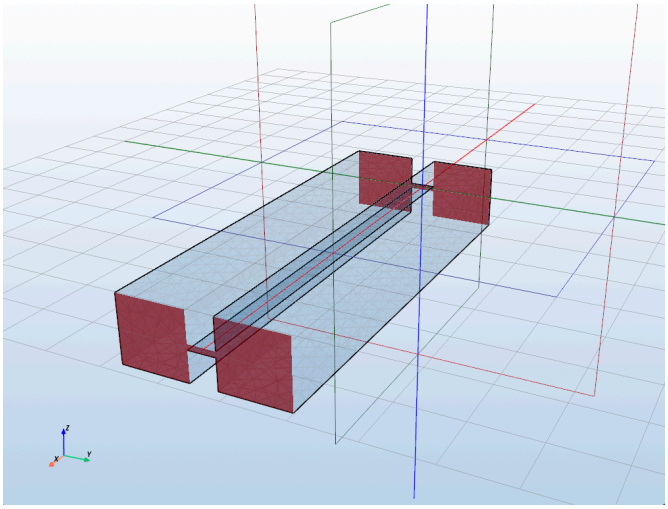


Figure 52: The ridged waveguide geometry defined by the extrusion.

8.7.1 Pipes

Pipes are extrusions of shapes along a path. To do this we must first define a curved path.

A Curve class object is specified by a list of x, y and z coordinates and a curve interpolation type. These are directly implemented in the OpenCASCADE Kernel. By default, a series of x, y and z-coordinates will be interpolated using a 'Spline'. Other options are 'BSpline' and 'Beziers'.

After defining a curve, we can create a pipe by calling the `.pipe()` method. The `.pipe()` method needs either an `XYPolygon` object or a `GeoSurface` object to define its cross-section. An `XYPolygon` object will automatically be placed such that the Z-axis is tangent to the curve start path. The user may provide an `xaxis` and `start_tangent` axis to define under what orientation the input polygon (if its an XYPolygon) will be piped. The `start_tangent` defines the orientation of the planes z-axis. The X-axis parameter defines under what orientation the XY-polygons X-axis is placed. The Y-axis will then be derived from the cross product.

The following code example defines a helical antenna with a hexagonal cross-section:

```python
h_curve = em.geo.Curve.helix_lh((0,0,porth+dfeed/2), (0,0,porth+dfeed/2+L),
  rad0, 13, r_end=0.8*rad0)
cross_section = em.geo.XYPolygon.circle(radw, Nsections=6)
helix = h_curve.pipe(cross_section).set_material(em.lib.MET_COPPER)
```

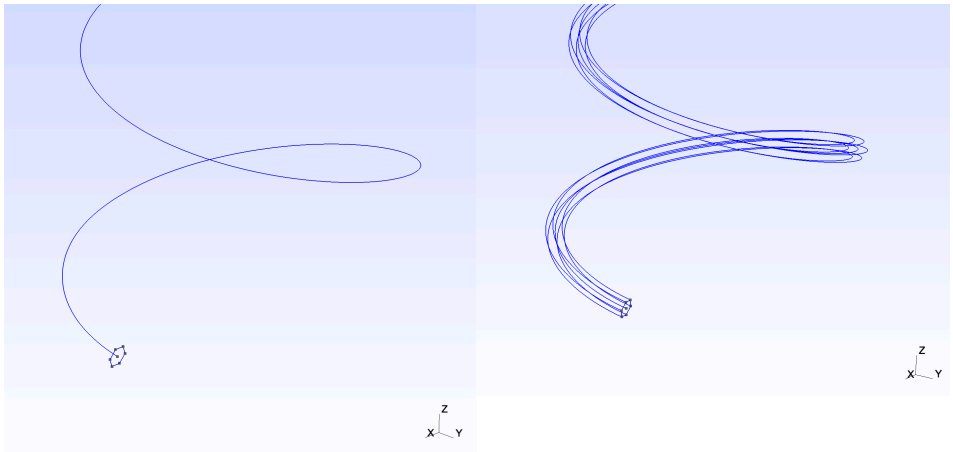


Figure 53: A hexagonal cross section and a helical curve before and after the pipe operation

8.7.2 Connecting Faces

Two XYPolygon objects can be connected to each other to create a solid using the `.connect()` method. To do this, each XYPolygon must first be embedded in a coordinate system. This can be done without transforming it into a GeoSurface object using the `.incs()` method.

OCC connects the geometry point wise so if the two geometries have a different number of edges, the pairing might not happen neatly.

```python
xy1 = em.geo.XYPolygon.rect(0.2,0.4, (0,0))
xy1.incs(em.GCS)
xy2 = em.geo.XYPolygon.rect(0.6,0.5, (0,0))
x2y.incs(em.GCS.displace(0,0,1))

geo = xy1.connect(xy2)
```

The resultant geo object has the two selected faces which can be selected using “front” for the first face and “back” for the second face. The resultant geometry looks like this:

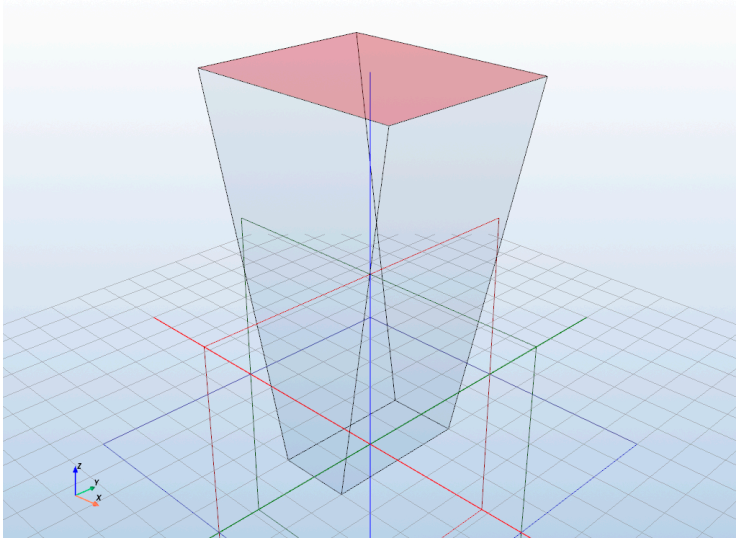


Figure 54: Two connected rectangles

8.8 Alternative basis functions

The finite element method uses a set of polynomal basis functions to express and solve for the E-field inside a tetrahedron. Starting in version 2.8, the user is able to change the basis functions used by EMerge.

For technical reasons, the basis functions used in time harmonic electromagnetic are sometimes referred to as **rotational** or **gradient** basis functions. This is because the fundamental buidling block basis functions are either basis functions that have a curl (rotational) or have zero-curl and are strictly vector basis functions modeled as gradients of a scalar field (gradient).

For most plane wave propagation, using more rotational basis functions is more optimal and reduces spurious noise. In some applications with high gradient fields such as patch antennas and other capacitive narrow components with strong E-fields close to sharp edges, it is beneficial to add more gradient field modes.

Rotational basis functions model plane wave propagation most efficiently. They focus all the numerical efficiency on modelling free space solutions where there is no charge accumulation and thus purely rotational fields.

Gradient basis functions model gradient fields well. These are important when very fine structures locally store a lot of energy in the Electric field. Rotational basis functions struggle to model these accurately and as a consequence, the parasitic capacitance of these structures is numerically underestimated. This typically causes patch antennas for exmample to have their resonance frequency underestimated because the artificial line lengthening effect due to local energy storage is not included sufficiently accurately.

8.8.1 Order and completeness

When choosing the basis functions, in its simplest form, one is prompted to choose for the order and the completeness of the basis function set. For the order one can only choose form first order (1) or second order (2).

Then there is the choice for **complete** vs **mixed**. A **complete** set of basis funtions contains all basis functions to fully span the polynomial order. For second order complete this would constitute 30 degrees of freedom per tetrahedron. The alternative (non-complete) is called **mixed**. Mixed order basis functions contain a selection of these in order to be more efficient computationally. In some cases, one can have better performance with complete ordered sets versus mixed order but these are relatively rare.

You can configure the basis functions as following

python

```python
my_model.mw.set_basis_space(2,True) # Complete second order
```

The first argument is the order and the second if its complete or not.

8.8.2 Basis function sets

EMerge has a predefined set of basis functions to include in the basis funtion space. These are numbered by integer and expressed in terms of the barycentric coordinates of the tetrahedron (or triangle). The following definitions exist:

Table 2: Edge based basis functions

Number	Type	Expression
0	rotational	$\lambda_i\nabla\lambda_j - \lambda_j\nabla\lambda_i$
1	gradient	$\nabla\left(\lambda_i\lambda_j\right)$
2	rotational	$\left(\lambda_i - \lambda_j\right)\left(\lambda_i\nabla\lambda_j - \lambda_j\nabla\lambda_i\right)$
3	gradient	$\nabla\left(\lambda_i\lambda_j\left(\lambda_i - \lambda_j\right)\right)$
4	rotational	$\lambda_i\left(\lambda_i\nabla\lambda_j - \lambda_j\nabla\lambda_i\right)$
5	rotational	$\lambda_j\left(\lambda_i\nabla\lambda_j - \lambda_j\nabla\lambda_i\right)$

Table 3: Face based basis functions

Number	Type	Expression
0	rotational	$\lambda_i\left(\lambda_j\nabla\lambda_k - \lambda_k\nabla\lambda_j\right)$
1	rotational	$\lambda_j\left(\lambda_i\nabla\lambda_k - \lambda_k\nabla\lambda_i\right)$
2	rotational	$\lambda_k\left(\lambda_j\nabla\lambda_i - \lambda_i\nabla\lambda_j\right)$
3	rotational	$\lambda_j\lambda_k\nabla\lambda_i + \lambda_i\lambda_k\nabla\lambda_j - 2\lambda_i\lambda_j\nabla\lambda_k$
4	rotational	$\lambda_k\lambda_i\nabla\lambda_j + \lambda_j\lambda_i\nabla\lambda_k - 2\lambda_j\lambda_k\nabla\lambda_i$
5	rotational	$\lambda_i\lambda_j\nabla\lambda_k + \lambda_k\lambda_j\nabla\lambda_i - 2\lambda_k\lambda_i\nabla\lambda_j$
6	gradient	$\nabla\left(\lambda_i\lambda_j\lambda_k\right)$

From these you can form your own set of basis functions as a selection of edge based basis functions and face based basisfunctions using the `DoFSet` class:

python
```
dofset = em.DoFSet([0,1],[3,4], 'YourName')
my_model.mw.set_basis_space(dof_set=dofset)
```

8.8.3 Predefined Sets

There are also a set of predefined basisfunctions available. These are the following:

Value	Name	Edge	Face	Label
0	FIRST_ORDER_MIXED	0		1st Order Mixed
1	FIRST_ORDER_COMPLETE	0, 1		1st Order Complete
2	SECOND_MIXED_SAVAGE	0, 1	0, 1	2nd Order Mixed (Savage)
3	SECOND_MIXED_VOLAKIS	0, 2	0, 1	2nd Order Mixed (Volakis)
4	SECOND_MIXED_WEBB	0, 1	3, 4	2nd Order Mixed (Webb)
5	SECOND_COMPLETE_WEBB	0, 1, 3	3, 4, 6	2nd Order Complete (Webb)
6	SECOND_COMPLETE_VOLAKIS	0, 2, 3	0, 1, 6	2nd Order Complete (Volakis)

These can be configured as following

python
```
my_model.mw.set_basis_space(elementspace=em.ElementSpace.FIRST_ORDER_MIXED)
```

8.8.4 When to choose which one

In most cases the most efficient is any second order mixed set. Second order complete is much slower in terms of solve time due to the larger dense linear algebra operations needed. In some instances it can be more efficient though.

The primary effect is in terms of the modelling of local capacitive energy storage. For example, Even extremely course meshes with second order complete basis functions will have a very accurate resonance frequency for patch antennas. This is not the case for any of

the others. Similarly, mixed second order tends to under-estimate the resonance frequency and complete first order tends to overestimate. This is shown in Figure 55.

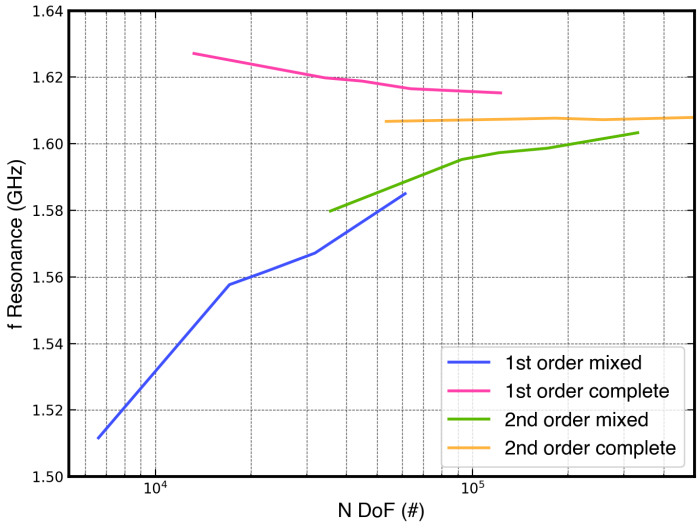


Figure 55: A patch antenna's resonance frequency as a function of the total number of degrees of freedom.

Mixed first order is almost never a good choice as it requires very fine meshes to get even reasonable accuracy. One may use mixed first order for very quick evaluation.

Complete first order may at times be efficient for some capacitive structures like patch antennas or capacitive waveguide irises. The biggest downside of first order basis functions is that far-field patters and phase dispersion (causing phase errors) is much worse.

Mixed second order is almost always the best.

In some cases, complete second order can help with models such as Patch antennas given a coarse mesh if validating the resonance frequency is critical.

9 Extending EMerge

It is possible to extend EMerge with custom functions and libraries. This chapter will discuss how to plug in custom code at various points in the EMerge ecosystem. On request, more of these features can be added.

9.1 Custom geometry

It is fairly trivial in EMerge to plug in your own GMSH based geometry creation into the EMerge ecosystem.

If you wish to create your own geometries you **have** to use the OpenCASCADE kernel within GMSH. You cannot use its normal geometry features. For more information on how modelling in GMSH works you can look at the extensive documentation on the website:

https://gmsh.info/doc/texinfo/gmsh.html

Here is a basic example to generate a box:

python

```python
import gmsh

bog_tag = gmsh.model.occ.addBox(0, 0, 0, 1, 1, 1, 1)
```

A call to OpenCASCADE will return a so called **tag** which is sipmly an integer that specifies which volume is assigned to it. All volumes must be known to the EMerge solver. This can easily be done by passing a list of tags that belong to a volume to any of EMerge's domain classes:

- 0D - Point = `GeoPoint`
- 1D - Point = `GeoEdge`
- 2D - Point = `GeoSurface`
- 3D - Point = `GeoVolume`

It is important that your Simulation model is defined before creating this class instance. You can get access to each of these through the extension submodule called `.ext`. Below is a basic example:

python

```python
import emerge as em
from emerge.ext import GeoVolume
import gmsh

model = em.Simulation('MyModel')
box_tag = gmsh.model.occ.addBox(0, 0, 0, 1, 1, 1, 1)
box_obj = ext.GeoVolume([box_tag,])
...
```

From this point on the box GeoVolume object is also known internally. Within EMerge there is a `_GeometryManager` class with a globally defined `_GEOMANAGER` instance. When you create a Simulation object, it signs itself in as the current active my_model. Each creation of a `GeoObject` class, the parent class of GeoVolume, will automatically submit itself to the `_GEOMANAGER` instance under that simulation file. Currently only one can be active at the same time.

Thus, after creating your GeoVolume object, you are relieved of your responsibilities. A simple demonstration of a geometry class that happens to not exist yet would be this (if you have a good implementation please share it, you will get copyright).

```python
import emerge as em
from emerge.ext import GeoVolume
import gmsh

class MyHelix(GeoVolume):

  def __init__(self, properties...):
      # your generation code
      volume_tag = some_function_or_line(...)
      super().__init__([volume_tag,])

```

There are some methods of the GeoVolume class (or GeoObject in general) that might be interesting to implement.

Important

Make sure that all 3D geometries created are known to some GeoObject class instance. For the final boolean fusion, EMerge must know of all 3D objects known inside GMSH as well.

9.1.1 Boolean operations background information

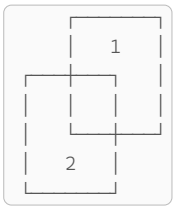
It is important to know how EMerge handles boolean operations with instances of GeoObject and the final boolean fragment that is executed.

First, when a boolean operation is implemented on a GeoObject, EMerge's implementation of these binary operators will turn on and off `_exists` attributes. If two objects are subtracted the resulting object will get a new tag. The older object still exists but are deactivated.

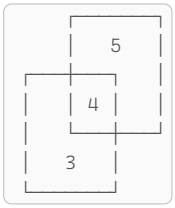
More than one tag may be associated to a single GeoObject. In that sense, the only thing that functionally bundles it is its material assignment.

After you commit the geometries, a boolean fragment is executing splitting all known geometries in unique disjoint volumes. GMSH returns a new mapping from input geometries to resultant output geometries. It will then tell the GeoObject what its new tags are after the boolean fragment. Multiple GeoObjects that overlap will get the same domain tags assigned to them.

Lets take the following example again:



We start with two volumes (3,1) and (3,2). Both intersect with each other. We may imagine that these are two GeoVolume objects `GeoVolume[(3,1)]` and `GeoVolume[(3,2)]`. After boolean fusion they come three new domains: (3,3), (3,4) and (3,5)



EMerge will automatically update the assignments which means that:

- `GeoVolume[(3,1)]` Turns into `GeoVolume[(3,5),(3,4)]`,
- `GeoVolume[(3,2)]` Turns into `GeoVolume[(3,3),(3,4)]`

As you can see, both domains will refer to the same geometric volume (3,4) because they both occupy this space. Material assignment will therefore be handled such that the highest priority or the last to be defined geometry will dominate over the lower priority or earlier defined geometry. This is all handled automatically.

It is thus very important to keep, specifically, all GeoVolume objects within the EMerge ecosystem. If volume tags exist within the GMSH kernal but not known within EMerge, strange behavior might occur because EMerge will not be able to do anything with a subset of of the mesh that GMSH returns.

9.1.2 Face assignment

OpenCASCADE allows for the generation of volumes from a collection of surfaces that enclose a certain space. Each of the surfaces will also get tags assigned to them. Selections and Surfaces within EMerge are both defined by the tags that GMSH keeps track of internally. It is tempting to offer these tags as assignments exposed to the user but this will not work robustly. After the boolean fragment operation, faces may be split into multiple sub faces. This reassignment information is not exposed to the user. Consequently, if a geometry you have thinks that one of its faces has some tag (say 5), after the boolean fragment this face likely no longer exists causing the assignment to become invalid.

To recover some of this behavior, EMerge has a `_FacePointer` class that "weakly" defines a surface within EMerge. A face is defined in essence by the infinite plane in 3D space it finds itself in:

$$ax + by + cz = d$$

This may also be derived from an origin and normal vector:

$$\mathcal{O} = \begin{pmatrix} x_0 \\ y_0 \\ z_0 \end{pmatrix}, \qquad \hat{n} = \begin{pmatrix} n_x \\ n_y \\ n_z \end{pmatrix}$$

For each face you create you may create a face pointer object and submit them with an associated name to the `._face_pointers: dict` attribute.

python

```python
class MyHelix(GeoVolume):

def __init__(self, properties...):
    # your generation code
    volume_tag = _last_function_call(..)
    super().__init__([volume_tag,])

    front_cap = _FacePointer((x0,y0,z0), (nx,ny,nz))
    self._face_pointers['front_cap'] = front_cap
    # or shorter
    self._add_face_pointer('name',(x0,y0,z0), (nx,ny,nz))
    # or even shorter
    self._add_face_pointer('name', tag=face_tag)
```

You can efficiently compute the origin and normal data using GMSH as following. This is also what happens if the `tag= ...` optional argument is used.

python

```python
origin = gmsh.model.occ.get_center_of_mass(2, face_tag)
normal = gmsh.model.get_normal(tag, (0,0))
self._add_face_pointer('my_name', origin, normal)
```

Every time you change your geometry, move it, rotate, mirror etc, all the face pointers will be co-transformed and passed to subsequent GeoObjects when its created. You can access the faces later using `myobj.face('my_face_name')`. When called, EMerge will look at all faces

associated with the domains at the moment of the call and check if its in the plane, if it is it will select it. If two faces are parallel (in the same plane) both will be included. Curved faces, while supported in GMSH are not yet identifiable in EMerge.

9.2 Custom Solvers

It is possible to create a custom solve class for EMerge. This example will focus on a generic Solver but you may also implement eigenmode solvers, pre-conditioners and sorters. Use the following classes:

- $Ax = b$ solvers – `Solver`
- $Ax = \lambda Bx$ generalized eigenmode – `EigSolver`
- pre-conditioner – `pre-conditioner`
- Sorter – `Sorter`

```python
from emerge.ext import Solver

class CustomSolverName(Solver):
    """ Your solver class."""
    req_sorter: bool = False # Set to demand sorting algorithms.
    real_only: bool = False # Will provide the matrix as a lager real matrix

    def __init__(self, pre: str):
        super().__init__(pre) # Pre must be included for logging.
        # custom properties

    def initialize(self) -> None:
        # Execute code you want to happen before running sims.

    def duplicate(self) -> Solver:
        new_solver = self.__class__(self.pre)
        # Copy any settings you wish to transfer when solver copies
        # are distributed to multiple threads.
        return new_solver

    def reset(self) -> None:
        # This will be called at the end of a simulation to allow
        # you to reset stored data such as factorizations
        # and factorization states.
        pass

    def solve(self,
              A: csr_matrix,
              b: np.ndarray,
              precon: LinearOperator,
              reuse_factorization: bool = False,
              id: int = -1) -> tuple[np.ndarray, SolveReport]:

        # Solve the matrix problem Ax=b
        aux_data = {} # put any string based data in this
        return x, SolveReport(solver=str(self),
                              exit_code=0,
                              aux=aux_dict)
```

The following properties are critical:

- **real_only**: If set to True, the SolveRoutine in EMerge will convert the complex matrix to a real matrix:

$$A \longrightarrow \begin{pmatrix} \Re(A) & -\Im(A) \\ \Im(A) & \Re(A) \end{pmatrix}, \qquad b \longrightarrow \begin{pmatrix} \Re(b) \\ \Im(b) \end{pmatrix}$$

- **precon**: A pre-conditioner that will be passed to the solver as calculated by a pre-conditioner class. Currently never used.
- **reuse_factorization**: Reuse factorization will be set to True if EMerge requests the user to reuse a numeric factorization. This means that the same matrix A is supplied but with a different vector b.
- **id**: This argument can be used to implement print statements. You may import `logger` from loguru and use the id integer to detail print information.

9.2.1 Solver Methods

We will now discuss what some important methods are for:

.initialize()

The `.initialize()` method is run before the solve method is ever called. During initialization of the SolveRoutine class, each Solver class will be created. Sometimes one does not wish to also immediately initialize a solver itself because of communication overhead, path searches etc. Especially if that solver is not even used. As an example, the cuDSS solver will initialize a communication with the Video card. You don't want this to be done for each parallel solve thread. Thus the initializatio method can be used to actually set up and configure your solver when you will be using it.

.duplicate()

The duplicate method will be called when parallel Python threads are spawned. Crucial solver settings will be copied here. Thus if your solver can run on multiple threads, make sure that all user settings are copied to each child instance.

.reset()

After a simulation is completed, the reset method will be called. If you store a symbolic factorization or pre-conditioner that you wish to reuse for subesquent frequency sweeps, you will probably want to reset these if you start solving a different geometry. Thus you can reset the internal state of your solver using this method.

.solve()

This is of course the final method that actually is supposed to solve the problem. You will be passed the following data:

- A: The matrix of problem $\boldsymbol{Ax} = \boldsymbol{b}$
- b: The vector in problem $\boldsymbol{Ax} = \boldsymbol{b}$
- precon: The pre-conditioner matrix or None if no one is defined.
- reuse_factorization: A boolean that is True if the solve routine is called for the same matrix A but a different vector b.
- id: An integer of the dataset. You can use this for printing statements to clarify information for the user.

`.pre` parameter

All solvers are initialized with a `pre` argument which is a string prompt added in the log statements to identify the solver thread/process. Just make sure to always pass this argument stored in `self.pre` to each initializer.

9.3 Boundary conditions

It is not possible to add numerically unique boundary conditions in EMerge without adapting the entire code-base. For some type of boundary conditions like boundary conditions of the third kind (Robin boundary condition) you are able to make custom variations.

9.3.1 Ports

It is possible to define your own port mode boundary condition. A specific set of functions have to be defined in order to have them work in EMerge. Below is a template version

python

```python
class MyPortBC(PortBC):

  _include_stiff: bool = True
  _include_mass: bool = False
  _include_force: bool = True

  def __init__(self,
               face: FaceSelection | GeoSurface,
               port_number: int):
      super().__init__(face)

      self.port_number: int = port_number
      self.cs: CoordinateSystem

  def get_basis(self) -> np.ndarray:
      return None

  def get_inv_basis(self) -> np.ndarray:
      return None

  def portZ0(self, k0: float) -> complex:
      return ...

  def modetype(self, k0):
      return 'TEM' or 'TE' or 'TM'

  def get_beta(self, k0: float) -> float:
      return ...

  def port_mode_3d(self,
                   x_local: np.ndarray,
                   y_local: np.ndarray,
                   k0: float,
                   which: Literal['E','H'] = 'E') -> np.ndarray:
      # function
      return Exyz

  def port_mode_3d_global(self,
                          x_global: np.ndarray,
                          y_global: np.ndarray,
                          z_global: np.ndarray,
                          k0: float,
                          which: Literal['E','H'] = 'E') -> np.ndarray:
      # function
      return Exyz
```

This is the minimum set you need to conclude. First the class properties `_include_stiff` and `_include_force` need to be True. For the initializer you need to set the `port_number` property. It is possible not but necessary to define a coordinate system for the port.

The matrix assembler will compute a local basis for a port mode field. In this basis each point of the port mode is in the XY plane. Depending on how you define your port function you might want a tight control over where you define the origin. If you want EMerge to use

this origin you need to pass the CoordinateSystem class's basis and inverse basis matrix to the `.get_basis()` and `.get_inv_basis()` methods.

For TEM modes you may define a function that computes the characteristic impedance. Make sure that the method `.modetype(k0)` returns "TE", "TM" or "TEM".

You must also define the out-of-plane propagation constant in the `.get_beta()` method.

Finally, you have to define both the `.port_mode_3d()` method and the `.port_mode_3d_global()` method. The first computes the port mode field in based on coordinates in the local XY plane. It must however return the XYZ components in the global coordinate system. The last function must return the mode field based on global coordinates. You may of course derive one from the other or reverse.

10 Performance and Optimization

10.1 Parallel Computing

Parallel computing is possible in many different ways and at many different stages in the solution process. In Python we may discriminate between three different styles of parallel computing:

- Multi-threading
- Multi-processing
- GPU acceleration

For GPU acceleration, EMerge has an experimental interface with NVidia's cuDSS solver. Besides this there are two different ways Python can run tasks in parallel, either through multi-processing or multi-threading. In multi-threading Python will spawn multiple threads that can only perform tasks simultaneously if they "release the GIL".

The GIL is a feature in Python that prevents threads from accessing the same data. You can imagine it as a single entity that is allowed to parse lines of Python code. In normal operation, the Python process just runs through each line of code, waits for it to finish and then it moves on to the next (roughly speaking). In multi threaded processing, there still only is one GIL but it can cue up multiple "commands" that it wants to run. If a line of Python code "releases the GIL", then Python is allowed to execute other tasks while its waiting for other tasks to finish.

In multi-processing, Python launches actual multiple parallel Python processes that get copies of data and their own GIL that can execute tasks in parallel perfectly.

Thun when EMerge refers to **multi threading**, it refers to solving tasks in parallel using a single GIL. When EMerge talks about **multi processing** it talks about running multiple Python threads with each having a GIL of their own.

In FEM simulations, there are many different points where tasks could be distributed across multiple cores. This could include:

- Parameter sweeps
- **Frequency sweeps**
- **Matrix assembly**
- **Matrix factorization**
- **Post processing**

Out of these tasks, only parallel parameter sweeps are not natively supported in EMerge. One could of course manually implement this through the multi-processing module by implementing a single simulation inside a function call. All the other tasks are in some way or another distributed across multiple threads/processes.

The other tasks will be discussed now.

10.1.1 Matrix assembly

Matrix assembly is optimized using Numba. Only the assembly of the matrices are parallelized. This cannot be turned off. This means that the OS will always try to recruit as many cores as possible. Matrix assembly is usually very fast so you should not notice issues with your OS. During very demanding simulations, you might notice a hiccups during assembly. If this causes an issue for your system/application, make sure to reach out so that a solution can be found.

10.1.2 Post processing

Both the interpolation and far-field calculation functions are also optimized and parallelized using Numba and will claim your all your CPU cores if available. This also cannot be turned off.

10.1.3 Matrix Factorization

In theory, there are direct solvers that can distribute parts of the factorization process over multiple cores. For Intel machines, EMerge offers an interface to Intel's PARDISO solver that can run on multiple cores. For MacOS only there is the Accelerate solver which can function both as a single threaded (sequential) or parallel solver. Installation instructions can be found in Section 2.5.3.

Because matrix factorization is not easily parallelized, distributed processing can only use the cores so efficiently. A lot of back and forth communication, starting and halting is involved. Especially if matrices get very large, parallel direct solvers see a huge performance boost over single threaded ones. PARDISO might see significant performance boost on very large problems but the use of multiple cores makes it impossible to use further parallelization in frequency sweeps.

10.1.4 Frequency Sweeps

The easiest way to parallelize computation in EMerge is by distributing a frequency sweep across multiple cores. As mentioned before, there are two ways to do this: Multi-threaded and multi-processing. There are two single threaded direct solvers in EMerge that can be parallelized:

- SuperLU
- UMFPACK

Only SuperLU releases the GIL which means it can run as a Multi-threaded and with multi-processing. In **Multi processing** mode, one can thus use both SuperLU and UMFPACK. SuperLU is significantly slower for larger problems than UMFPACK and requires much more Memory bandwidth. Thus, for parallel sweeps of problems larger than say 20 to 30k degrees of freedom, UMFPACK is preferred. SuperLU does see very fast performance on small EM problems.

We will now discuss how to setup either a multi-threaded simulation or a multi-processing simulation.

10.1.5 How to set it up

10.1.5.1 Multi-threading

To run a multi-threaded simulation in EMerge processes, all you have to do is simply set the arguments `parallel=True` in the `run_sweep` method:

python

```python
import emerge as em

m = em.Simulation(...):
# Generate your geometry
# Setup your physics
data = m.mw.run_sweep(parallel=True, n_workers=3)
```

This will distribute your frequency sweep across multiple cores.

Important

EMerge will automatically pick **SuperLU** as the default solver for multi-threaded simulations. UMFPACK does not release the GIL and cannot be distributed using multi-threading.

10.1.5.2 Multi-processing

Multi-processing requires a bit more setup. The most important action by the user to enable multi-processing is by setting up a so called **entry point protection**. Due to the nature of how multi-processing is implemented, the main script you run is imported as module on each new Python process. Without entry point protection, each new parallel process will keep spawning more and more processes. In order to prevent this from happening EMerge has implemented a check to make sure that the main-script is run in the right

way or crash otherwise. To setup an entry-point protection, all you have to do is wrap your simulation in a `def main()` function and call that function from an `if` statement block that checks if the global `__name__` variable is `"__main__"`. Then you have to set the variable `multi_processing=True` in the frequency domain function.

python
```python
import emerge as em

def main():
  m = em.Simulation(...):
  # Generate your geometry
  # Setup your physics
  data = m.mw.run_sweep(parallel=True, n_workers=3, multi_processing=True)

if __name__ == "__main__":
    main()
```

Important
Multi-processing offers no advantage for the SuperLU solver due to process communication overhead. UMFPACK in contrast cannot run multi-threaded. Its more efficient RAM uses is better suited for distributed sweeps of larger problems.

10.1.5.3 Parallel solve of single frequencies

To solve single frequency problems in parallel, you only need to pick the right solver.

If you have an x86 CPU (Intel or AMD) you can simply use PARDISO and it will automatically solve with multipole cores. To change the number of cores either use:

python
```python
import os
os.environ["MKL_NUM_THREADS"] = "4"
import emerge as em
```

Important
It is crucial to first import os and set the environment variable before import EMerge. Secondly, the environment variables must be provided as a string.

If you use the Accelerate solver, you have to specify the correct number of threads (it default to just 1).

python
```python
import os
os.environ["VECLIB_MAXIMUM_THREADS"] = "4"
import emerge as em
```

or

python
```python
from emerge_config import config
config.set_acc_threads(4)
```

The latter can also be used to change the MKL threads:

python
```python
from emerge_config import config
config.set_pardiso_threads(4)
```

to specify a specific solver use `.set_solver()` (see Section 4.10.1).

10.1.6 Choosing the right solver

The simplest rule for picking a solver is the following:

1. Is it a small problem? Use any
2. Is it a larger problem? Use UMFPACK, or Accelerate on Apple ARM
3. If you have an x86 machine, use PARDISO

4. If you have a MacOS with ARM CPU's, use Accelerate
4. If you have a good NVidia GPU, use cuDSS

! WARNING !
When using CuDSS, make sure that you video card has sufficient VRAM for the problem you are solving. If your GPU runs short, it will generate bad results without crashing.

Now will follow a more detailed explanation of what solvers do and why and when some are better than others.

The types of linear systems that FEM solves for are so called Sparse linear systems. This means that the matrix $\boldsymbol{A}$ in $\boldsymbol{Ax} = \boldsymbol{b}$ is a sparse matrix with mostly zeros.

The final field solution is made by defining so called "basis" functions inside each tetrahedron. These basis functions are polynomials with a certain amplitude. The complex amplitude of such a polynomial is called a **degree of freedom**. Tetrahedrons in a mesh share edges and faces. These basis function amplitudes are defined on faces and edges so those degrees of freedom will be shared between tetrahedra.

If a certain EM problem has $\boldsymbol{N}$ degrees of freedom then the matrix $\boldsymbol{A}$ will be an $\boldsymbol{N} \times \boldsymbol{N}$ matrix. If certain faces and edges lie on a PEC boundary then the degree of freedom will always be 0. This means that EMerge will cut those corresponding rows and columns out of the matrix $\boldsymbol{A}$.

A separate quantity for these sparse matrices are the **number of non-zeros**, often referred to in Python as the attribute `nnz`.

If you put EMerge's logger in `DEBUG` mode:

```python
model = em.Simulation('MyName', loglevel='DEBUG)
```

Then EMerge will put more details about the solution process in the terminal. Lets look at an example for the stepped impedance filter:

```bash
0:00:23.128395  DEBUG  : Number of tets: 18,891
0:00:23.128395  DEBUG  : Number of DoF: 134,962
0:00:23.128395  DEBUG  : Number of non-zero: 5,404,796
0:00:23.143490  INFO   : Starting single threaded solve of 8 jobs.
0:00:23.242141  DEBUG  : [ID=1] Removed 35098 prescribed DOFs (99,864 left, 3,356,832≠0)
0:00:23.242141  INFO   : [ID=1] Calling Pardiso Solver
0:00:23.242141  DEBUG  : [ID=1] Executing symbollic factorization.
0:00:24.651799  INFO   : [ID=1] Elapsed time taken: 1.410 seconds
0:00:24.651799  DEBUG  : [ID=1] O(N²) performance = 7074.634 MDoF/s
0:00:24.652921  DEBUG  : [ID=1] Solver complete!
0:00:35.226219  DEBUG  : [ID=1] Removed 35098 prescribed DOFs (99,864 left, 3,356,832≠0)
0:00:35.226219  INFO   : [ID=1] Calling Pardiso Solver
0:00:35.303978  INFO   : [ID=1] Elapsed time taken: 0.078 seconds
0:00:35.303978  DEBUG  : [ID=1] O(N²) performance = 128250.435 MDoF/s
0:00:35.303978  DEBUG  : [ID=1] Solver complete!
```

Important
The log prompt is from on a older EMerge version.

We can see that this geometry contains 18,891 tetrahedra with about 135 thousand degrees of freedom. This amounts to about 7.5 unique degrees of freedom per tetrahedra. Each tetrahedra contains 20 basis functions so you can see that the sharing of basis function reduces the total number by about a factor of 3. On the next line you see that the resultant matrix contains about 5 million non-zeros. This means that about 0.03% of the total matrix is filled with entries.

Next you see that EMerge starts a single threaded solve. This actually means just one frequency at the same time. The first line states that it removed about 35,000 degrees of freedom leaving about 100,000 left. These are the PEC degrees of freedom. Next it calls PARDISO and then shows that PARIDISO solved the problem with a performance of 7074 MDoF/s. What does this mean?

In general it is hard to define a fair metric to express the performance of a sparse solver. Clearly when one has more degrees of freedom it will take longer to solve but by how much? There is actually not a clear answer to this. Two matrices with the exact same number of degrees of freedom and the exact same number of non-zeros may take a different amount of time to solve depending on the adjacency structure/sparsity pattern and the matrix values. Elongated problems like a filter in this case take less time to solve than round problems like antennas in a sphere. We will discuss this a bit more later.

To measure the performance of a direct solver, EMerge measures the total number of matrix entries ($N \times N$) and then divides that by the total solution time. This yields te result of 7000 MDoF/s ($\mathbf{MDoF}^2$/s to be technically correct). This is not a fair way to compute performance because a dense matrix with a lot of non-zeros would have the same problem size ($N \times N$) but a vastly different performance. One could also measure the performance in terms of the number of non-zeros solved per second. Again, this is not fair because a dense set of N non-zeros would solve much slower than a very sparse matrix. In general, larger problems become more and more sparse. This can mean that they can be solved more efficiently (relatively speaking) but this is also not true. LU decomposition often does not keep the same sparsity ratio and adds more “fill-in” (non zero terms) which means that the time complexity of larger EM problems scales somewhere between $\mathcal{O}(N)$ and $\mathcal{O}(N^2)$.

Thus an exact fair comparison metric is hard to formulate. Of course, within each problem, different solvers may be compared fairly to see which is faster. But there is no absolute metric possible such that some solver will always achieve the same “score”. To do this, one would need some measure of the difficulty to solve that problem and these actually also depend on the physics of the specific simulation.

Lastly you also notice that the second solve of ID=1 is much faster, over 128,000 MDoF/s! What is going on here? Well solving linear systems with direct solvers involves three steps roughly speaking:

1. Symbolic Factorization
2. Numeric Factorization
3. Solve

The first step executes an analysis of the sparsity pattern of the matrix which reorders the rows and columns to maximize the performance of the solve process. The second step actually executes the factorization into a lower and upper-diagonal matrix called LU-decomposition. Then once the solver has the LU decomposition, it can solve for $\boldsymbol{Ax = b}$. The final solve step takes the least amount of time. Most of it is done during the first and second step.

When one executes a multi-port sweep, only the vector $\boldsymbol{b}$ in $\boldsymbol{Ax = b}$ changes. This means that the same LU decomposition can be reused. This explains the immense performance boost of the second solve and direct solvers in general.

The UMFPACK, PARDISO and cuDSS solver can also reuse the symbolic factorization of step 1. During a frequency sweep, the sparsity structure of a matrix remains the same. Only the values on each non-zero. So this factorization can also be reused. This is not possible with SuperLU.

We can see this when we look at the performance of each solver on the stepped impedance filter problem.

- **SuperLU(Single Threaded)**
  - 4,900 MDoF/s

- **UMFPACK(Single Threaded)**
  - 4,400 MDoF/s (Initial)
  - 6,300 MDoF/s (Subsequent)
- **PARDISO(Parallel)**
  - 7,500 MDoF/s (Initial)
  - 11,500 MDoF/s (Subsequent)
- **cuDSS(GPU)**
  - 7,500 MDoF/s (Initial)
  - 18,000 MDoF/s (Subsequent)

We see quite a significant speedup for subsequent solves. PARDISO and cuDSS profit more off of this than UMFPACK. This is because numeric factorization is easier to parallelize than symbolic factorization. We can also run this same problem with SuperLU and UMFPACK in 4 parallel processes:

- **SuperLU(Multi Threaded)**
  - 3,500 MDoF/s (14,000 MDoF/s effectively)
- **UMFPACK(Multi Threaded)**
  - 3,200 MDoF/s Initial - 14,800 MDoF/s effective
  - 4,400 MDoF/s Initial - 17,600 MDoF/s effective (subsequent)

We see that the performance of SuperLU and UMFPACK degraded but, as we are solving 4 frequencies in parallel, we have to multiply the performance by the number of parallel solves to be fair. Thus we see a fairly significant speedup over PARDISO.

Now lets look at what happens when we significantly reduce the total size of the problem to 30,000 total degrees of freedom:

- **SuperLU(Distributed 4 threads)**
  - 1,500 MDoF/s (6,000 MDoF/s effective)
- **SuperLU(Distributed 8 threads)**
  - 1,100 MDoF/s (8,800 MDoF/s effective)
- **UMFPACK(Distributed 4 threads)**
  - 1,500 MDoF/s Initial - 6,000 MDoF/s effective
  - 1,900 MDoF/s Initial - 7,600 MDoF/s effective
- **PARDISO(Parallel)**
  - 4,000 MDoF/s
- **cuDSS (GPU)**
  - 9,000 MDoF/s

We see that SuperLU with 8 parallel threads can see a significant performance boost. The total simulation time of this sweep was 10.16 seconds. For cuDSS this is 11.93 seconds.

Now lets look at a version of the Patch antenna example (older one) with 63,000 Degrees of Freedom

- **SuperLU (Single)**
  - 645 MDoF/s
- **UMFPACK (Single)**
  - 1,900 MDoF/s
  - 2,600 MDoF/s (subsequent)
- **SuperLU(Distributed 4 threads)**
  - 500 MDoF/s (2,000 MDoF/s effectively)
- **UMFPACK(Distributed 4 proc.)**
  - 1,300 MDoF/s single - 5,200 MDoF/s effective
- **PARDISO(Parallel)**
  - 2,700 MDoF/s (Initial)
  - 3,500 MDoF/s (Subsequent)
- **cuDSS (GPU)**
  - 3,100 MDoF/s (Initial)

- 6,000 MDoF/s (Subsequent)

Now we see a slightly different story unfold. SuperLU takes a big performance penalty with this less elongated topology where UMFPACK remains fairly consistent. Its 4 core performance is now on par with cuDSS. The story changes even more if we make the problem much larger. We will do the following comparison on a single thread only with a 91,000 DoF problem

- **SuperLU(Single Threaded)**
  - 400 MDoF/s
- **UMFPACK(Single Threaded)**
  - 1,900 MDoF/s (Initial)
  - 2,300 MDoF/s (Subsequent)
- **UMFPACK(Distributed 2 proc.)**
  - 1,900 MDoF/s - 3,800 Effective
  - 2,100 MDoF/s - 4,200 Effective
- **UMFPACK(Distributed 4 proc.)**
  - 1,400 MDoF/s - 5,600 Effective
  - 1,500 MDoF/s - 6,000 Effective
- **PARDISO(Parallel)**
  - 3,000 MDoF/s (Initial)
  - 4,000 MDoF/s (Subsequent)
- **cuDSS(GPU)**
  - 3,400 MDoF/s (Initial)
  - 5,900 MDoF/s (Subsequent)

To summarize, we see that depending on the problem and problem size, solver behave very differently. Particularly in cases of small problems below 30k DoF, SuperLU can be very efficient. However, due to the way the algorithm is implemented, it takes a big performance penalty for large problems with certain sparsity patterns such as patch antennas. You may imagine standing on a central node in the mesh. On each step you look at all connected edges and add the connected nodes. You iterate this step by step until you passed all nodes in your mesh. If the total number of nodes grows with the number of steps cubed, you have a really spherical problem. For long problems like waveguides and filters, the number of nodes at some point grows constant for each step you take. In those scenarios, SuperLU is fast.

10.1.7 RAM Management

By default, when you set `parallel=True`, all matrices for each frequency point will be pre-assembled and stored in the RAM memory. A rule of thumb is that every `100k DoF` yield a Sparse matrix occupying approximately `100MB` of RAM memory.

$$M_{RAM}(MB) \approx N_{DOF} \ / \ 1000$$

Therefore, 100k DoF with a 100 point frequency sweep will take up 10GB of RAM. Especially with large sweeps and larger systems this can quickly clog your RAM memory. There are two ways in which you can prevent this from happening. First the `frequency_groups` optional argument can be used to specify exactly how many frequency steps you want to pre-assemble before solving. Setting this as a whole integer multiple of the number of threads used will solve your problem most efficiently.

```python
data = m.mw.run_sweep(parallel=True, n_workers=3, frequency_groups=6)
```

This can be visualized as following:

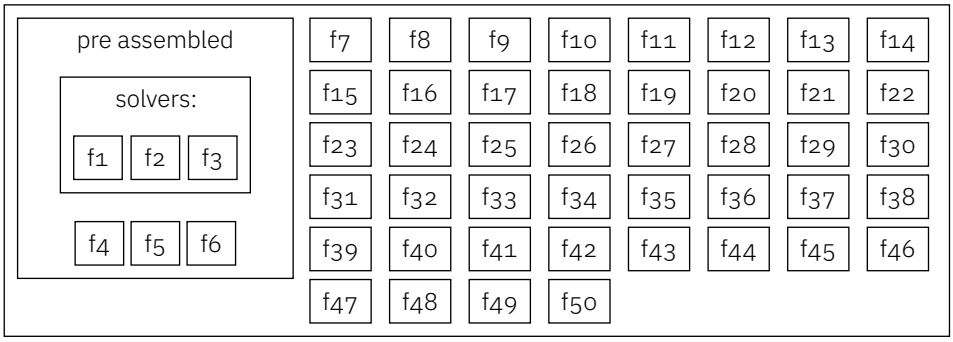


Figure 62: A total of 50 frequency points are solved by pre-assembling 6 frequencies. Three of them are solved by parallel threads.

10.1.7.1 Harddrive caching

Alternatively, if the systems get very large, you can cache them on the hard-drive. This incurs an IO penalty and is only advised if you work with large problems. In this case you can set `cache_harddisk` to `True` and optionally specify a `harddisc_path`.The path will always be a new path specified by `harddisc_path` relative to the Python script. If caching is set to true, the assembler will write the CSR matrices to the harddrive after assembly and then remove them from RAM memory. Then when solving, it will load them from the harddrive again. Once a problem is solved, it will then again write the solution to the harddrive. Finally before post processing, all solutions will be loaded and processed. Afterwards it will automatically cleanup all the cached files. If the path `harddisc_path` is empty afterwards, it will also remove the newly created path.

11 Troubleshooting and FAQ

Here you will find some common problems and the solutions. This list is very likely incomplete and will be extended in the future. If any question is left unanswered feel free to reach out at:

- `bugs@emerge-software.com`

Or post your problems on GitHub:

- `https://github.com/FennisRobert/EMerge`

Or join our discord (link is available on GitHub).

11.1 Common Issues and Solutions

11.1.1 EMerge debug messages

Starting from 1.1.0 onwards, EMerge will maintain its own internal troubleshooting log. Sometimes it will detect situations that can possible cause problems. Because these are heuristics, EMerge will not crash. However, errors will be kept and printed at the end of a simulation.

11.1.2 Common issues

EMerge is stuck importing/frozen when solving

EMerge uses Numba to compile its high performing functions such as those associated with matrix assembly and field interpolation. Each time you run a fresh install of EMerge, it has to compile these Numba functions **once**. After this the functions are cached. Not all functions are imported when you import EMerge. For instance, the matrix assembly scripts are only imported (and thus compiled) when you run your simulation for the first time. Compilation can take up to a minute depending on the machine. After running it once, you should not have any more problems.

Solving runs slower on more cores

If your problem is relatively large or your computer relatively slow, more threads/processes might not always equal more performance. In general the solution speed does not scale linearly with the number of cores. Try reducing the total number of parallel threads.

My computer runs out of memory during solving

This might have two causes. In the simplest case, the problem might simply be too big for your computer to Solve.

It is also possible that you are pre-assembling too many matrices. By default, EMerge will pre-assemble all matrices before starting a frequency sweep. You can decrease the number of frequencies to pre-assemble bo decreasing the `frequency_groups` argument in the frequency domain solver. Make it an integer multiple of the number of jobs. The one taking the least amount of memory would be:

python

```
n_jobs = 2
> Fill in whatever number of jobs your machine can handle
data = my_model.mw.run_sweep(parallel=True, n_workers=n_jobs,
frequency_groups=n_jobs)
```

Animation windows are pushed behind other windows after several seconds (MacOS)

This is default MacOS behavior. To fix this do the following steps:

1. Move your mouse to the top left corner of your screen, click the Apple icon and open System Settings
2. Scroll down and click on “Desktop & Dock”
3. Scroll all the way to the bottom “Mission Control” section and and deactivate the switch “When switching to an application, switch to a Space with open windows for the application.”

11.2 Frequency Asked Questions (FAQ)

to come

Index

.

.globals attribute 52

3

3D Far-field plots 85
3D plot functions 82
3D plot generation 81
3D Plots 76

A

Adaptive Mesh Refinement 93
Add 29
AMR 93
Anchor points 36
Automatic open regions 65

B

BaseDataset 54
Boolean 29
Boundary condition 48
Boundary Condition
- AbsorbingBoundary 64
- Floquet 101
- Lumped Port 61
- Modal Port 58
- ScatteredField 66
- Surface Impedance 63
- Thin Conductor 63
- User Defined Port 61

Box 28

C

Choosing a Solver 49
Class
- Curve 103
- EHDataFF 84
- HexCell 101
- PeriodicCell 101
- RectCell 101
- Settings 26
- XYPolygon 102

Class
- Axes 28
- CoordinateSystem 28
- PCB 96
- PCBNew 96
- Plane 28
- Selector 32

Color Themes 78
Connecting faces 104
Cylinder 28

D

Distributed sweep 117

E

Entry point protection 118
Export
- DXF 87
- Farfield Data 88
- STEP 87
- Touchstone 87
- VTK 87

Extending EMerge
- Custom Geometry 108
- Custom Solvers 112

Extrusion 102

F

Face labels 32
Far-field calculation 84
FEMBasis 75
Finding Data 54
Floquet 101
Function
- Select 34

Functions
- .view 77

G

Geometry 28
Geometry faces 32
Ground plane 98

H

Hard drive caching 123
Horn 28

I

Import
- .STEP 40

Install 11
Installation 11
Intersect 29

L

Loading files 49
Loft 104

M

Material 43
- Custom 44
- Priority 43

Measure in plots 76
Mesh settings 46
Mesher 46
Mirror 30
Modal analysis 60
Multi-processing 117
Multi-threading 117
MWData class 54

N

Notifications 90
NTFY 90

O

Open Boundary 64
Operation 29
Optimization 91

P

Parallel computing 116
Parallel Processing 117
Parameter sweep 55
PCB Ports 99
Periodic 101
Pipe 103
Plane 28
PML 64
Polygon 102
Port
  Differential vs Common Mode 62
  Impedance Definition 60
  Lumped Port 61
  Mode alignment 60
  Multi-mode sweep 62
Ports 58
PyVista 76

R

RAM Management 122
Relative Field 68
Remove 29
Revolution 102
Rotate 30

S

Saving files 49
Scattered Field 66
Select
  Face labels 32
  Face names 34
  Multiple 34
  Selector Class 32
  Visual (GUI) 35
Selection 31
Simulation Data 52
SimulationDataset 52
Smartphone notification, NTFY 90
SolveRoutine 4, 50
Solvers 4, 50
Sphere 28
Spherical axis definitions 86
STEP file
  Export 87
  Import 40
Subtract 29
SuperLU 4, 50
Symmetry planes 85, 95

T

Transform 30
Translate 30

V

Via 99

Appendix

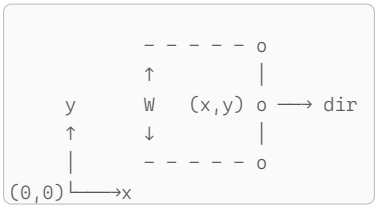
A. Classes and functions

A.1. PCB methods

This appendix will show in more detail how specific operations of the PCB class are defined.

A.1.1. Stripline segments

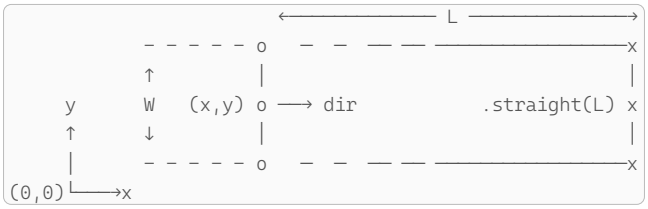
At each point in the builder process the PCB router stores an X,Y coordinate, a width and a direction:



Listing 20: The geometry of a StripLine segment

A.1.2. Straight sections

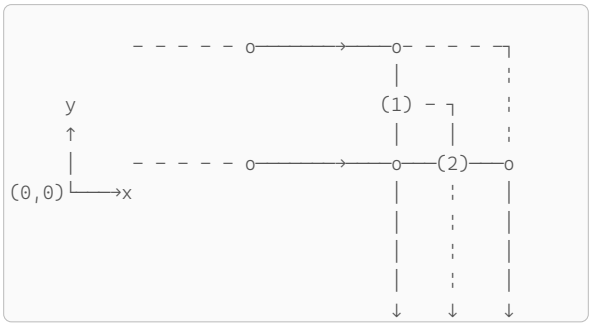
A straight section simply displaces a line in the current direction and moves the current state to a new coordinate.



Listing 21: The current router position at (x,y) with some width W is extended by a distance L using the `.straight(L) method.`

A.1.3. Turn

A turn is made in place but does not stay at the same coordinate. The turn pivots around the inside corner of a stripline. This is illustarted below. There are two corner types in EMerge, the “square” corner shown below and the default “chamfer” which takes a 70 degree half angle cutoff. Important is to know that a turn call makes a jump from (1) to (2) in the image below.



Listing 22: A square cornered 90 degree turn

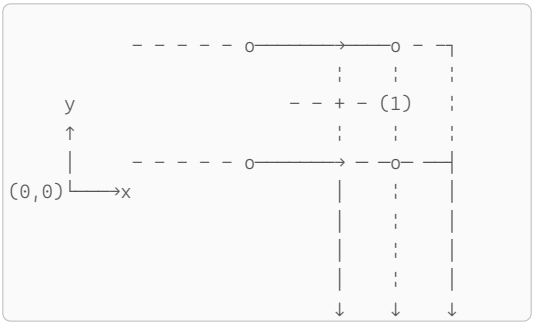
A.1.4. PTurn

A pointwise turn is made in the same way as a normal turn but such that point (1) and point (2) are on the same place by moving back half a stripline width before making the normal turn. This function is intended for routing where total phase length is important.

A normal trace adds a total amount of line length equal to the sum of half of the start and half of the departure widts (distance from (1) to (2) in the normal turn). Therefore the total length of the followith path:

```python
pcb.new(0,0,width,(1,0).straight(5).turn(90).straight(5)
```

Is equal to 5 + width + 5.
Thus the Pturn, removes this total amount of added with by performing this in place turn.

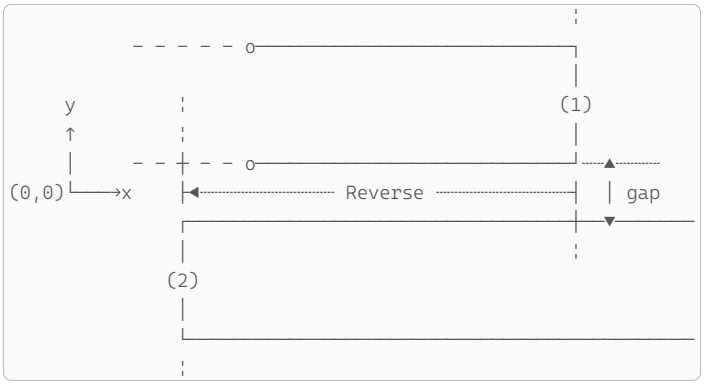


Listing 23: A square cornered 90 degree turn

A.1.5. Jump

The Jump call is essentially a shorthand for terminating the current stripline and starting a new one at some displacement. There are two ways to specify how much a jump would displace the stripline:

- Absolute coordinates: (dx, dy) Specifies how much the stripline moves in absolute x/y coordinates.
- Relative movement: (gap, reverse, side) For a sideways jump. Gap defines how much space is left in between the next line and reverse how much the line backs up from the current position. This is illustrated below



Listing 24: A jump call moving some `gap` distance to the right and reversing by some amount defined by `reverse`. The side may either be “right” or “left”.

A.1.6. Curve

The curve command makes a turn with some rotation radius as seen from the center line from the stripline to the left or right. The total rotation angle is specified in degrees. The Turn is always done in discrete straight steps. The discretization angle can be specified and is 10 degrees by default.

A.1.7. To

The `.to()` method allows the user to find a routing to a final destination. The routing algorithm is simple.

1. Take the final destination and arrival direction and a defined “backoff” distance. This is the minimum length of straight section at which it ought to arrive at the destination.
2. Find a rotation direction from the current position to intercept the arrival direction line. The direction is discretized by some amount. If a 90 degree step is defined it will only make straight turns.
3. Rotate the current position to intercept the final arrival point, move to it, make the final rotation into the final direction and more to the destination.

A.1.8. Split and merge

The split command makes a recording of the current location and proceeds in a different direction (defined by the user). The user may proceed from this point. Then, the user can return to the last known split coordinate by calling the .merge() method.

A.1.9. Cut

Cut terminates the current stripline and immediately starts a new one. It can be used to split up a path. This is required if a path is meant to be circular. Currently, the router does not support circular paths because the resultant polygon would have a hole in it. To make a strip path end where it started, cut the path somewhere in between so that it consists of two parts.